



(19) **United States**

(12) **Patent Application Publication**
KUMAR et al.

(10) **Pub. No.: US 2025/0272398 A1**

(43) **Pub. Date: Aug. 28, 2025**

(54) **RANSOMWARE DETECTION ACCURACY
BASED ON MACHINE LEARNING
ANALYSIS OF FILENAME EXTENSION
PATTERNS**

(71) Applicant: **Commvault Systems, Inc.**, Tinton
Falls, NJ (US)

(72) Inventors: **Sanath KUMAR**, Woodbridge, NJ
(US); **Adwait Vinay THATTEY**,
Boisar (IN); **Arun Prasad
AMARENDRAN**, Manalapan, NJ (US);
Brian F. BROCKWAY, Shrewsbury,
NJ (US)

(73) Assignee: **Commvault Systems, Inc.**, Tinton
Falls, NJ (US)

(21) Appl. No.: **18/586,618**

(22) Filed: **Feb. 26, 2024**

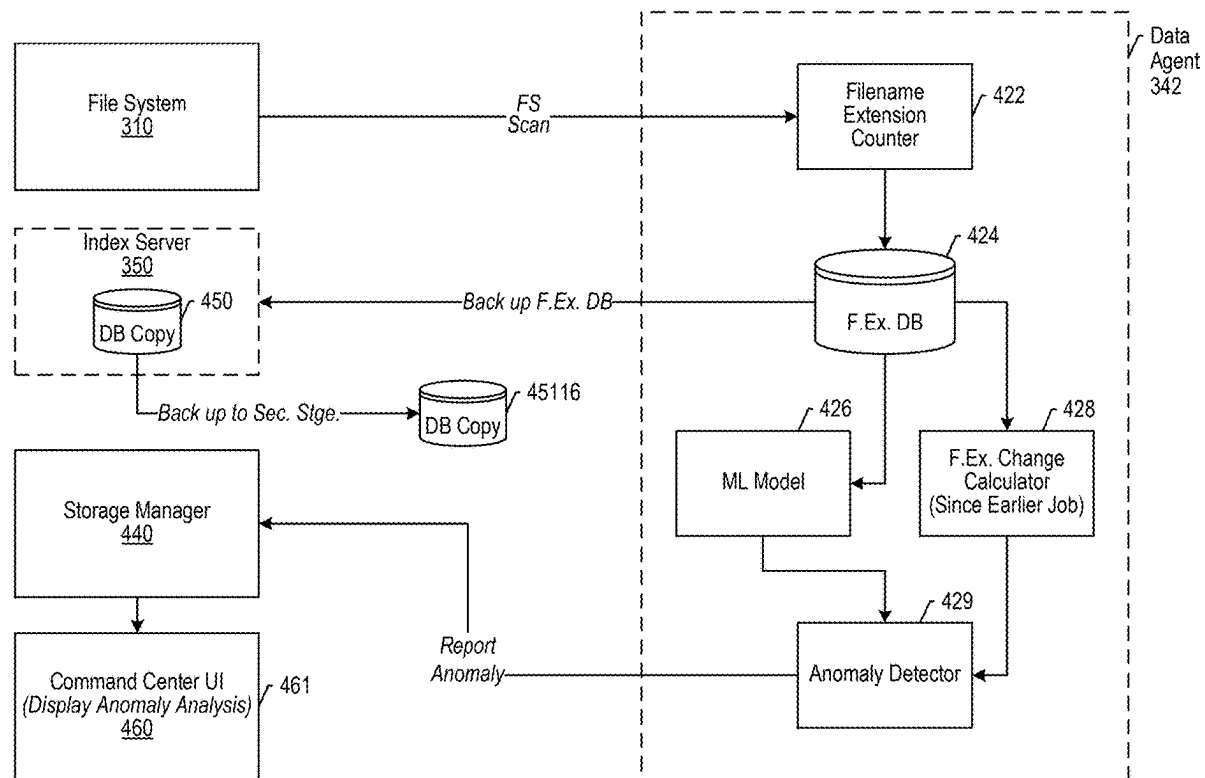
Publication Classification

(51) **Int. Cl.**
G06F 21/56 (2013.01)
G06F 11/14 (2006.01)

(52) **U.S. Cl.**
CPC **G06F 21/565** (2013.01); **G06F 11/1458**
(2013.01); **G06F 2201/80** (2013.01); **G06F**
2221/034 (2013.01)

(57) **ABSTRACT**

Ransomware detection accuracy is improved by analyzing patterns of changes in filename extension counts, relative to each other, that occur in a file system over time. The disclosed approach is malware-agnostic and does not rely on recognizing malware extensions or on real-time monitoring of the target file system. Instead, during each successive backup job of the target file system, the disclosed technology counts different types of filename extensions and compares the counts to each other and to corresponding counts taken in earlier backup jobs. Preferably, the anomaly detection analysis uses machine learning to discern a behavior pattern of the file system, which indicates how filename extensions are distributed and how much they change between backup jobs over time.



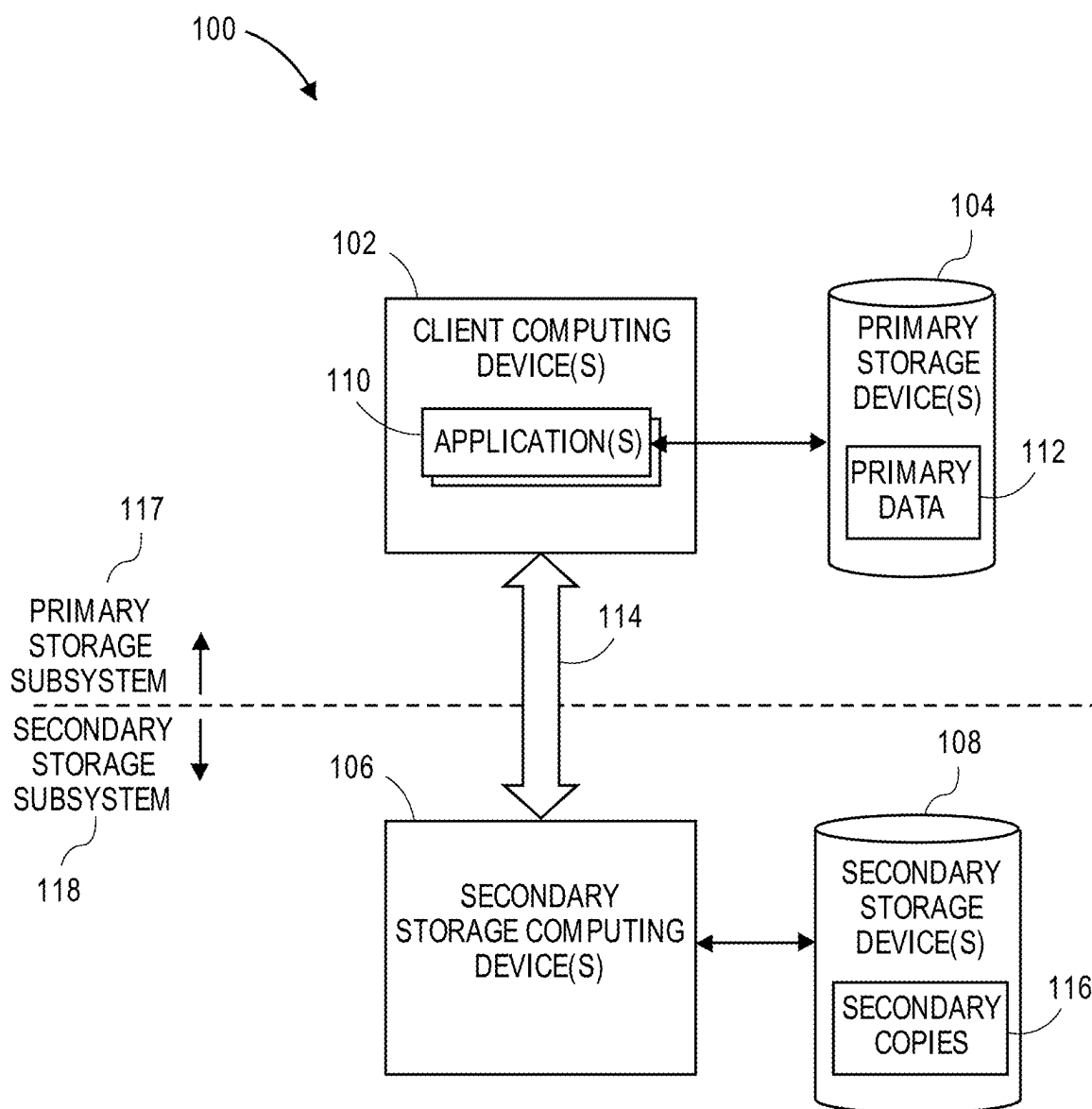
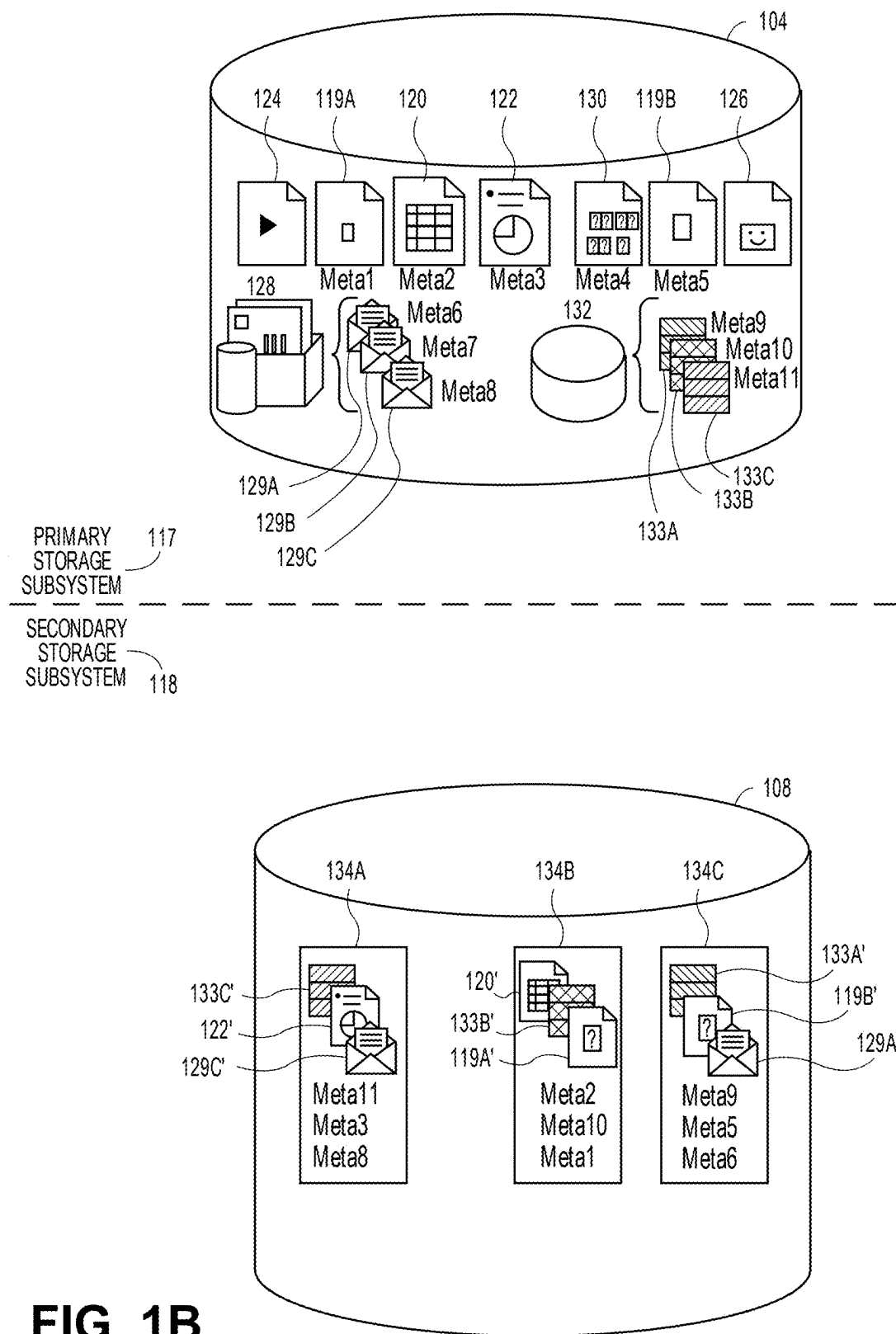


FIG. 1A



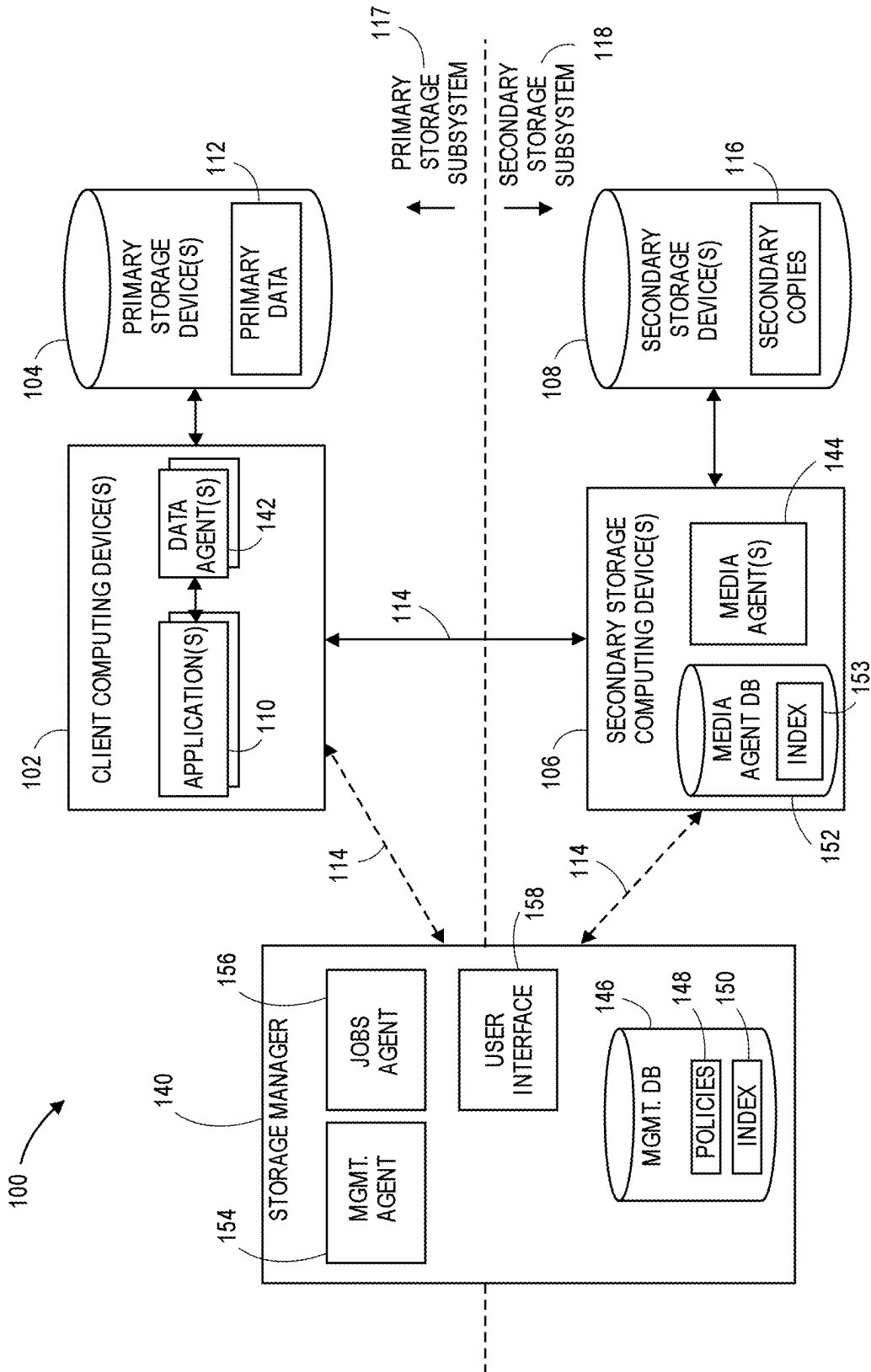


FIG. 1C

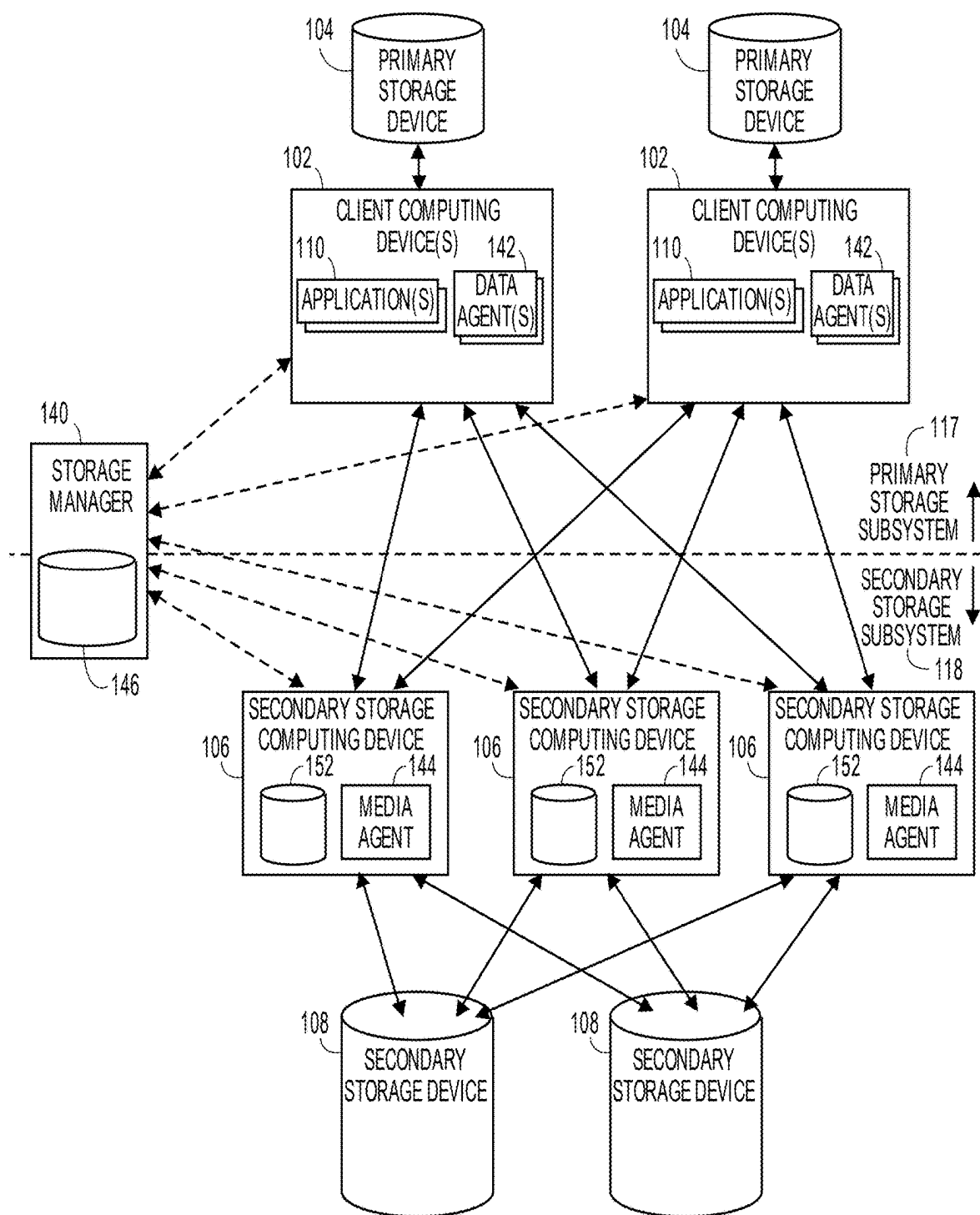


FIG. 1D

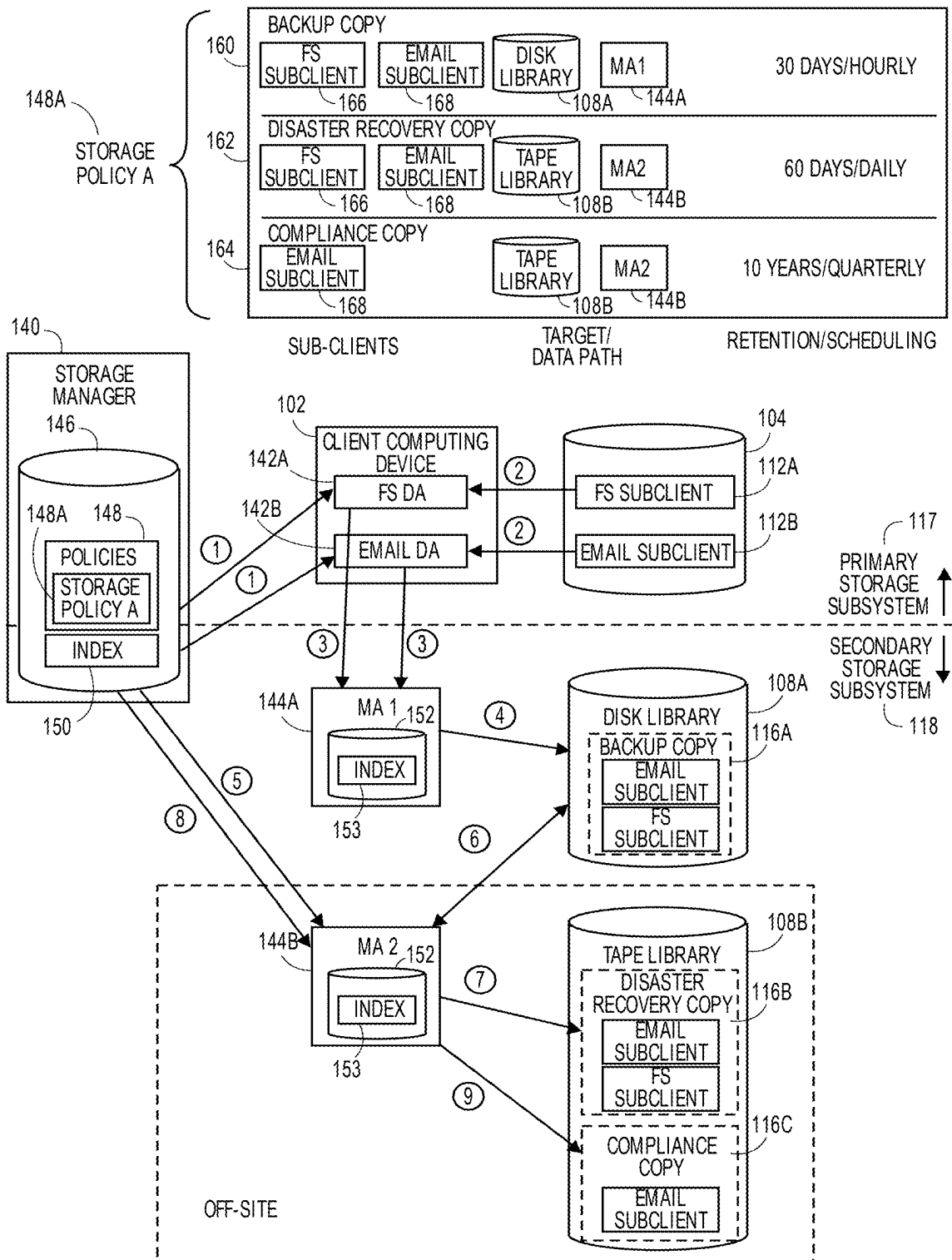


FIG. 1E

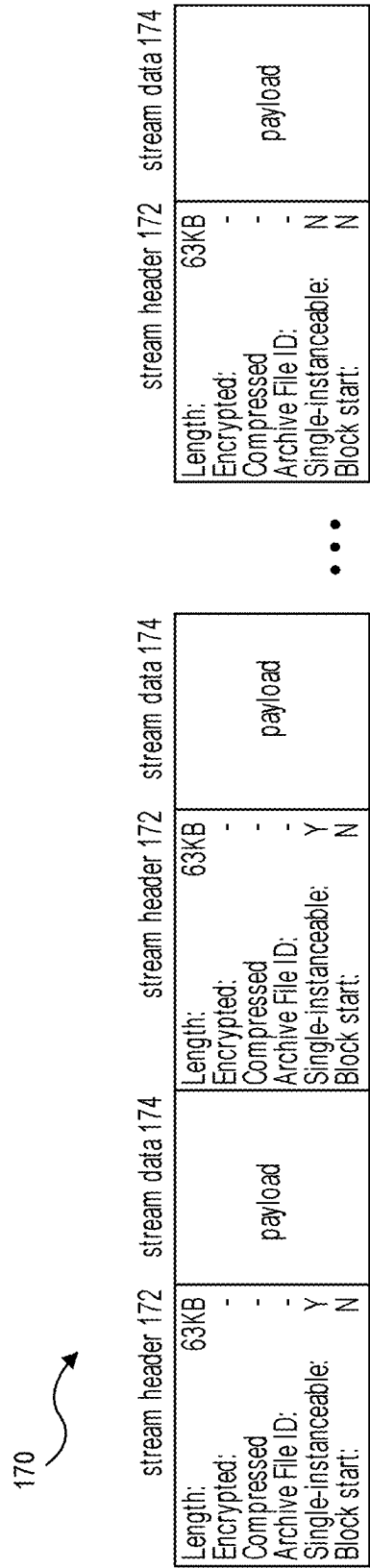


FIG. 1F

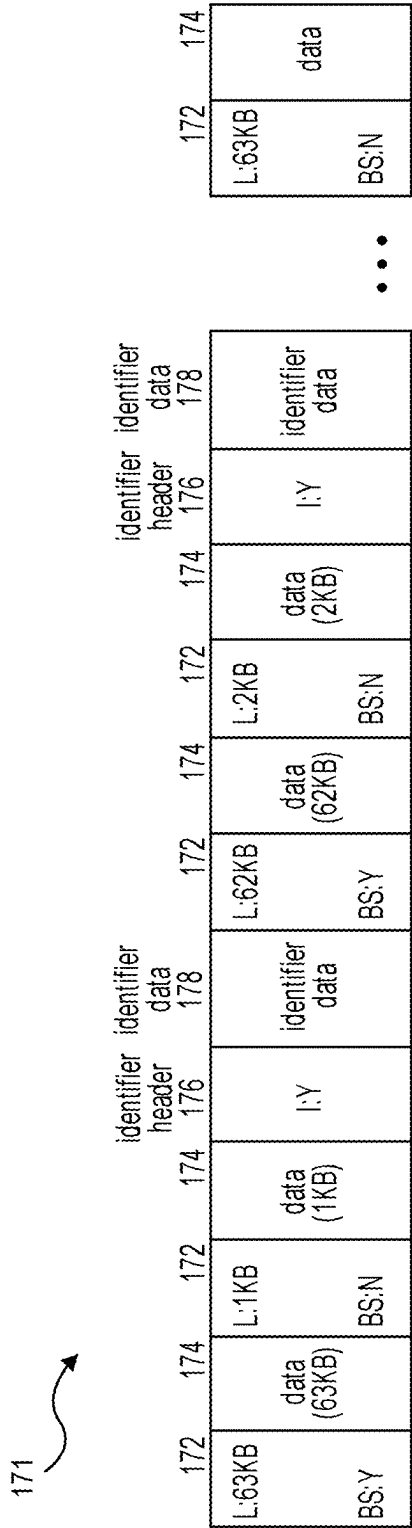


FIG. 1G

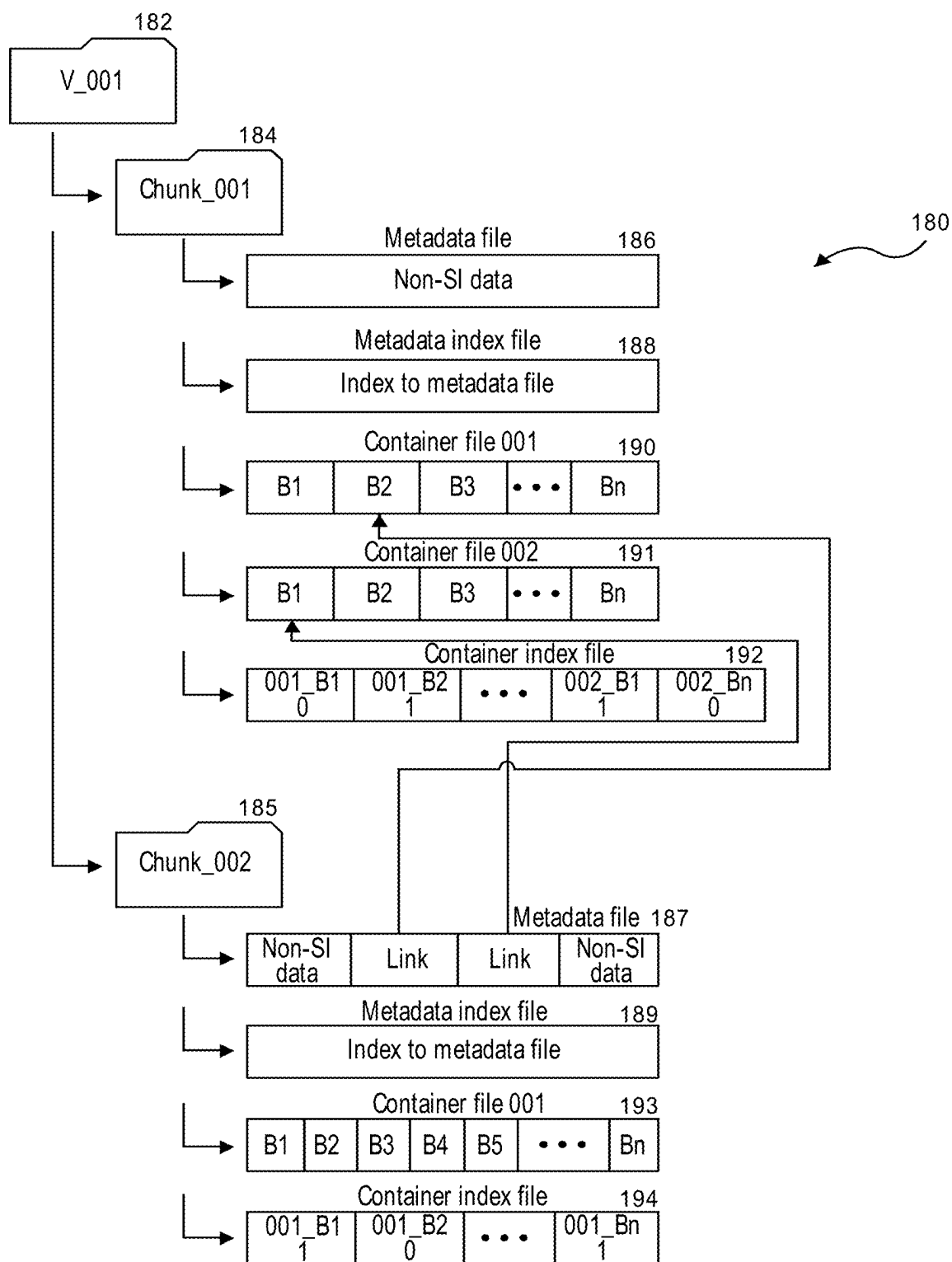


FIG. 1H

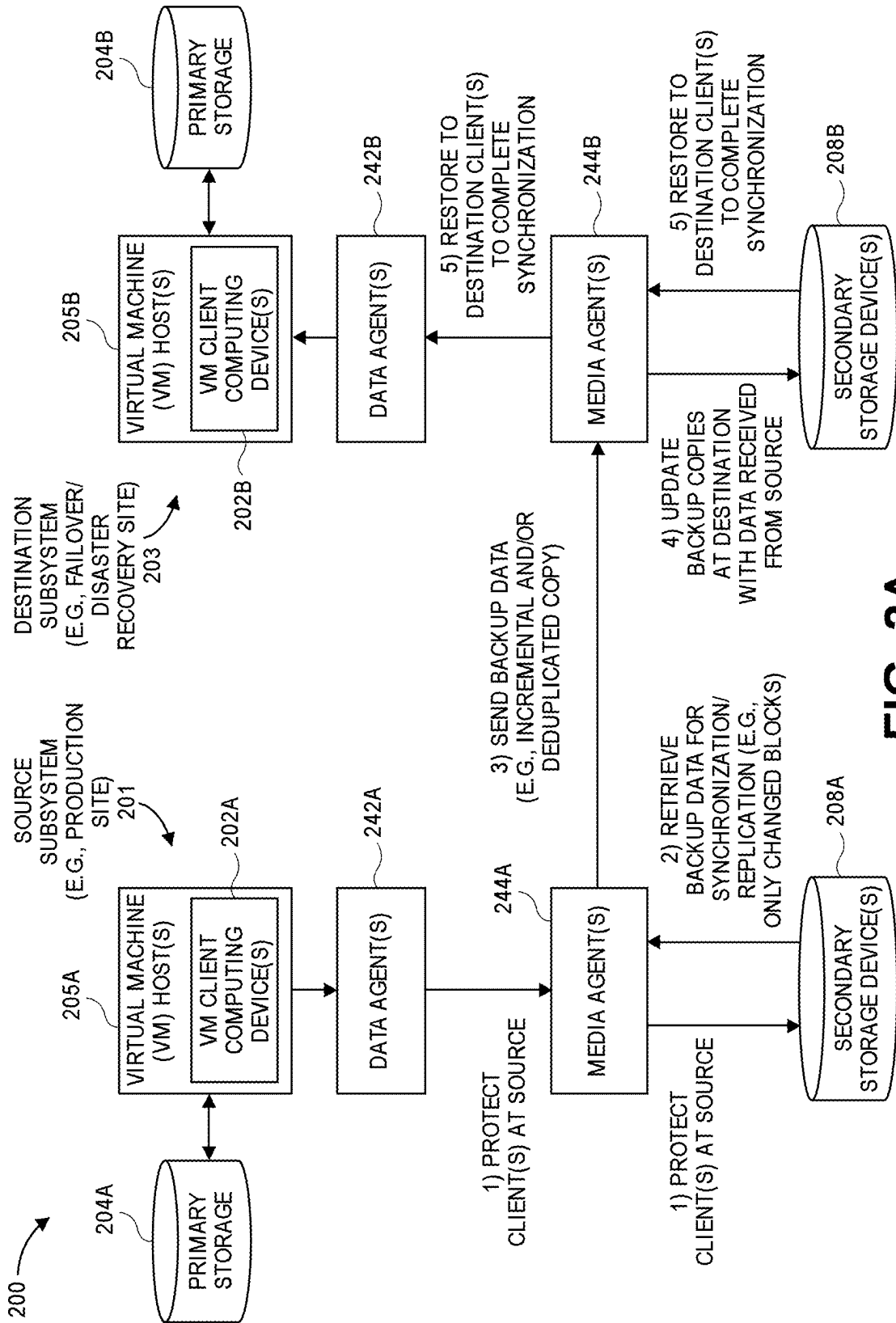


FIG. 2A

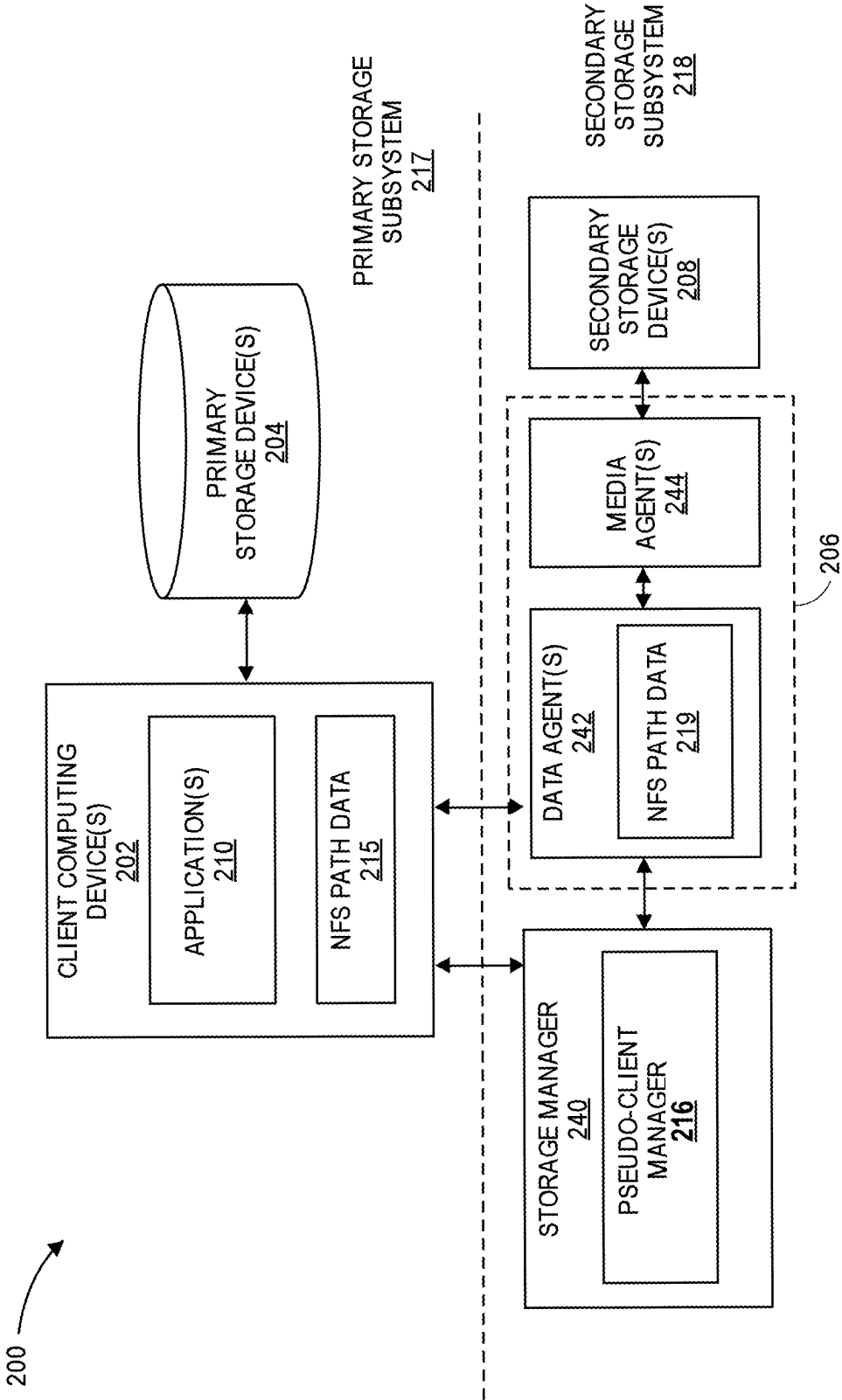


FIG. 2B

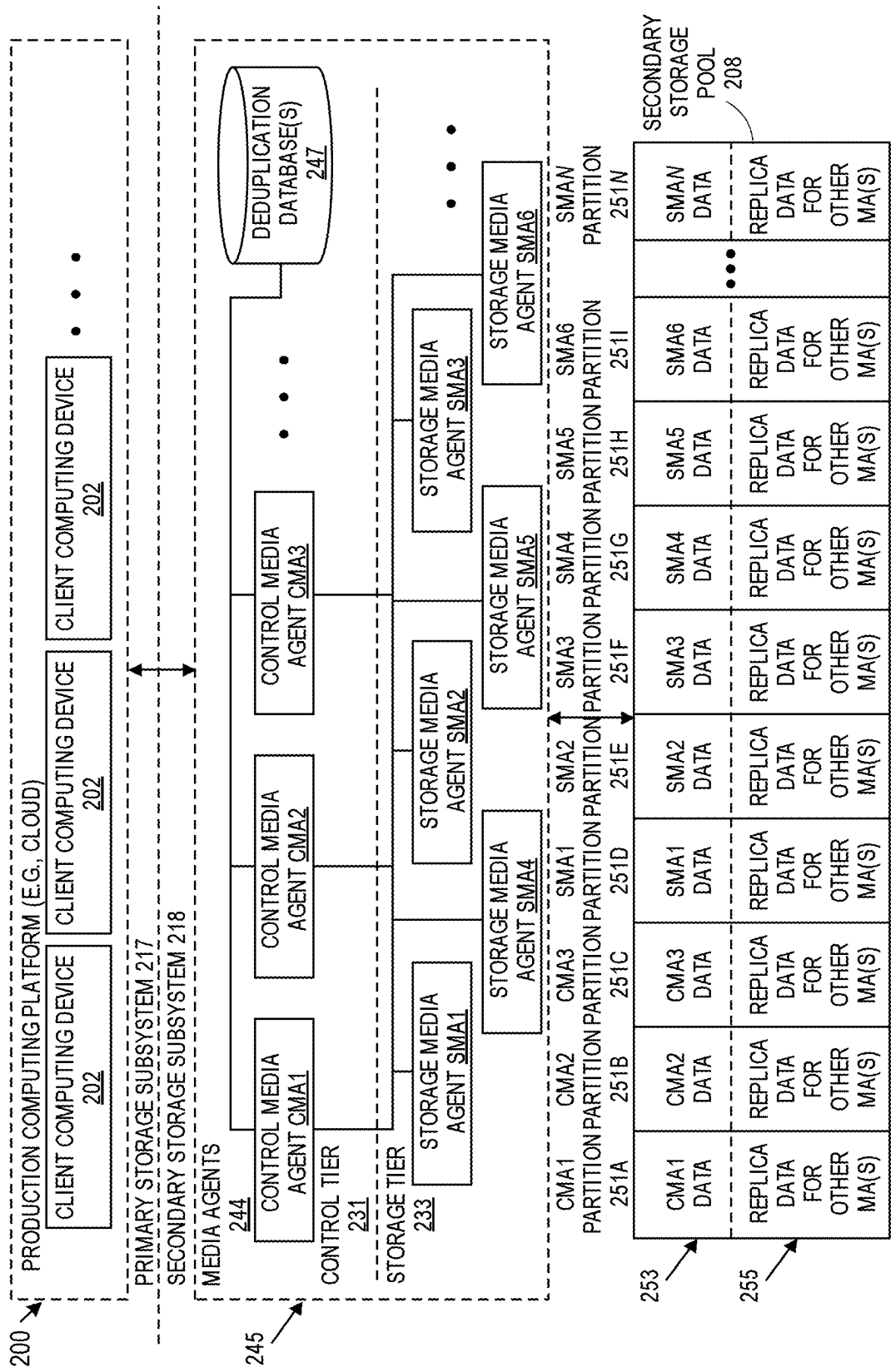


FIG. 2C

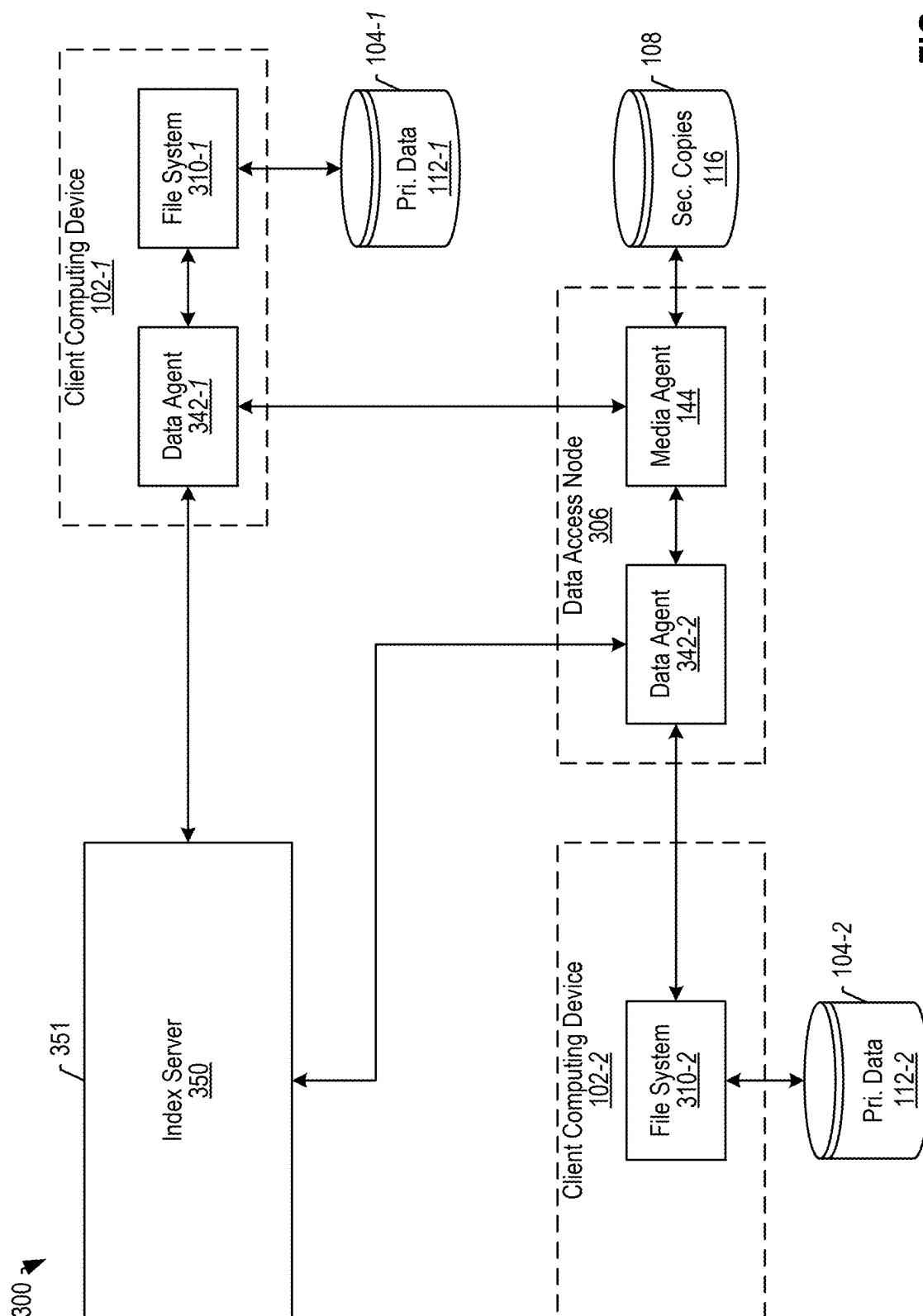


FIG. 3

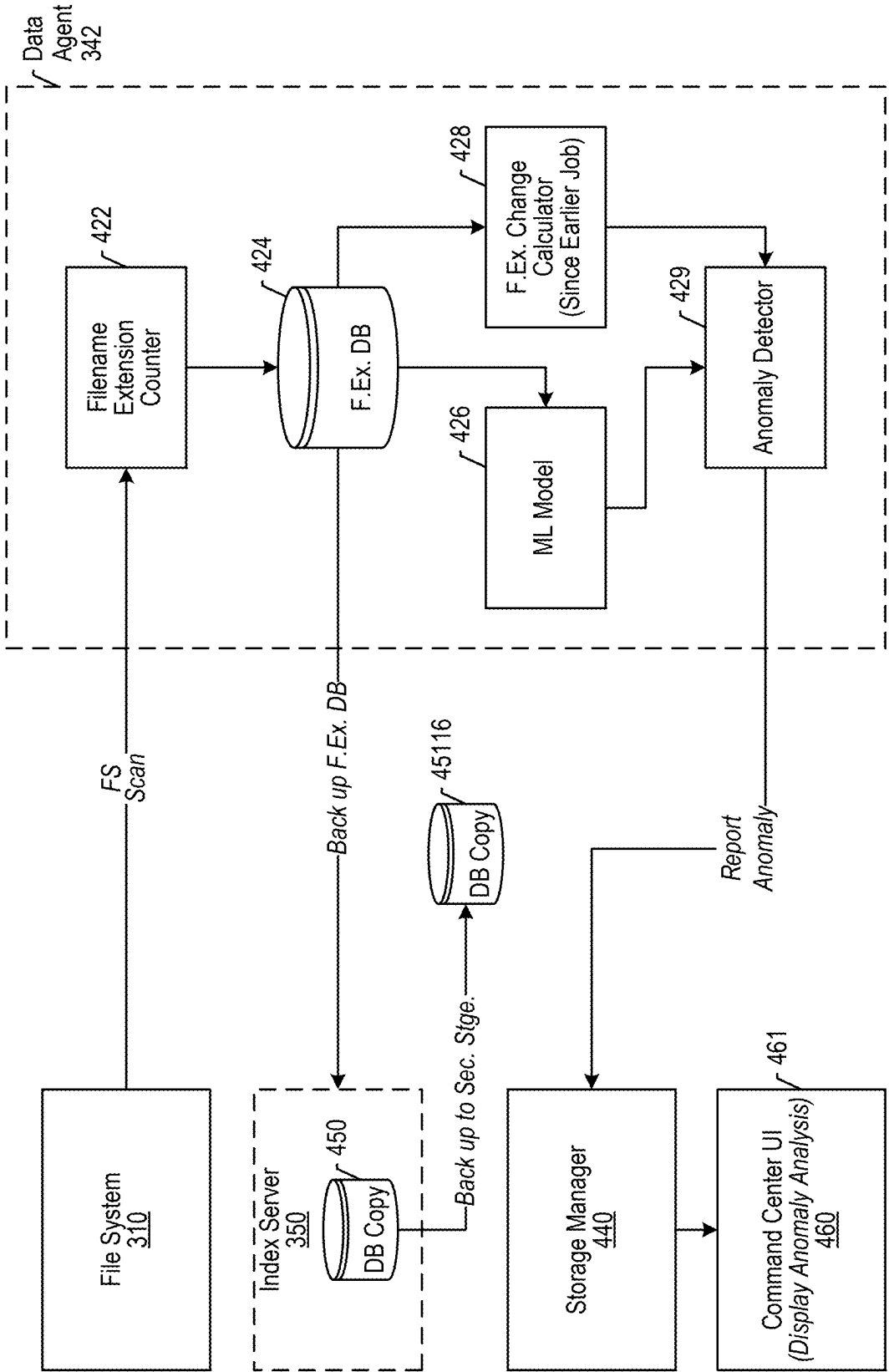


FIG. 4

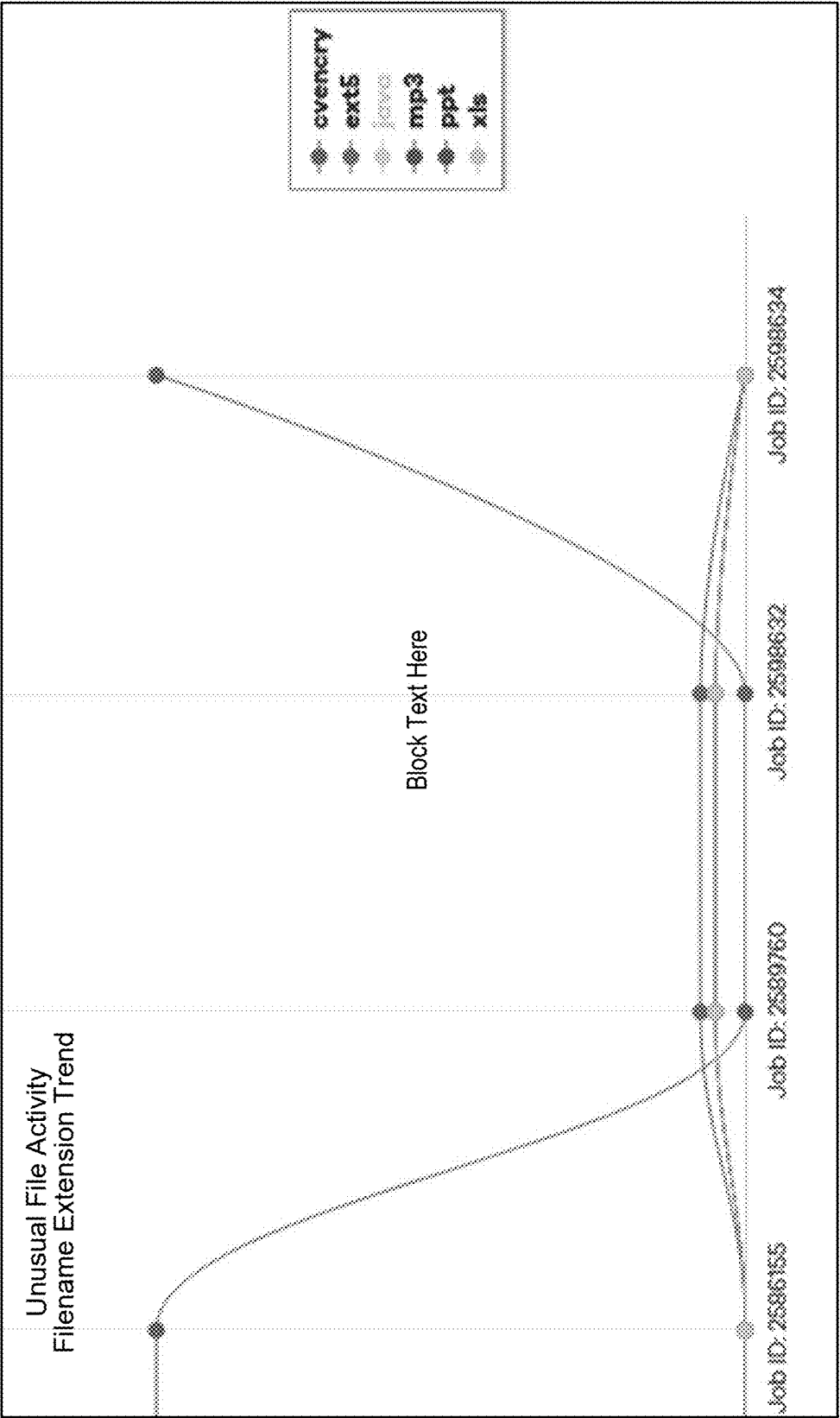


FIG. 4A

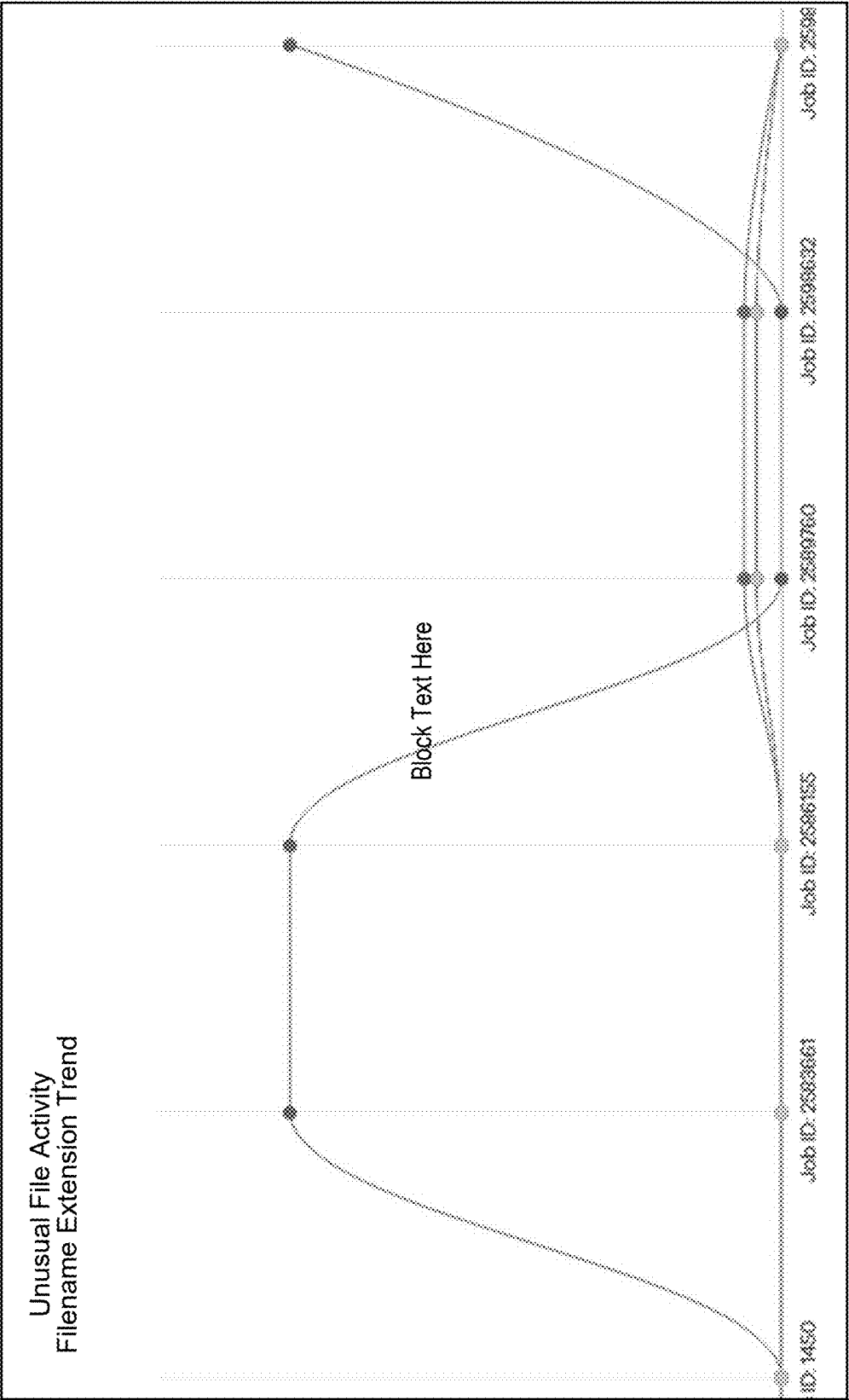


FIG. 4AA

500

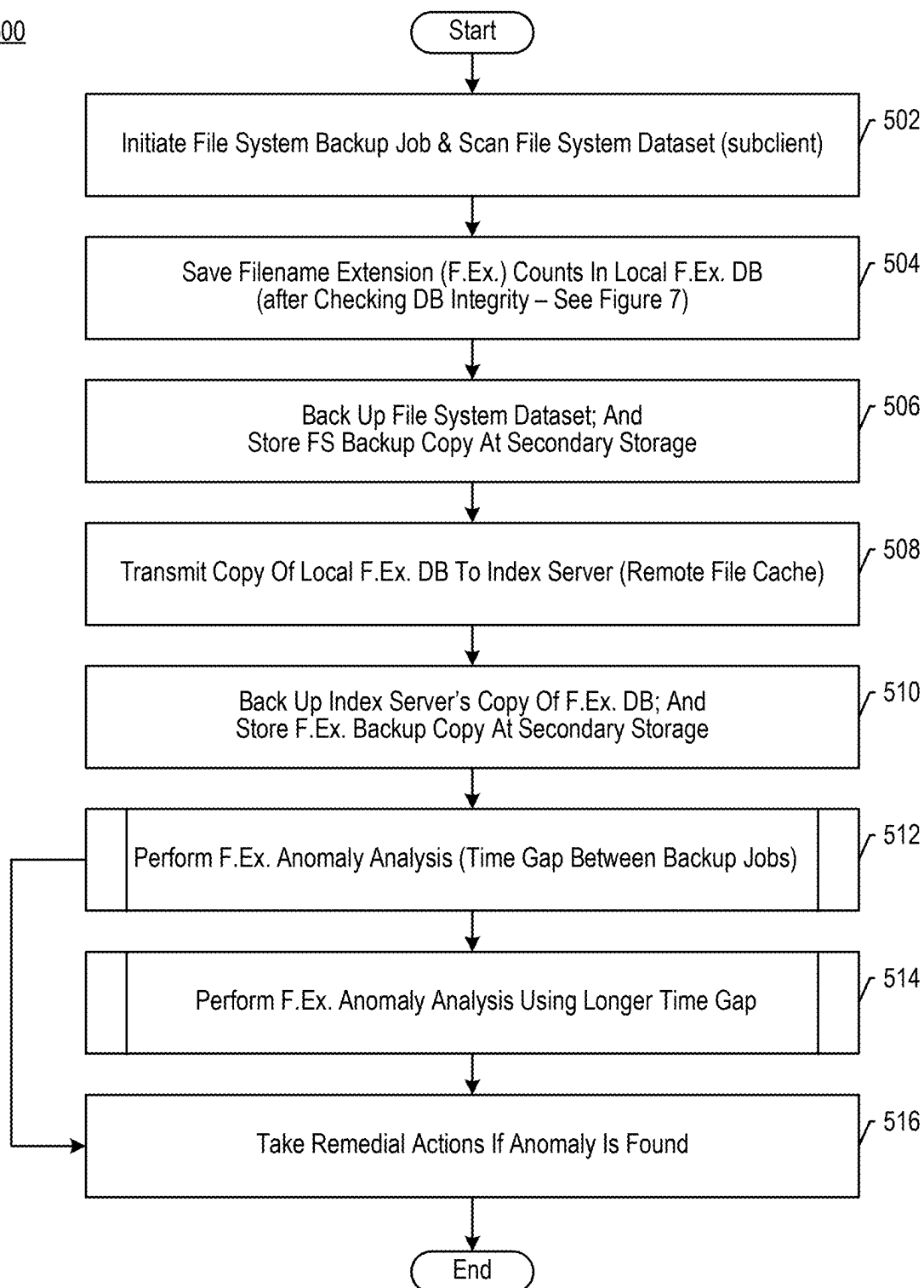


FIG. 5

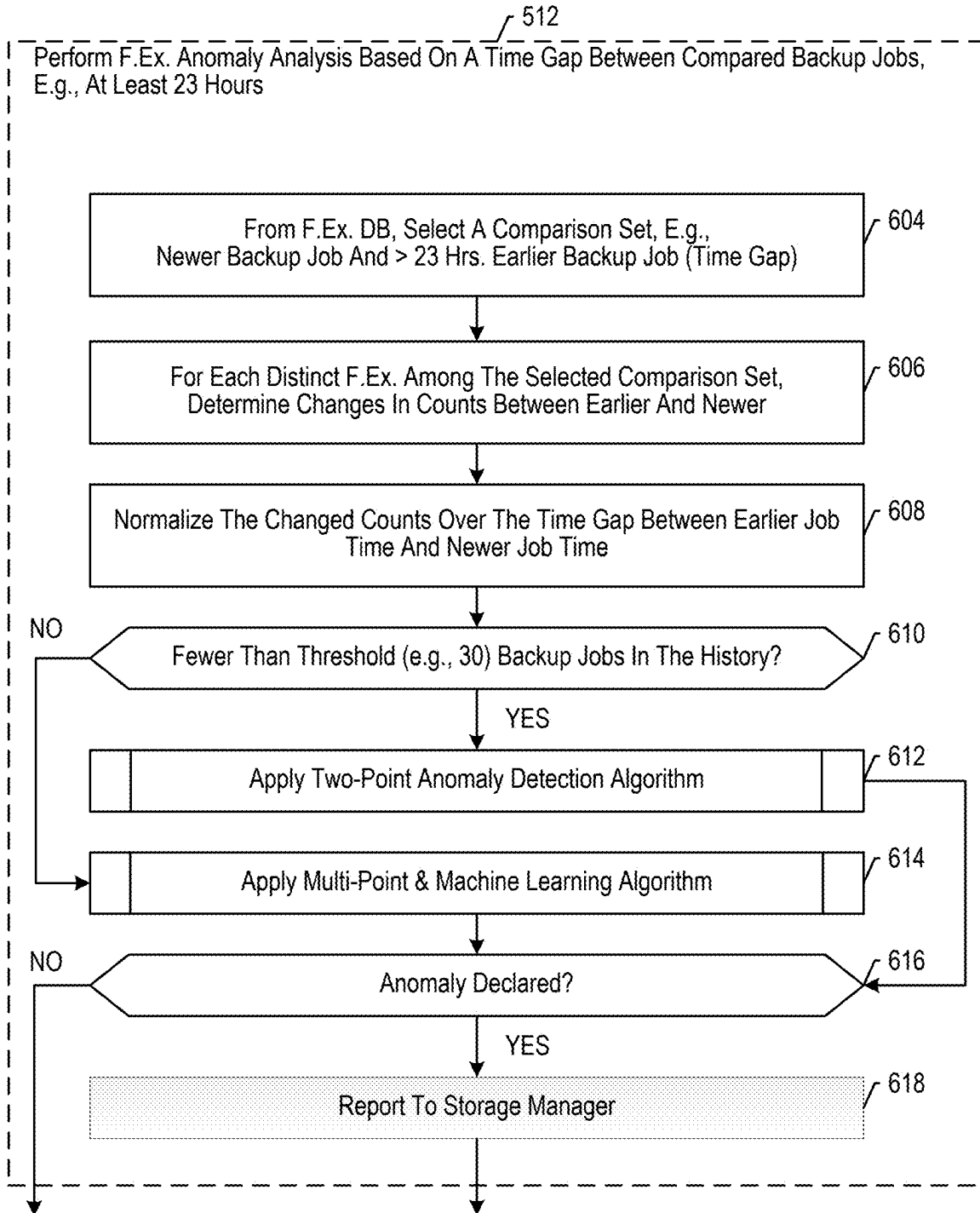


FIG. 6

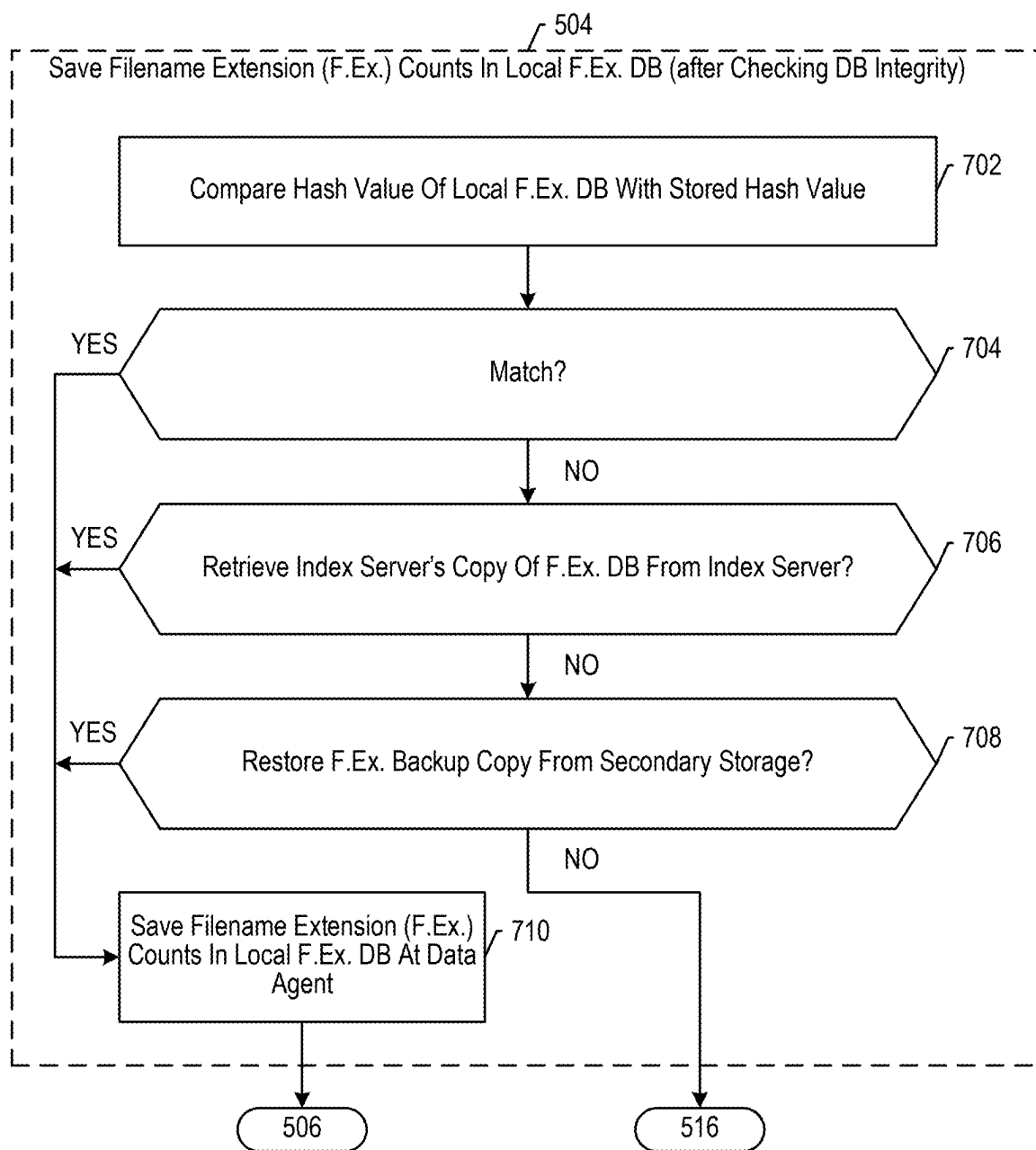


FIG. 7

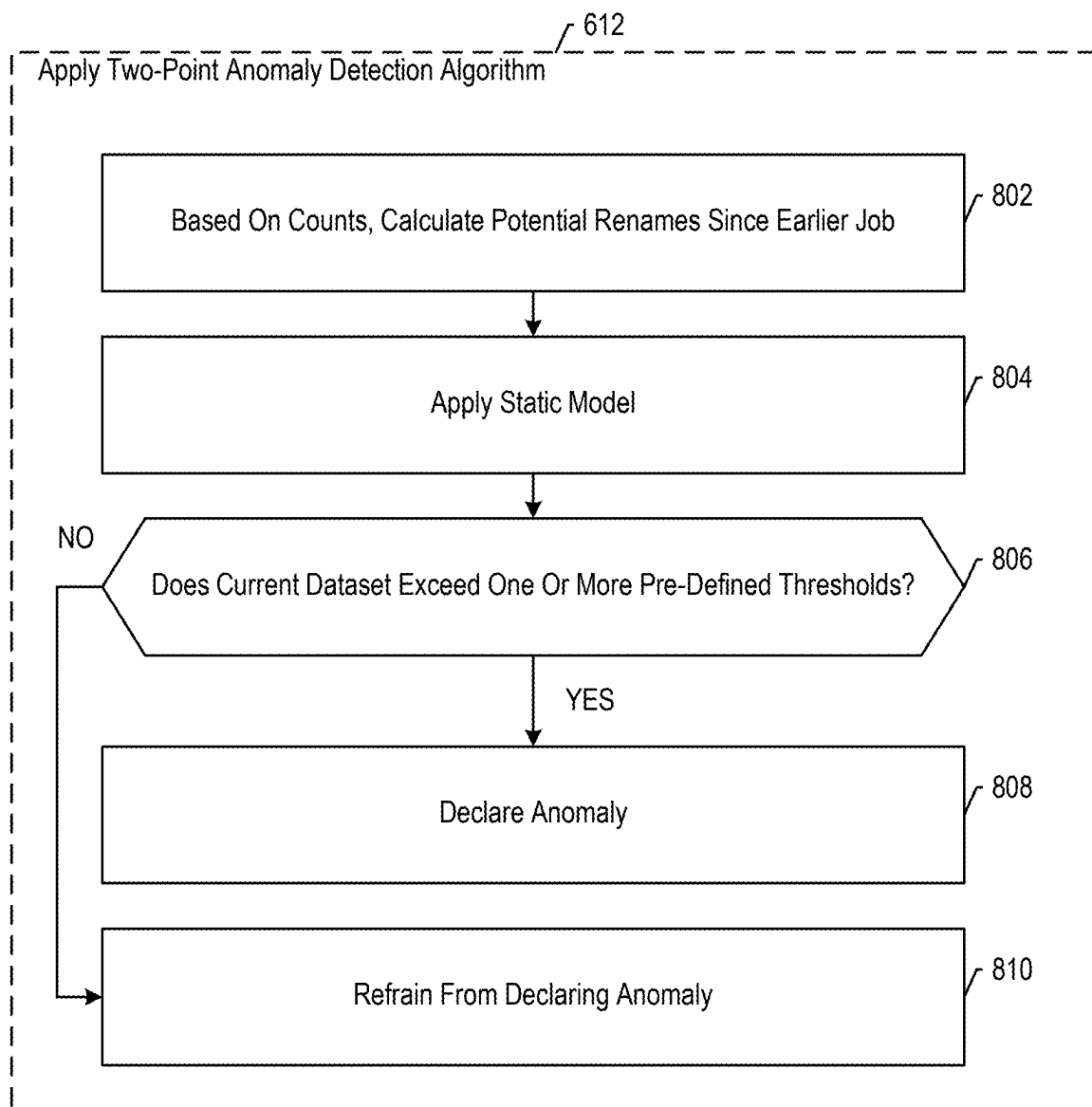


FIG. 8

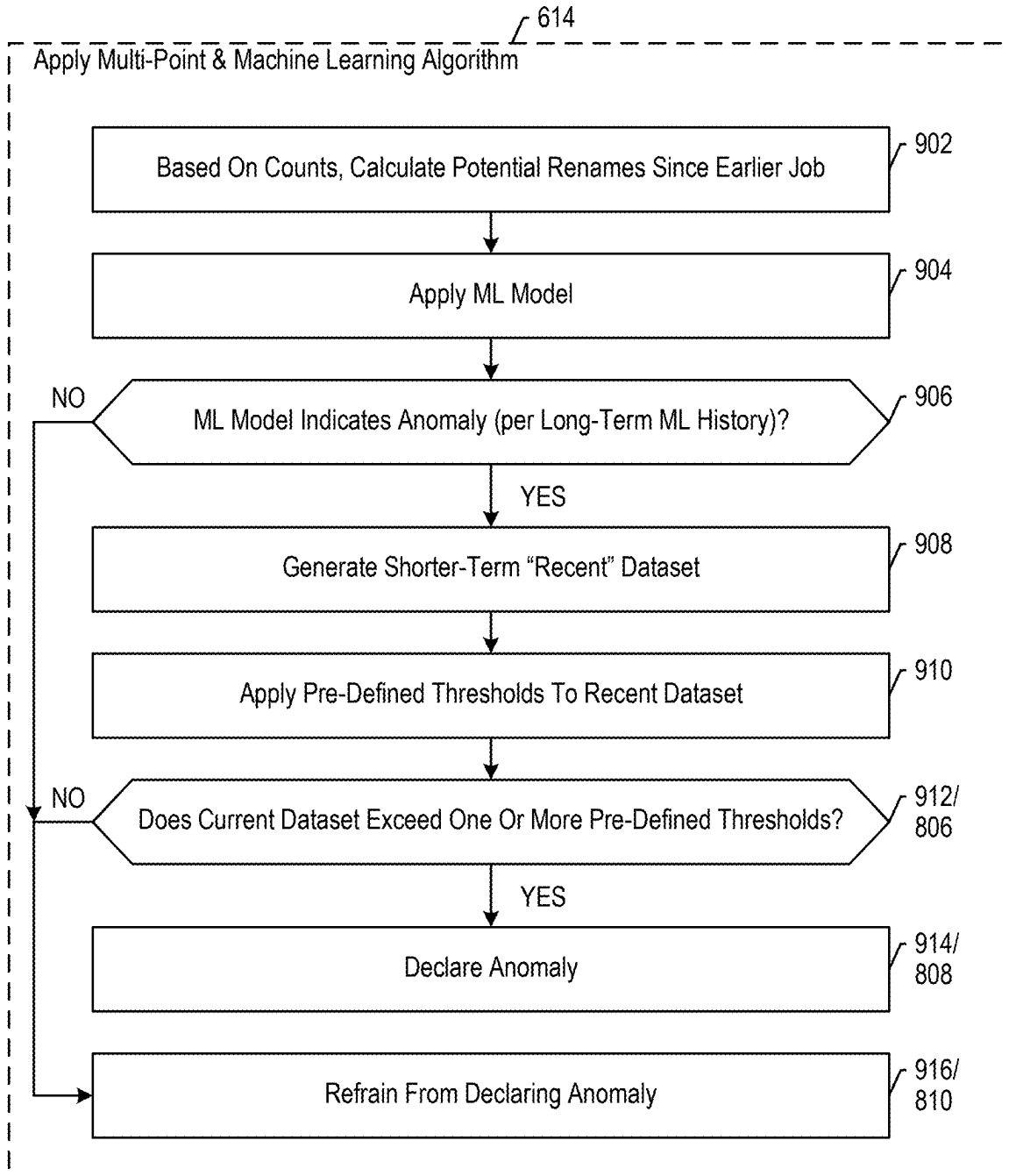


FIG. 9

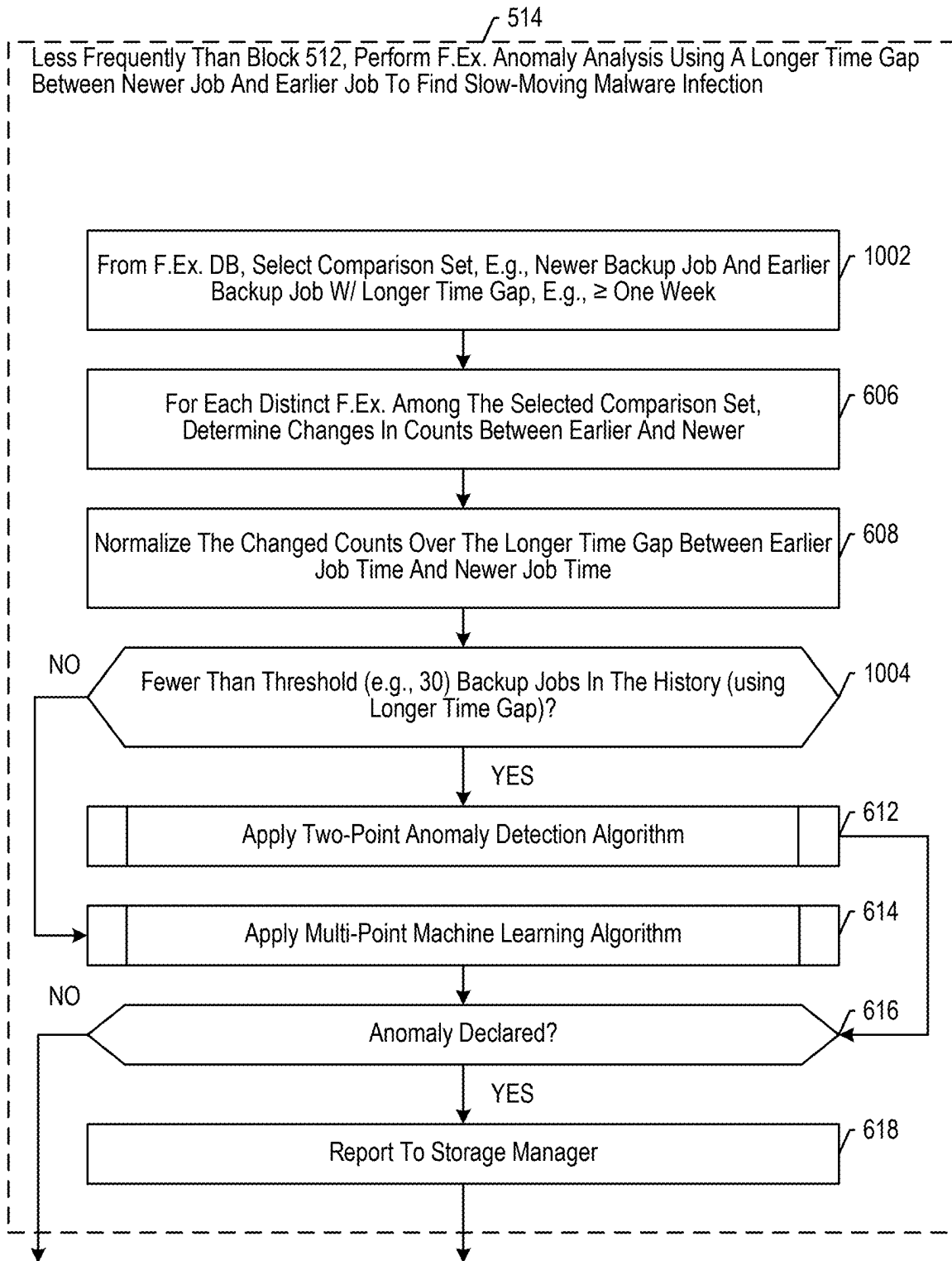


FIG. 10

**RANSOMWARE DETECTION ACCURACY
BASED ON MACHINE LEARNING
ANALYSIS OF FILENAME EXTENSION
PATTERNS**

**INCORPORATION BY REFERENCE TO ANY
PRIORITY APPLICATIONS**

[0001] Any and all applications, if any, for which a foreign or domestic priority claim is identified in the Application Data Sheet of the present application are hereby incorporated by reference in their entireties under 37 CFR 1.57.

COPYRIGHT NOTICE

[0002] A portion of this patent document contains material that is subject to copyright protection. The copyright owner has no objection to the facsimile reproduction by anyone of the patent document and/or the patent disclosure as it appears in the United States Patent and Trademark Office patent file and/or records, but otherwise reserves all copy-rights whatsoever.

BACKGROUND

[0003] Businesses recognize the commercial value of their data and seek reliable, cost effective ways to protect the information stored on their computer networks while minimizing impact on productivity. A company may issue one or more computing devices to its employees. Through their use of the computing devices, employees may be targeted by ransomware. Ransomware is a general term that refers to malicious software (or malware) that typically encrypts and/or obfuscates data on a computing device against the will or desire of the user. A third-party entity attempts to place the ransomware on a computer using nefarious techniques, such as phishing, spear phishing, malicious e-mail attachments, embedded document scripts, or other such techniques. A user may inadvertently install the ransomware on their computer, in which case, the ransomware then proceeds to encrypt and/or obfuscate file(s) on the computer and on associated storage. Furthermore, the user is not informed of the decryption key or the manner in which to restore the data, and may be extorted into paying a fee or performing a task in order to decrypt or unlock the encrypted data. Having ransomware on a computing device with scheduled backups is particularly troublesome because the ransomware may impact the backup copies that are created while the computing device is infected. The value of stored backup data and the associated backup infrastructure that created it can evaporate if the backup data is unavailable due to malware that infected the source data. Backup copies of malware-infected data may be unusable and unrecoverable due to malware encryption, and/or may cause malware to spread within the production environment when they are restored.

SUMMARY

[0004] The present inventors devised techniques that improve ransomware detection accuracy by analyzing filename extension patterns. One of the technological improvements of the disclosed approach is that it is malware-agnostic. The disclosed approach analyzes patterns of changes in filename extension counts, relative to each other, that occur in a file system over time. Prior-art techniques identify possible malware infections by monitoring activity

in a file system and comparing filename extensions therein to a catalog or dictionary of known malware extensions. This catalog can become dated as new malware emerges, and conducting the comparisons tends to slow down system performance in the production environment. This prior-art approach also is prone to false positives when customers introduce new applications or workloads that generate novel extensions that are not in the catalog. Advantageously, the disclosed technology does not rely on knowing or recognizing malware extensions, and does not rely on real-time monitoring of the target file system. Instead, during each successive backup job of the file system, the disclosed technology counts distinct filename extensions and compares the counts to each other and to corresponding counts taken in earlier backup jobs. The disclosed approach is malware-agnostic and will detect anomalies in extension counts that arise between backup jobs, regardless of the particular filename extension. Accordingly, new extension types will be taken into consideration when they appear.

[0005] A filename extension is well-known in the art to mean a suffix to the name of a computer file (e.g., .txt, .docx, .ppt, .jpg). The filename extension may indicate an association with an application or workload that created the file, e.g., Microsoft Word, Microsoft PowerPoint, etc. or may indicate a characteristic of the file contents, e.g., picture, movie, text, type of encoding, etc. Malware may generate new filename extensions or use previously known filename extensions, depending on the nature of the malware.

[0006] The disclosed anomaly detection analysis is performed preferably within a backup job, so that an anomaly in the source data may be detected timely and reported. Illustratively, the disclosed techniques are performed by a file system data agent operating at or with the data source. During the file system scan conducted at the start of the backup job, the data agent counts the distinct filename extensions found in the scanned file system and stores the counts and other statistics in a local “extensions database.” After backup copies of the file system are safely stored in secondary storage, the data agent uses the information in the extensions database to perform the disclosed anomaly detection analysis.

[0007] The anomaly detection analysis preferably uses machine learning to discern a behavior pattern of the file system, a pattern that indicates how filename extensions are distributed and how much they change between backup jobs over time. The anomaly detection analysis determines whether the scanned file system is anomalous compared to the pattern—the anomaly being based on how the filename extensions are distributed relative to each other. Machine learning advantageously discerns dynamic aspects of the file system, i.e., periodic changes in the number of files, trends in the distribution of different filename extensions, periodic peaks and troughs, etc. Machine learning requires a substantial amount of historical data (i.e., information collected during backup jobs) before the machine learning model is reliable. Until enough historical data has been collected, i.e., before enough backup jobs have been run on the file system, the disclosed techniques use static parameters and pre-defined threshold values to evaluate the filename extensions counted in the current backup job. If the data agent that performs the anomaly detection analysis determines that an anomaly is present, the data agent raises alarms to other components so that remedial actions can be taken. Additionally, the illustrative system is configured to generate

displays of unusual file activity and filename extension trends that graphically depict possible anomalies.

[0008] Because the disclosed analysis evaluates numerical relationships as between different filename extensions, whatever they happen to be in the customer's data, and regardless of whether the filename extensions were previously known or unknown, the disclosed analysis will improve ransomware/malware detection accuracy. Moreover, the disclosed approach does not require frequent catalog or dictionary upgrades to keep up with newly identified malwares. More details are given below and in the accompanying figures.

BRIEF DESCRIPTION OF THE DRAWINGS

[0009] FIG. 1A is a block diagram illustrating an example information management system.

[0010] FIG. 1B is a detailed view of a primary storage device, a secondary storage device, and some examples of primary data and secondary copy data.

[0011] FIG. 1C is a block diagram of an example information management system including a storage manager, one or more data agents, and one or more media agents.

[0012] FIG. 1D is a block diagram illustrating a scalable information management system.

[0013] FIG. 1E illustrates certain secondary copy operations according to an example storage policy.

[0014] FIGS. 1F-1H are block diagrams illustrating suitable data structures that may be employed by the information management system.

[0015] FIG. 2A illustrates a system and technique for synchronizing primary data to a destination such as a failover site using secondary copy data.

[0016] FIG. 2B illustrates an information management system architecture incorporating use of a network file system (NFS) protocol for communicating between the primary and secondary storage subsystems.

[0017] FIG. 2C is a block diagram of an example of a highly scalable managed data pool architecture.

[0018] FIG. 3 is a block diagram that depicts some components of a system 300 that improves ransomware detection accuracy based on machine learning analysis of filename extension patterns, according to an illustrative embodiment.

[0019] FIG. 4 is a block diagram that depicts several components of system 300, including some subcomponents of data agent 342, according to an embodiment.

[0020] FIGS. 4A and 4AA depict an example screenshot of command center user interface 460, which displays an "unusual file activity" screen depicting filename extension trends over several backup jobs.

[0021] FIG. 5 is a flowchart that depicts some operations of a method 500, according to an illustrative embodiment.

[0022] FIG. 6 is a flow chart depicting some operations of block 512 of method 500, according to an illustrative embodiment.

[0023] FIG. 7 is a flow chart depicting some operations of block 504 of method 500, according to an illustrative embodiment.

[0024] FIG. 8 is a flow chart depicting some operations of block 612 of method 500, according to an illustrative embodiment.

[0025] FIG. 9 is a flow chart depicting some operations of block 614 of method 500, according to an illustrative embodiment.

[0026] FIG. 10 is a flow chart depicting some operations of block 514 of method 500, according to an illustrative embodiment.

DETAILED DESCRIPTION

[0027] Detailed descriptions and examples of systems and methods according to one or more illustrative embodiments of the present invention may be found in the section entitled IMPROVED RANSOMWARE DETECTION ACCURACY BASED ON MACHINE LEARNING ANALYSIS OF FILENAME EXTENSION PATTERNS, as well as in the section entitled Example Embodiments, and also in FIGS. 3-10 herein. Furthermore, components and functionality for ransomware detection based on machine learning analysis of filename extension patterns may be configured and/or incorporated into information management systems such as those described herein in FIGS. 1A-1H and 2A-2C. Various embodiments described herein are intimately tied to, enabled by, and would not exist except for, computer technology. For example, the machine learning analysis of filename extension patterns described herein in reference to various embodiments cannot reasonably be performed by humans alone, without the computer technology upon which they are implemented.

Information Management System Overview

[0028] With the increasing importance of protecting and leveraging data, organizations simply cannot risk losing critical data. Moreover, runaway data growth and other modern realities make protecting and managing data increasingly difficult. There is therefore a need for efficient, powerful, and user-friendly solutions for protecting and managing data and for smart and efficient management of data storage. Depending on the size of the organization, there may be many data production sources which are under the purview of tens, hundreds, or even thousands of individuals. In the past, individuals were sometimes responsible for managing and protecting their own data, and a patchwork of hardware and software point solutions may have been used in any given organization. These solutions were often provided by different vendors and had limited or no interoperability. Certain embodiments described herein address these and other shortcomings of prior approaches by implementing scalable, unified, organization-wide information management, including data storage management.

[0029] FIG. 1A shows one such information management system 100 (or "system 100"), which generally includes combinations of hardware and software configured to protect and manage data and metadata that are generated and used by computing devices in system 100. System 100 may be referred to in some embodiments as a "storage management system" or a "data storage management system." System 100 performs information management operations, some of which may be referred to as "storage operations" or "data storage operations," to protect and manage the data residing in and/or managed by system 100. The organization that employs system 100 may be a corporation or other business entity, non-profit organization, educational institution, household, governmental agency, or the like.

[0030] Generally, the systems and associated components described herein may be compatible with and/or provide some or all of the functionality of the systems and corresponding components described in one or more of the

following U.S. patents/publications and patent applications assigned to Commvault Systems, Inc., each of which is hereby incorporated by reference in its entirety herein:

- [0031] U.S. Pat. No. 7,035,880, entitled “Modular Backup and Retrieval System Used in Conjunction With a Storage Area Network”;
- [0032] U.S. Pat. No. 7,107,298, entitled “System And Method For Archiving Objects In An Information Store”;
- [0033] U.S. Pat. No. 7,246,207, entitled “System and Method for Dynamically Performing Storage Operations in a Computer Network”;
- [0034] U.S. Pat. No. 7,315,923, entitled “System And Method For Combining Data Streams In Pipelined Storage Operations In A Storage Network”;
- [0035] U.S. Pat. No. 7,343,453, entitled “Hierarchical Systems and Methods for Providing a Unified View of Storage Information”;
- [0036] U.S. Pat. No. 7,395,282, entitled “Hierarchical Backup and Retrieval System”;
- [0037] U.S. Pat. No. 7,529,782, entitled “System and Methods for Performing a Snapshot and for Restoring Data”;
- [0038] U.S. Pat. No. 7,617,262, entitled “System and Methods for Monitoring Application Data in a Data Replication System”;
- [0039] U.S. Pat. No. 7,734,669, entitled “Managing Copies Of Data”;
- [0040] U.S. Pat. No. 7,747,579, entitled “Metabase for Facilitating Data Classification”;
- [0041] U.S. Pat. No. 8,156,086, entitled “Systems And Methods For Stored Data Verification”;
- [0042] U.S. Pat. No. 8,170,995, entitled “Method and System for Offline Indexing of Content and Classifying Stored Data”;
- [0043] U.S. Pat. No. 8,230,195, entitled “System And Method For Performing Auxiliary Storage Operations”;
- [0044] U.S. Pat. No. 8,285,681, entitled “Data Object Store and Server for a Cloud Storage Environment, Including Data Deduplication and Data Management Across Multiple Cloud Storage Sites”;
- [0045] U.S. Pat. No. 8,307,177, entitled “Systems And Methods For Management Of Virtualization Data”;
- [0046] U.S. Pat. No. 8,364,652, entitled “Content-Aligned, Block-Based Deduplication”;
- [0047] U.S. Pat. No. 8,578,120, entitled “Block-Level Single Instancing”;
- [0048] U.S. Pat. No. 8,954,446, entitled “Client-Side Repository in a Networked Deduplicated Storage System”;
- [0049] U.S. Pat. No. 9,020,900, entitled “Distributed Deduplicated Storage System”;
- [0050] U.S. Pat. No. 9,098,495, entitled “Application-Aware and Remote Single Instance Data Management”;
- [0051] U.S. Pat. No. 9,239,687, entitled “Systems and Methods for Retaining and Using Data Block Signatures in Data Protection Operations”;
- [0052] U.S. Pat. No. 9,444,811, entitled “Using An Enhanced Data Agent To Restore Backed Up Data Across Autonomous Storage Management Systems”;
- [0053] U.S. Pat. No. 9,633,033 entitled “High Availability Distributed Deduplicated Storage System”;

[0054] U.S. Pat. No. 10,228,962 entitled “Live Synchronization and Management of Virtual Machines across Computing and Virtualization Platforms and Using Live Synchronization to Support Disaster Recovery”;

[0055] U.S. Pat. No. 10,255,143 entitled “Deduplication Replication In A Distributed Deduplication Data Storage System”. U.S. Pat. No. 10,592,145, entitled “Machine Learning-Based Data Object Storage”;

[0056] U.S. Pat. No. 10,684,924 entitled “Data Restoration Operations Based on Network Path Information”;

[0057] U.S. Patent Pub. No. 2006/0224846, entitled “System and Method to Support Single Instance Storage Operations” now abandoned;

[0058] U.S. Patent Pub. No. 2016/0350391 entitled “Replication Using Deduplicated Secondary Copy Data” now abandoned;

[0059] U.S. Patent Pub. No. 2017/0235647 entitled “Data Protection Operations Based on Network Path Information” now abandoned;

[0060] U.S. Patent Pub. No. 2019/0108341 entitled “Ransomware Detection And Data Pruning Management” now abandoned;

[0061] U.S. Patent Pub. No. 2022/0292188 entitled “Detecting Ransomware In Secondary Copies Of Client Computing Devices” (0692.US2);

[0062] U.S. Patent Pub. No. 2022/0292196 entitled “Detecting Ransomware In Monitored Data” (0692.US1).

[0063] System **100** includes computing devices and computing technologies. For instance, system **100** can include one or more client computing devices **102** and secondary storage computing devices **106**, as well as storage manager **140** or a host computing device for it. Computing devices can include, without limitation, one or more: workstations, personal computers, desktop computers, or other types of generally fixed computing systems such as mainframe computers, servers, and minicomputers. Other computing devices can include mobile or portable computing devices, such as one or more laptops, tablet computers, personal data assistants, mobile phones (such as smartphones), and other mobile or portable computing devices such as embedded computers, set top boxes, vehicle-mounted devices, wearable computers, etc. Servers can include mail servers, file servers, database servers, virtual machine servers, and web servers. Any given computing device comprises one or more hardware processors (e.g., CPU and/or single-core or multi-core processors), as well as corresponding non-transitory computer memory (e.g., random-access memory (RAM)) for storing computer programs which are to be executed by the one or more hardware processors. Other computer memory for mass storage of data may be packaged/configured with the computing device (e.g., an internal hard disk) and/or may be external and accessible by the computing device (e.g., network-attached storage, a storage array, etc.). In some cases, a computing device includes cloud computing resources, which may be implemented as virtual machines. For instance, one or more virtual machines may be provided to the organization by a third-party cloud service vendor.

[0064] In some embodiments, computing devices can include one or more virtual machine(s) running on a physical host computing device (or “host machine”) operated by

the organization. As one example, the organization may use one virtual machine as a database server and another virtual machine as a mail server, both virtual machines operating on the same host machine. A Virtual machine (“VM”) is a software implementation of a computer that does not physically exist and is instead instantiated in an operating system of a physical computer (or host machine) to enable applications to execute within the VM’s environment, i.e., a VM emulates a physical computer. A VM includes an operating system and associated virtual resources, such as computer memory and processor(s). A hypervisor operates between the VM and the hardware of the physical host machine and is generally responsible for creating and running the VMs. Hypervisors are also known in the art as virtual machine monitors or a virtual machine managers or “VMMs”, and may be implemented in software, firmware, and/or specialized hardware installed on the host machine. Examples of hypervisors include ESX Server, by VMware, Inc. of Palo Alto, California; Microsoft Virtual Server and Microsoft Windows Server Hyper-V, both by Microsoft Corporation of Redmond, Washington; Sun xVM by Oracle America Inc. of Santa Clara, California; and Xen by Citrix Systems, Santa Clara, California. The hypervisor provides resources to each virtual operating system such as a virtual processor, virtual memory, a virtual network device, and a virtual disk. Each virtual machine has one or more associated virtual disks. The hypervisor typically stores the data of virtual disks in files on the file system of the physical host machine, called virtual machine disk files (“VMDK” in VMware lingo) or virtual hard disk image files (in Microsoft lingo). For example, VMware’s ESX Server provides the Virtual Machine File System (VMFS) for the storage of virtual machine disk files. A virtual machine reads data from and writes data to its virtual disk much the way that a physical machine reads data from and writes data to a physical disk. Examples of techniques for implementing information management in a cloud computing environment are described in U.S. Pat. No. 8,285,681. Examples of techniques for implementing information management in a virtualized computing environment are described in U.S. Pat. No. 8,307,177.

[0065] Information management system **100** can also include electronic data storage devices, generally used for mass storage of data, including, e.g., primary storage devices **104** and secondary storage devices **108**. Storage devices can generally be of any suitable type including, without limitation, disk drives, storage arrays (e.g., storage-area network (SAN) and/or network-attached storage (NAS) technology), semiconductor memory (e.g., solid state storage devices), network attached storage (NAS) devices, tape libraries, or other magnetic, non-tape storage devices, optical media storage devices, combinations of the same, etc. In some embodiments, storage devices form part of a distributed file system. In some cases, storage devices are provided in a cloud storage environment (e.g., a private cloud or one operated by a third-party vendor), whether for primary data or secondary copies or both. Depending on context, the term “information management system” can refer to generally all of the illustrated hardware and software components in FIG. **1C**, or the term may refer to only a subset of the illustrated components. For instance, in some cases, system **100** generally refers to a combination of specialized components used to protect, move, manage, manipulate, analyze, and/or process data and metadata generated by client computing devices **102**. However, system **100** in some cases does not

include the underlying components that generate and/or store primary data **112**, such as the client computing devices **102** themselves, and the primary storage devices **104**. Likewise secondary storage devices **108** (e.g., a third-party provided cloud storage environment) may not be part of system **100**. As an example, “information management system” or “storage management system” may sometimes refer to one or more of the following components, which will be described in further detail below: storage manager, data agent, and media agent.

[0066] One or more client computing devices **102** may be part of system **100**, each client computing device **102** having an operating system and at least one application **110** and one or more accompanying data agents executing thereon; and associated with one or more primary storage devices **104** storing primary data **112**. Client computing device(s) **102** and primary storage devices **104** may generally be referred to in some cases as primary storage subsystem **117**.

Client Computing Devices, Clients, and Subclients

[0067] Typically, a variety of sources in an organization produce data to be protected and managed. As just one illustrative example, in a corporate environment such data sources can be employee workstations and company servers such as a mail server, a web server, a database server, a transaction server, or the like. In system **100**, data generation sources include one or more client computing devices **102**. A computing device that has a data agent **142** installed and operating on it is generally referred to as a “client computing device” **102**, and may include any type of computing device, without limitation. A client computing device **102** may be associated with one or more users and/or user accounts.

[0068] A “client” is a logical component of information management system **100**, which may comprise a logical grouping of one or more data agents installed on a client computing device **102**. Storage manager **140** recognizes a client as a component of system **100**, and in some embodiments, may automatically create a client component the first time a data agent **142** is installed on a client computing device **102**. Because data generated by executable component(s) **110** is tracked by the associated data agent **142** so that it may be properly protected in system **100**, a client may be said to generate data and to store the generated data to primary storage, such as primary storage device **104**. However, the terms “client” and “client computing device” as used herein do not imply that a client computing device **102** is necessarily configured in the client/server sense relative to another computing device such as a mail server, or that a client computing device **102** cannot be a server in its own right. As just a few examples, a client computing device **102** can be and/or include mail servers, file servers, database servers, virtual machine servers, and/or web servers.

[0069] Each client computing device **102** may have application(s) **110** executing thereon which generate and manipulate the data that is to be protected from loss and managed in system **100**. Applications **110** generally facilitate the operations of an organization, and can include, without limitation, mail server applications (e.g., Microsoft Exchange Server), file system applications, mail client applications (e.g., Microsoft Exchange Client), database applications or database management systems (e.g., SQL, Oracle, SAP, Lotus Notes Database), word processing applications (e.g., Microsoft Word), spreadsheet applications, financial applications, presentation applications, graphics and/or

video applications, browser applications, mobile applications, entertainment applications, and so on. Each application **110** may be accompanied by an application-specific data agent **142**, though not all data agents **142** are application-specific or associated with only application. A file manager application, e.g., Microsoft Windows Explorer, may be considered an application **110** and may be accompanied by its own data agent **142**. Client computing devices **102** can have at least one operating system (e.g., Microsoft Windows, Mac OS X, IOS, IBM z/OS, Linux, other Unix-based operating systems, etc.) installed thereon, which may support or host one or more file systems and other applications **110**. In some embodiments, a virtual machine that executes on a host client computing device **102** may be considered an application **110** and may be accompanied by a specific data agent **142** (e.g., virtual server data agent). Client computing devices **102** and other components in system **100** can be connected to one another via one or more electronic communication pathways **114**. For example, a first communication pathway **114** may communicatively couple client computing device **102** and secondary storage computing device **106**; a second communication pathway **114** may communicatively couple storage manager **140** and client computing device **102**; and a third communication pathway **114** may communicatively couple storage manager **140** and secondary storage computing device **106**, etc. (see, e.g., FIG. 1A and FIG. 1C). A communication pathway **114** can include one or more networks or other connection types including one or more of the following, without limitation: the Internet, a wide area network (WAN), a local area network (LAN), a Storage Area Network (SAN), a Fibre Channel (FC) connection, a Small Computer System Interface (SCSI) connection, a virtual private network (VPN), a token ring or TCP/IP based network, an intranet network, a point-to-point link, a cellular network, a wireless data transmission system, a two-way cable system, an interactive kiosk network, a satellite network, a broadband network, a baseband network, a neural network, a mesh network, an ad hoc network, other appropriate computer or telecommunications networks, combinations of the same or the like. Communication pathways **114** in some cases may also include application programming interfaces (APIs) including, e.g., cloud service provider APIs, virtual machine management APIs, and hosted service provider APIs. The underlying infrastructure of communication pathways **114** may be wired and/or wireless, analog and/or digital, or any combination thereof; and the facilities used may be private, public, third-party provided, or any combination thereof, without limitation.

[0070] A “subclient” is a logical grouping of all or part of a client’s primary data **112**. Thus, a subclient is a data source. In general, a subclient may be defined according to how the subclient data is to be protected as a unit in system **100**. For example, a subclient may be associated with a certain storage policy. A given client may thus comprise several subclients, each subclient associated with a different storage policy. For example, some files may form a first subclient that requires compression and deduplication and is associated with a first storage policy. Other files of the client may form a second subclient that requires a different retention schedule as well as encryption, and may be associated with a different, second storage policy. As a result, though the primary data may be generated by the same application **110** and may belong to one given client, portions of the data

may be assigned to different subclients for distinct treatment by system **100**. More detail on subclients is given in regard to storage policies below.

Primary Data and Example Primary Storage Devices

[0071] Primary data **112** is generally production data or “live” data generated by the operating system and/or applications **110** executing on client computing device **102**. Primary data **112** is generally stored on primary storage device(s) **104** and is organized via a file system operating on the client computing device **102**. Thus, client computing device(s) **102** and corresponding applications **110** may create, access, modify, write, delete, and otherwise use primary data **112**. Primary data **112** is generally in the native format of the source application **110**. Primary data **112** is an initial or first stored body of data generated by the source application **110**. Primary data **112** in some cases is created substantially directly from data generated by the corresponding source application **110**. It can be useful in performing certain tasks to organize primary data **112** into units of different granularities. In general, primary data **112** can include files, directories, file system volumes, data blocks, extents, or any other hierarchies or organizations of data objects. As used herein, a “data object” can refer to (i) any file that is currently addressable by a file system or that was previously addressable by the file system (e.g., an archive file), and/or to (ii) a subset of such a file (e.g., a data block, an extent, etc.). Primary data **112** may include structured data (e.g., database files), unstructured data (e.g., documents), and/or semi-structured data. See, e.g., FIG. 1B.

[0072] It can also be useful in performing certain functions of system **100** to access and modify metadata within primary data **112**. Metadata generally includes information about data objects and/or characteristics associated with the data objects. For simplicity herein, it is to be understood that, unless expressly stated otherwise, any reference to primary data **112** generally also includes its associated metadata, but references to metadata generally do not include the primary data. Metadata can include, without limitation, one or more of the following: the data owner (e.g., the client or user that generates the data), the last modified time (e.g., the time of the most recent modification of the data object), a data object name (e.g., a file name), a data object size (e.g., a number of bytes of data), information about the content (e.g., an indication as to the existence of a particular search term), user-supplied tags, to/from information for email (e.g., an email sender, recipient, etc.), creation date, file type (e.g., format or application type), last accessed time, application type (e.g., type of application that generated the data object), location/network (e.g., a current, past or future location of the data object and network pathways to/from the data object), geographic location (e.g., GPS coordinates), frequency of change (e.g., a period in which the data object is modified), business unit (e.g., a group or department that generates, manages or is otherwise associated with the data object), aging information (e.g., a schedule, such as a time period, in which the data object is migrated to secondary or long term storage), boot sectors, partition layouts, file location within a file folder directory structure, user permissions, owners, groups, access control lists (ACLs), system metadata (e.g., registry information), combinations of the same or other similar information related to the data object. In addition to metadata generated by or related to file systems and operating systems, some applications **110** and/or other

components of system 100 maintain indices of metadata for data objects, e.g., metadata associated with individual email messages. The use of metadata to perform classification and other functions is described in greater detail below.

[0073] Primary storage devices 104 storing primary data 112 may be relatively fast and/or expensive technology (e.g., flash storage, a disk drive, a hard-disk storage array, solid state memory, etc.), typically to support high-performance live production environments. Primary data 112 may be highly changeable and/or may be intended for relatively short term retention (e.g., hours, days, or weeks). According to some embodiments, client computing device 102 can access primary data 112 stored in primary storage device 104 by making conventional file system calls via the operating system. Each client computing device 102 is generally associated with and/or in communication with one or more primary storage devices 104 storing corresponding primary data 112. A client computing device 102 is said to be associated with or in communication with a particular primary storage device 104 if it is capable of one or more of: routing and/or storing data (e.g., primary data 112) to the primary storage device 104, coordinating the routing and/or storing of data to the primary storage device 104, retrieving data from the primary storage device 104, coordinating the retrieval of data from the primary storage device 104, and modifying and/or deleting data in the primary storage device 104. Thus, a client computing device 102 may be said to access data stored in an associated storage device 104. Primary storage device 104 may be dedicated or shared. In some cases, each primary storage device 104 is dedicated to an associated client computing device 102, e.g., a local disk drive. In other cases, one or more primary storage devices 104 can be shared by multiple client computing devices 102, e.g., via a local network, in a cloud storage implementation, etc. As one example, primary storage device 104 can be a storage array shared by a group of client computing devices 102, such as EMC Clariion, EMC Symmetrix, EMC Celerra, Dell EqualLogic, IBM XIV, NetApp FAS, HP EVA, and HP 3PAR.

[0074] System 100 may also include hosted services (not shown), which may be hosted in some cases by an entity other than the organization that employs the other components of system 100. For instance, the hosted services may be provided by online service providers. Such service providers can provide social networking services, hosted email services, or hosted productivity applications or other hosted applications such as software-as-a-service (SaaS), platform-as-a-service (PaaS), application service providers (ASPs), cloud services, or other mechanisms for delivering functionality via a network. As it services users, each hosted service may generate additional data and metadata, which may be managed by system 100, e.g., as primary data 112. In some cases, the hosted services may be accessed using one of the applications 110. As an example, a hosted mail service may be accessed via browser running on a client computing device 102.

Secondary Copies and Example Secondary Storage Devices

[0075] Primary data 112 stored on primary storage devices 104 may be compromised in some cases, such as when an employee deliberately or accidentally deletes or overwrites primary data 112. Or primary storage devices 104 can be damaged, lost, or otherwise corrupted. For recovery and/or regulatory compliance purposes, it is therefore useful to

generate and maintain copies of primary data 112. Accordingly, system 100 includes one or more secondary storage computing devices 106 and one or more secondary storage devices 108 configured to create and store one or more secondary copies 116 of primary data 112 including its associated metadata. The secondary storage computing devices 106 and the secondary storage devices 108 may be referred to as secondary storage subsystem 118.

[0076] Secondary copies 116 can help in search and analysis efforts and meet other information management goals as well, such as: restoring data and/or metadata if an original version is lost (e.g., by deletion, corruption, or disaster); allowing point-in-time recovery; complying with regulatory data retention and electronic discovery (e-discovery) requirements; reducing utilized storage capacity in the production system and/or in secondary storage; facilitating organization and search of data; improving user access to data files across multiple computing devices and/or hosted services; and implementing data retention and pruning policies. A secondary copy 116 can comprise a separate stored copy of data that is derived from one or more earlier-created stored copies (e.g., derived from primary data 112 or from another secondary copy 116). Secondary copies 116 can include point-in-time data, and may be intended for relatively long-term retention before some or all of the data is moved to other storage or discarded. In some cases, a secondary copy 116 may be in a different storage device than other previously stored copies; and/or may be remote from other previously stored copies. Secondary copies 116 can be stored in the same storage device as primary data 112. For example, a disk array capable of performing hardware snapshots stores primary data 112 and creates and stores hardware snapshots of the primary data 112 as secondary copies 116. Secondary copies 116 may be stored in relatively slow and/or lower cost storage (e.g., magnetic tape). A secondary copy 116 may be stored in a backup or archive format, or in some other format different from the native source application format or other format of primary data 112.

[0077] Secondary storage computing devices 106 may index secondary copies 116 (e.g., using a media agent 144), enabling users to browse and restore at a later time and further enabling the lifecycle management of the indexed data. After creation of a secondary copy 116 that represents certain primary data 112, a pointer or other location indicia (e.g., a stub) may be placed in primary data 112, or be otherwise associated with primary data 112, to indicate the current location of a particular secondary copy 116. Since an instance of a data object or metadata in primary data 112 may change over time as it is modified by application 110 (or hosted service or the operating system), system 100 may create and manage multiple secondary copies 116 of a particular data object or metadata, each copy representing the state of the data object in primary data 112 at a particular point in time. Moreover, since an instance of a data object in primary data 112 may eventually be deleted from primary storage device 104 and the file system, system 100 may continue to manage point-in-time representations of that data object, even though the instance in primary data 112 no longer exists. For virtual machines, the operating system and other applications 110 of client computing device(s) 102 may execute within or under the management of virtualization software (e.g., a VMM), and the primary storage device(s) 104 may comprise a virtual disk created on a

physical storage device. System 100 may create secondary copies 116 of the files or other data objects in a virtual disk file and/or secondary copies 116 of the entire virtual disk file itself (e.g., of an entire .vmdk file).

[0078] Secondary copies 116 are distinguishable from corresponding primary data 112. First, secondary copies 116 can be stored in a different format from primary data 112 (e.g., backup, archive, or other non-native format). For this or other reasons, secondary copies 116 may not be directly usable by applications 110 or client computing device 102 (e.g., via standard system calls or otherwise) without modification, processing, or other intervention by system 100 which may be referred to as “restore” operations. Secondary copies 116 may have been processed by data agent 142 and/or media agent 144 in the course of being created (e.g., compression, deduplication, encryption, integrity markers, indexing, formatting, application-aware metadata, etc.), and thus secondary copy 116 may represent source primary data 112 without necessarily being exactly identical to the source. Second, secondary copies 116 may be stored on a secondary storage device 108 that is inaccessible to application 110 running on client computing device 102 and/or hosted service. Some secondary copies 116 may be “offline copies,” in that they are not readily available (e.g., not mounted to tape or disk). Offline copies can include copies of data that system 100 can access without human intervention (e.g., tapes within an automated tape library, but not yet mounted in a drive), and copies that the system 100 can access only with some human intervention (e.g., tapes located at an offsite storage site).

Using Intermediate Devices for Creating Secondary Copies—Secondary Storage Computing Devices

[0079] Creating secondary copies can be challenging when hundreds or thousands of client computing devices 102 continually generate large volumes of primary data 112 to be protected. Also, there can be significant overhead involved in the creation of secondary copies 116. Moreover, specialized programmed intelligence and/or hardware capability is generally needed for accessing and interacting with secondary storage devices 108. Client computing devices 102 may interact directly with a secondary storage device 108 to create secondary copies 116, but in view of the factors described above, this approach can negatively impact the ability of client computing device 102 to serve/service application 110 and produce primary data 112. Further, any given client computing device 102 may not be optimized for interaction with certain secondary storage devices 108.

[0080] Thus, system 100 may include one or more software and/or hardware components which generally act as intermediaries between client computing devices 102 (that generate primary data 112) and secondary storage devices 108 (that store secondary copies 116). In addition to off-loading certain responsibilities from client computing devices 102, these intermediate components provide other benefits. For instance, as discussed further below with respect to FIG. 1D, distributing some of the work involved in creating secondary copies 116 can enhance scalability and improve system performance. For instance, using specialized secondary storage computing devices 106 and media agents 144 for interfacing with secondary storage devices 108 and/or for performing certain data processing operations can greatly improve the speed with which system 100 performs information management operations and can also

improve the capacity of the system to handle large numbers of such operations, while reducing the computational load on the production environment of client computing devices 102. The intermediate components can include one or more secondary storage computing devices 106 as shown in FIG. 1A and/or one or more media agents 144. Media agents are discussed further below (e.g., with respect to FIGS. 1C-1E). These special-purpose components of system 100 comprise specialized programmed intelligence and/or hardware capability for writing to, reading from, instructing, communicating with, or otherwise interacting with secondary storage devices 108.

[0081] Secondary storage computing device(s) 106 can comprise any of the computing devices described above, without limitation. In some cases, secondary storage computing device(s) 106 also include specialized hardware componentry and/or software intelligence (e.g., specialized interfaces) for interacting with certain secondary storage device(s) 108 with which they may be specially associated. To create a secondary copy 116 involving the copying of data from primary storage subsystem 117 to secondary storage subsystem 118, client computing device 102 may communicate the primary data 112 to be copied (or a processed version thereof generated by a data agent 142) to the designated secondary storage computing device 106, via a communication pathway 114. Secondary storage computing device 106 in turn may further process and convey the data or a processed version thereof to secondary storage device 108. One or more secondary copies 116 may be created from existing secondary copies 116, such as in the case of an auxiliary copy operation, described further below.

Example Primary Data and an Example Secondary Copy

[0082] FIG. 1B is a detailed view of some specific examples of primary data stored on primary storage device (s) 104 and secondary copy data stored on secondary storage device(s) 108, with other components of the system removed for the purposes of illustration. Stored on primary storage device(s) 104 are primary data 112 objects including word processing documents 119A-B, spreadsheets 120, presentation documents 122, video files 124, image files 126, email mailboxes 128 (and corresponding email messages 129A-C), HTML/XML or other types of markup language files 130, databases 132 and corresponding tables or other data structures 133A-133C. Some or all primary data 112 objects are associated with corresponding metadata (e.g., “Meta1-11”), which may include file system metadata and/or application-specific metadata. Stored on the secondary storage device(s) 108 are secondary copy 116 data objects 134A-C which may include copies of or may otherwise represent corresponding primary data 112.

[0083] Secondary copy data objects 134A-C can individually represent more than one primary data object. For example, secondary copy data object 134A represents three separate primary data objects 133C, 122, and 129C (represented as 133C', 122', and 129C', respectively, and accompanied by corresponding metadata Meta11, Meta3, and Meta8, respectively). Moreover, as indicated by the prime mark ('), secondary storage computing devices 106 or other components in secondary storage subsystem 118 may process the data received from primary storage subsystem 117 and store a secondary copy including a transformed and/or supplemented representation of a primary data object and/or metadata that is different from the original format, e.g., in a

compressed, encrypted, deduplicated, or other modified format. For instance, secondary storage computing devices **106** can generate new metadata or other information based on said processing, and store the newly generated information along with the secondary copies. Secondary copy data object **134B** represents primary data objects **120**, **133B**, and **119A** as **120'**, **133B'**, and **119A'**, respectively, accompanied by corresponding metadata **Meta2**, **Meta10**, and **Meta1**, respectively. Also, secondary copy data object **134C** represents primary data objects **133A**, **119B**, and **129A** as **133A'**, **119B'**, and **129A'**, respectively, accompanied by corresponding metadata **Meta9**, **Meta5**, and **Meta6**, respectively.

Example Information Management System Architecture

[0084] System **100** can incorporate a variety of different hardware and software components, which can in turn be organized with respect to one another in many different configurations, depending on the embodiment. There are critical design choices involved in specifying the functional responsibilities of the components and the role of each component in system **100**. Such design choices can impact how system **100** performs and adapts to data growth and other changing circumstances. FIG. 1C shows a system **100** designed according to these considerations and includes: storage manager **140**, one or more data agents **142** executing on client computing device(s) **102** and configured to process primary data **112**, and one or more media agents **144** executing on one or more secondary storage computing devices **106** for performing tasks involving secondary storage devices **108**.

Storage Manager

[0085] Storage manager **140** is a centralized storage and/or information manager that is configured to perform certain control functions and also to store certain critical information about system **100**—hence storage manager **140** is said to manage system **100**. As noted, the number of components in system **100** and the amount of data under management can be large. Managing the components and data is therefore a significant task, which can grow unpredictably as the number of components and data scale to meet the needs of the organization. For these and other reasons, according to certain embodiments, responsibility for controlling system **100**, or at least a significant portion of that responsibility, is allocated to storage manager **140**. Storage manager **140** can be adapted independently according to changing circumstances, without having to replace or re-design the remainder of the system. Moreover, a computing device for hosting and/or operating as storage manager **140** can be selected to best suit the functions and networking needs of storage manager **140**. These and other advantages are described in further detail below and with respect to FIG. 1D.

[0086] Storage manager **140** may be a software module or other application hosted by a suitable computing device. In some embodiments, storage manager **140** is itself a computing device (comprising computer hardware processors and computer memory) that performs the functions described herein. Storage manager **140** comprises or operates in conjunction with one or more associated data structures such as a dedicated database (e.g., management database **146**), depending on the configuration. The storage manager **140** generally initiates, performs, coordinates, and/or controls storage and other information management

operations performed by system **100**, e.g., to protect and control primary data **112** and secondary copies **116**. In general, storage manager **140** is said to manage system **100**, which includes communicating with, instructing, and controlling in some circumstances components such as data agents **142** and media agents **144**, etc. As shown by the dashed arrowed lines **114** in FIG. 1C, storage manager **140** may communicate with, instruct, and/or control some or all elements of system **100**, such as data agents **142** and media agents **144**. In this manner, storage manager **140** manages the operation of various hardware and software components in system **100**. In certain embodiments, control information originates from storage manager **140** and status as well as index reporting is transmitted to storage manager **140** by the managed components, whereas payload data and metadata are generally communicated between data agents **142** and media agents **144** (or otherwise between client computing device(s) **102** and secondary storage computing device(s) **106**), e.g., at the direction of and under the management of storage manager **140**. Control information can generally include parameters and instructions for carrying out information management operations, such as, without limitation, instructions to perform a task associated with an operation, timing information specifying when to initiate a task, data path information specifying what components to communicate with or access in carrying out an operation, and the like. In other embodiments, some information management operations are controlled or initiated by other components of system **100** (e.g., by media agents **144** or data agents **142**), instead of or in combination with storage manager **140**.

[0087] According to certain embodiments, storage manager **140** provides one or more of the following functions:

- [0088]** communicating with data agents **142** and media agents **144**, including transmitting instructions, messages, and/or queries, as well as receiving status reports, index information, messages, and/or queries, and responding to same;
- [0089]** initiating execution of information management operations;
- [0090]** initiating restore and recovery operations;
- [0091]** managing secondary storage devices **108** and inventory/capacity of the same;
- [0092]** allocating secondary storage devices **108** for secondary copy operations;
- [0093]** reporting, searching, and/or classification of data in system **100**;
- [0094]** monitoring completion of and status reporting related to information management operations and jobs;
- [0095]** tracking movement of data within system **100**;
- [0096]** tracking age information relating to secondary copies **116**, secondary storage devices **108**, comparing the age information against retention guidelines, and initiating data pruning when appropriate;
- [0097]** tracking logical associations between components in system **100**;
- [0098]** protecting metadata associated with system **100**, e.g., in management database **146**;
- [0099]** implementing job management, schedule management, event management, alert management, reporting, job history maintenance, user security management, disaster recovery management, and/or user interfacing for system administrators and/or end users of system **100**;

[0100] sending, searching, and/or viewing of log files; and

[0101] implementing operations management functionality.

[0102] Storage manager 140 may maintain an associated database 146 (or “storage manager database 146” or “management database 146”) of management-related data and information management policies 148. Database 146 is stored in computer memory accessible by storage manager 140. Database 146 may include a management index 150 (or “index 150”) or other data structure(s) that may store: logical associations between components of the system; user preferences and/or profiles (e.g., preferences regarding encryption, compression, or deduplication of primary data or secondary copies; preferences regarding the scheduling, type, or other aspects of secondary copy or other operations; mappings of particular information management users or user accounts to certain computing devices or other components, etc.; management tasks; media containerization; other useful data; and/or any combination thereof. For example, storage manager 140 may use index 150 to track logical associations between media agents 144 and secondary storage devices 108 and/or movement of data to/from secondary storage devices 108. For instance, index 150 may store data associating a client computing device 102 with a particular media agent 144 and/or secondary storage device 108, as specified in an information management policy 148.

[0103] Administrators and others may configure and initiate certain information management operations on an individual basis. But while this may be acceptable for some recovery operations or other infrequent tasks, it is often not workable for implementing on-going organization-wide data protection and management. Thus, system 100 may utilize information management policies 148 for specifying and executing information management operations on an automated basis. Generally, an information management policy 148 can include a stored data structure or other information source that specifies parameters (e.g., criteria and rules) associated with storage management or other information management operations. Storage manager 140 can process an information management policy 148 and/or index 150 and, based on the results, identify an information management operation to perform, identify the appropriate components in system 100 to be involved in the operation (e.g., client computing devices 102 and corresponding data agents 142, secondary storage computing devices 106 and corresponding media agents 144, etc.), establish connections to those components and/or between those components, and/or instruct and control those components to carry out the operation. In this manner, system 100 can translate stored information into coordinated activity among the various computing devices in system 100.

[0104] Management database 146 may maintain information management policies 148 and associated data, although information management policies 148 can be stored in computer memory at any appropriate location outside management database 146. For instance, an information management policy 148 such as a storage policy may be stored as metadata in a media agent database 152 or in a secondary storage device 108 (e.g., as an archive copy) for use in restore or other information management operations, depending on the embodiment. Information management policies 148 are described further below. According to certain embodiments, management database 146 comprises

a relational database (e.g., an SQL database) for tracking metadata, such as metadata associated with secondary copy operations (e.g., what client computing devices 102 and corresponding subclient data were protected and where the secondary copies are stored and which media agent 144 performed the storage operation(s)). This and other metadata may additionally be stored in other locations, such as at secondary storage computing device 106 or on the secondary storage device 108, allowing data recovery without the use of storage manager 140 in some cases. Thus, management database 146 may comprise data needed to kick off secondary copy operations (e.g., storage policies, schedule policies, etc.), status and reporting information about completed jobs (e.g., status and error reports on yesterday’s backup jobs), and additional information sufficient to enable restore and disaster recovery operations (e.g., media agent associations, location indexing, content indexing, etc.).

[0105] Storage manager 140 may include a jobs agent 156, a user interface 158, and a management agent 154, all of which may be implemented as interconnected software modules or application programs. These are described further below. Jobs agent 156 in some embodiments initiates, controls, and/or monitors the status of some or all information management operations previously performed, currently being performed, or scheduled to be performed by system 100. A job is a logical grouping of information management operations such as daily storage operations scheduled for a certain set of subclients (e.g., generating incremental block-level backup copies 116 at a certain time every day for database files in a certain geographical location). Thus, jobs agent 156 may access information management policies 148 (e.g., in management database 146) to determine when, where, and how to initiate/control jobs in system 100.

Storage Manager User Interfaces

[0106] User interface 158 may include information processing and display software, such as a graphical user interface (GUI), an application program interface (API), and/or other interactive interface(s) through which users and system processes can retrieve information about the status of information management operations or issue instructions to storage manager 140 and other components. Via user interface 158, users may issue instructions to the components in system 100 regarding performance of secondary copy and recovery operations. For example, a user may modify a schedule concerning the number of pending secondary copy operations. As another example, a user may employ the GUI to view the status of pending secondary copy jobs or to monitor the status of certain components in system 100 (e.g., the amount of capacity left in a storage device). Storage manager 140 may track information that permits it to select, designate, or otherwise identify content indices, deduplication databases, or similar databases or resources or data sets within its information management cell (or another cell) to be searched in response to certain queries. Such queries may be entered by the user by interacting with user interface 158.

[0107] Various embodiments of information management system 100 may be configured and/or designed to generate user interface data usable for rendering the various interactive user interfaces described. The user interface data may be used by system 100 and/or by another system, device, and/or software program (for example, a browser program), to render the interactive user interfaces. The interactive user

interfaces may be displayed on, for example, electronic displays (including, for example, touch-enabled displays), consoles, etc., whether direct-connected to storage manager **140** or communicatively coupled remotely, e.g., via an internet connection. The present disclosure describes various embodiments of interactive and dynamic user interfaces, some of which may be generated by user interface agent **158**, and which are the result of significant technological development. The user interfaces described herein may provide improved human-computer interactions, allowing for significant cognitive and ergonomic efficiencies and advantages over previous systems, including reduced mental workloads, improved decision-making, and the like. User interface **158** may operate in a single integrated view or console (not shown). The console may support a reporting capability for generating a variety of reports, which may be tailored to a particular aspect of information management. User interfaces are not exclusive to storage manager **140** and in some embodiments a user may access information locally from a computing device component of system **100**. For example, some information pertaining to installed data agents **142** and associated data streams may be available from client computing device **102**. Likewise, some information pertaining to media agents **144** and associated data streams may be available from secondary storage computing device **106**.

Storage Manager Management Agent

[0108] Management agent **154** can provide storage manager **140** with the ability to communicate with other components within system **100** and/or with other information management cells via network protocols and application programming interfaces (APIs) including, e.g., HTTP, HTTPS, FTP, REST, virtualization software APIs, cloud service provider APIs, and hosted service provider APIs, without limitation. Management agent **154** also allows multiple information management cells to communicate with one another. For example, system **100** in some cases may be one information management cell in a network of multiple cells adjacent to one another or otherwise logically related, e.g., in a WAN or LAN. With this arrangement, the cells may communicate with one another through respective management agents **154**. Inter-cell communications and hierarchy is described in greater detail in e.g., U.S. Pat. No. 7,343,453.

Information Management Cell

[0109] An “information management cell” (or “storage operation cell” or “cell”) may generally include a logical and/or physical grouping of a combination of hardware and software components associated with performing information management operations on electronic data, typically one storage manager **140** and at least one data agent **142** (executing on a client computing device **102**) and at least one media agent **144** (executing on a secondary storage computing device **106**). For instance, the components shown in FIG. 1C may together form an information management cell. Thus, in some configurations, a system **100** may be referred to as an information management cell or a storage operation cell. A given cell may be identified by the identity of its storage manager **140**, which is generally responsible for managing the cell. Multiple cells may be organized hierarchically, so that cells may inherit properties from hierarchically superior cells or be controlled by other cells in

the hierarchy (automatically or otherwise). Alternatively, in some embodiments, cells may inherit or otherwise be associated with information management policies, preferences, information management operational parameters, or other properties or characteristics according to their relative position in a hierarchy of cells. Cells may also be organized hierarchically according to function, geography, architectural considerations, or other factors useful or desirable in performing information management operations. For example, a first cell may represent a geographic segment of an enterprise, such as a Chicago office, and a second cell may represent a different geographic segment, such as a New York City office. Other cells may represent departments within a particular office, e.g., human resources, finance, engineering, etc. Where delineated by function, a first cell may perform one or more first types of information management operations (e.g., one or more first types of secondary copies at a certain frequency), and a second cell may perform one or more second types of information management operations (e.g., one or more second types of secondary copies at a different frequency and under different retention rules). In general, the hierarchical information is maintained by one or more storage managers **140** that manage the respective cells (e.g., in corresponding management database(s) **146**).

Data Agents

[0110] A variety of different applications **110** can operate on a given client computing device **102**, including operating systems, file systems, database applications, e-mail applications, and virtual machines, just to name a few. And, as part of the process of creating and restoring secondary copies **116**, the client computing device **102** may be tasked with processing and preparing the primary data **112** generated by these various applications **110**. Moreover, the nature of the processing/preparation can differ across application types, e.g., due to inherent structural, state, and formatting differences among applications **110** and/or the operating system of client computing device **102**. Each data agent **142** is therefore advantageously configured in some embodiments to assist in the performance of information management operations based on the type of data that is being protected at a client-specific and/or application-specific level.

[0111] Data agent **142** is a component of information system **100** and is generally directed by storage manager **140** to participate in creating or restoring secondary copies **116**. Data agent **142** may be a software program (e.g., in the form of a set of executable binary files) that executes on the same client computing device **102** as the associated application **110** that data agent **142** is configured to protect. Data agent **142** is generally responsible for managing, initiating, or otherwise assisting in the performance of information management operations in reference to its associated application (s) **110** and corresponding primary data **112** which is generated/accessed by the particular application(s) **110**. For instance, data agent **142** may take part in copying, archiving, migrating, and/or replicating of certain primary data **112** stored in the primary storage device(s) **104**. Data agent **142** may receive control information from storage manager **140**, such as commands to transfer copies of data objects and/or metadata to one or more media agents **144**. Data agent **142** also may compress, deduplicate, and encrypt certain primary data **112**, as well as capture application-related metadata before transmitting the processed data to media agent **144**.

Data agent **142** also may receive instructions from storage manager **140** to restore (or assist in restoring) a secondary copy **116** from secondary storage device **108** to primary storage **104**, such that the restored data may be properly accessed by application **110** in a suitable format as though it were primary data **112**.

[0112] Each data agent **142** may be specialized for a particular application **110**. For instance, different individual data agents **142** may be designed to handle Microsoft Exchange data, Lotus Notes data, Microsoft Windows file system data, Microsoft Active Directory Objects data, SQL Server data, SharePoint data, Oracle database data, SAP database data, virtual machines and/or associated data, and other types of data. A file system data agent, for example, may handle data files and/or other file system information. If a client computing device **102** has two or more types of data **112**, a specialized data agent **142** may be used for each data type. For example, to backup, migrate, and/or restore all of the data on a Microsoft Exchange server, the client computing device **102** may use: (1) a Microsoft Exchange Mailbox data agent **142** to back up the Exchange mailboxes; (2) a Microsoft Exchange Database data agent **142** to back up the Exchange databases; (3) a Microsoft Exchange Public Folder data agent **142** to back up the Exchange Public Folders; and (4) a Microsoft Windows File System data agent **142** to back up the file system of client computing device **102**. In this example, these specialized data agents **142** are treated as four separate data agents **142** even though they operate on the same client computing device **102**. Other examples may include archive management data agents such as a migration archiver or a compliance archiver, Quick Recovery® agents, and continuous data replication agents. Application-specific data agents **142** can provide improved performance as compared to generic agents. For instance, because application-specific data agents **142** may only handle data for a single software application, the design, operation, and performance of the data agent **142** can be streamlined. The data agent **142** may therefore execute faster and consume less persistent storage and/or operating memory than data agents designed to generically accommodate multiple different software applications **110**. Each data agent **142** may be configured to access data and/or metadata stored in the primary storage device(s) **104** associated with data agent **142** and its host client computing device **102**, and process the data appropriately. For example, during a secondary copy operation, data agent **142** may arrange or assemble the data and metadata into one or more files having a certain format (e.g., a particular backup or archive format) before transferring the file(s) to a media agent **144** or other component. The file(s) may include a list of files or other metadata. In some embodiments, a data agent **142** may be distributed between client computing device **102** and storage manager **140** (and any other intermediate components) or may be deployed from a remote location or its functions approximated by a remote process that performs some or all of the functions of data agent **142**. In addition, a data agent **142** may perform some functions provided by media agent **144**. Other embodiments may employ one or more generic data agents **142** that can handle and process data from two or more different applications **110**, or that can handle and process multiple data types, instead of or in addition to using specialized data agents **142**. For example, one generic data agent **142** may be used to back up, migrate and restore Microsoft Exchange Mailbox data and Microsoft Exchange

Database data, while another generic data agent may handle Microsoft Exchange Public Folder data and Microsoft Windows File System data.

Media Agents

[0113] As noted, off-loading certain responsibilities from client computing devices **102** to intermediate components such as secondary storage computing device(s) **106** and corresponding media agent(s) **144** can provide a number of benefits including improved performance of client computing device **102**, faster and more reliable information management operations, and enhanced scalability. In one example which will be discussed further below, media agent **144** can act as a local cache of recently-copied data and/or metadata stored to secondary storage device(s) **108**, thus improving restore capabilities and performance for the cached data. Media agent **144** is a component of system **100** and is generally directed by storage manager **140** in creating and restoring secondary copies **116**. Whereas storage manager **140** generally manages system **100** as a whole, media agent **144** provides a portal to certain secondary storage devices **108**, such as by having specialized features for communicating with and accessing certain associated secondary storage device **108**. Media agent **144** may be a software program (e.g., in the form of a set of executable binary files) that executes on a secondary storage computing device **106**. Media agent **144** generally manages, coordinates, and facilitates the transmission of data between a data agent **142** (executing on client computing device **102**) and secondary storage device(s) **108** associated with media agent **144**. For instance, other components in the system may interact with media agent **144** to gain access to data stored on associated secondary storage device(s) **108**, (e.g., to browse, read, write, modify, delete, or restore data). Moreover, media agents **144** can generate and store information relating to characteristics of the stored data and/or metadata, or can generate and store other types of information that generally provides insight into the contents of the secondary storage devices **108**—generally referred to as indexing of the stored secondary copies **116**. Each media agent **144** may operate on a dedicated secondary storage computing device **106**, while in other embodiments a plurality of media agents **144** may operate on the same secondary storage computing device **106**.

[0114] A media agent **144** may be associated with a particular secondary storage device **108** if that media agent **144** is capable of one or more of: routing and/or storing data to the particular secondary storage device **108**; coordinating the routing and/or storing of data to the particular secondary storage device **108**; retrieving data from the particular secondary storage device **108**; coordinating the retrieval of data from the particular secondary storage device **108**; and modifying and/or deleting data retrieved from the particular secondary storage device **108**. Media agent **144** in certain embodiments is physically separate from the associated secondary storage device **108**. For instance, a media agent **144** may operate on a secondary storage computing device **106** in a distinct housing, package, and/or location from the associated secondary storage device **108**. In one example, a media agent **144** operates on a first server computer and is in communication with a secondary storage device(s) **108** operating in a separate rack-mounted RAID-based system. A media agent **144** associated with a particular secondary storage device **108** may instruct secondary storage device

108 to perform an information management task. For instance, a media agent **144** may instruct a tape library to use a robotic arm or other retrieval means to load or eject a certain storage media, and to subsequently archive, migrate, or retrieve data to or from that media, e.g., for the purpose of restoring data to a client computing device **102**. As another example, a secondary storage device **108** may include an array of hard disk drives or solid state drives organized in a RAID configuration, and media agent **144** may forward a logical unit number (LUN) and other appropriate information to the array, which uses the received information to execute the desired secondary copy operation. Media agent **144** may communicate with a secondary storage device **108** via a suitable communications link, such as a SCSI or Fibre Channel link.

[0115] Each media agent **144** may maintain an associated media agent database **152**. Media agent database **152** may be stored to a disk or other storage device (not shown) that is local to the secondary storage computing device **106** on which media agent **144** executes. In other cases, media agent database **152** is stored separately from the host secondary storage computing device **106**. Media agent database **152** can include, among other things, a media agent index **153** (see, e.g., FIG. 1C). In some cases, media agent index **153** does not form a part of and is instead separate from media agent database **152**.

[0116] Media agent index **153** (or “index **153**”) may be a data structure associated with the particular media agent **144** that includes information about the stored data associated with the particular media agent and which may be generated in the course of performing a secondary copy operation or a restore. Index **153** provides a fast and efficient mechanism for locating/browsing secondary copies **116** or other data stored in secondary storage devices **108** without having to access secondary storage device **108** to retrieve the information from there. For instance, for each secondary copy **116**, index **153** may include metadata such as a list of the data objects (e.g., files/subdirectories, database objects, mailbox objects, etc.), a logical path to the secondary copy **116** on the corresponding secondary storage device **108**, location information (e.g., offsets) indicating where the data objects are stored in the secondary storage device **108**, when the data objects were created or modified, etc. Thus, index **153** includes metadata associated with the secondary copies **116** that is readily available for use from media agent **144**. In some embodiments, some or all of the information in index **153** may instead or additionally be stored along with secondary copies **116** in secondary storage device **108**. In some embodiments, a secondary storage device **108** can include sufficient information to enable a “bare metal restore,” where the operating system and/or software applications of a failed client computing device **102** or another target may be automatically restored without manually reinstalling individual software packages (including operating systems).

[0117] Because index **153** may operate as a cache, it can also be referred to as an “index cache.” In such cases, information stored in index cache **153** typically comprises data that reflects certain particulars about relatively recent secondary copy operations.

[0118] After some triggering event, such as after some time elapses or index cache **153** reaches a particular size, certain portions of index cache **153** may be copied or migrated to secondary storage device **108**, e.g., on a least-

recently-used basis. This information may be retrieved and uploaded back into index cache **153** or otherwise restored to media agent **144** to facilitate retrieval of data from the secondary storage device(s) **108**. In some embodiments, the cached information may include format or containerization information related to archives or other files stored on storage device(s) **108**.

[0119] In some alternative embodiments media agent **144** generally acts as a coordinator or facilitator of secondary copy operations between client computing devices **102** and secondary storage devices **108**, but does not actually write the data to secondary storage device **108**. For instance, storage manager **140** (or media agent **144**) may instruct a client computing device **102** and secondary storage device **108** to communicate with one another directly. In such a case, client computing device **102** transmits data directly or via one or more intermediary components to secondary storage device **108** according to the received instructions, and vice versa. Media agent **144** may still receive, process, and/or maintain metadata related to the secondary copy operations, i.e., may continue to build and maintain index **153**. In these embodiments, payload data can flow through media agent **144** for the purposes of populating index **153**, but not for writing to secondary storage device **108**. Media agent **144** and/or other components such as storage manager **140** may in some cases incorporate additional functionality, such as data classification, content indexing, deduplication, encryption, compression, and the like. Further details regarding these and other functions are described below.

Distributed, Scalable Architecture

[0120] As described, certain functions of system **100** can be distributed amongst various physical and/or logical components. For instance, one or more of storage manager **140**, data agents **142**, and media agents **144** may operate on computing devices that are physically separate from one another. This architecture can provide a number of benefits. For instance, hardware and software design choices for each distributed component can be targeted to suit its particular function. The secondary computing devices **106** on which media agents **144** operate can be tailored for interaction with associated secondary storage devices **108** and provide fast index cache operation, among other specific tasks. Similarly, client computing device(s) **102** can be selected to effectively service applications **110** in order to efficiently produce and store primary data **112**. Moreover, in some cases, one or more of the individual components of information management system **100** can be distributed to multiple separate computing devices. As one example, for large file systems where the amount of data stored in management database **146** is relatively large, database **146** may be migrated to or may otherwise reside on a specialized database server (e.g., an SQL server) separate from a server that implements the other functions of storage manager **140**. This distributed configuration can provide added protection because database **146** can be protected with standard database utilities (e.g., SQL log shipping or database replication) independent from other functions of storage manager **140**. Database **146** can be efficiently replicated to a remote site for use in the event of a disaster or other data loss at the primary site. Or database **146** can be replicated to another computing device within the same site, such as to a higher performance

machine in the event that a storage manager host computing device can no longer service the needs of a growing system 100.

[0121] The distributed architecture also provides scalability and efficient component utilization. FIG. 1D shows an embodiment of information management system 100 including a plurality of client computing devices 102 and associated data agents 142 as well as a plurality of secondary storage computing devices 106 and associated media agents 144. Additional components can be added or subtracted based on the evolving needs of system 100. For instance, depending on where bottlenecks are identified, administrators can add additional client computing devices 102, secondary storage computing devices 106, and/or secondary storage devices 108. Moreover, where multiple fungible components are available, load balancing can be implemented to dynamically address identified bottlenecks. As an example, storage manager 140 may dynamically select which media agents 144 and/or secondary storage devices 108 to use for storage operations based on a processing load analysis of media agents 144 and/or secondary storage devices 108, respectively.

[0122] Where system 100 includes multiple media agents 144 (see, e.g., FIG. 1D), a first media agent 144 may provide failover functionality for a second failed media agent 144. In addition, media agents 144 can be dynamically selected to provide load balancing. Each client computing device 102 can communicate with, among other components, any of the media agents 144, e.g., as directed by storage manager 140. And each media agent 144 may communicate with, among other components, any of secondary storage devices 108, e.g., as directed by storage manager 140. Thus, operations can be routed to secondary storage devices 108 in a dynamic and highly flexible manner, to provide load balancing, failover, etc. Further examples of scalable systems capable of dynamic storage operations, load balancing, and failover are provided in U.S. Pat. No. 7,246,207. While distributing functionality amongst multiple computing devices can have certain advantages, in other contexts it can be beneficial to consolidate functionality on the same computing device. In alternative configurations, certain components may reside and execute on the same computing device. As such, in other embodiments, one or more of the components shown in FIG. 1C may be implemented on the same computing device. In one configuration, a storage manager 140, one or more data agents 142, and/or one or more media agents 144 are all implemented on the same computing device. In other embodiments, one or more data agents 142 and one or more media agents 144 are implemented on the same computing device, while storage manager 140 is implemented on a separate computing device, etc. without limitation.

Example Types of Information Management Operations, Including Storage Operations

[0123] In order to protect and leverage stored data, system 100 can be configured to perform a variety of information management operations, which may also be referred to in some cases as storage management operations or storage operations. These operations can generally include (i) data movement operations, (ii) processing and data manipulation operations, and (iii) analysis, reporting, and management operations.

Data Movement Operations, Including Secondary Copy Operations

[0124] Data movement operations are generally storage operations that involve the copying or migration of data between different locations in system 100. For example, data movement operations can include operations in which stored data is copied, migrated, or otherwise transferred from one or more first storage devices to one or more second storage devices, such as from primary storage device(s) 104 to secondary storage device(s) 108, from secondary storage device(s) 108 to different secondary storage device(s) 108, from secondary storage devices 108 to primary storage devices 104, or from primary storage device(s) 104 to different primary storage device(s) 104, or in some cases within the same primary storage device 104 such as within a storage array. Data movement operations can include by way of example, backup operations, archive operations, information lifecycle management operations such as hierarchical storage management operations, replication operations (e.g., continuous data replication), snapshot operations, deduplication or single-instancing operations, auxiliary copy operations, disaster-recovery copy operations, and the like. As will be discussed, some of these operations do not necessarily create distinct copies. Nonetheless, some or all of these operations are generally referred to as “secondary copy operations” for simplicity because they involve secondary copies. Data movement also comprises restoring secondary copies.

Backup Operations

[0125] A backup operation creates a copy of a version of primary data 112 at a particular point in time (e.g., one or more files or other data units). Each subsequent backup copy 116 (which is a form of secondary copy 116) may be maintained independently of the first. A backup generally involves maintaining a version of the copied primary data 112 as well as backup copies 116. Further, a backup copy in some embodiments is generally stored in a form that is different from the native format, e.g., a backup format. This contrasts to the version in primary data 112 which may instead be stored in a format native to the source application (s) 110. In various cases, backup copies can be stored in a format in which the data is compressed, encrypted, deduplicated, and/or otherwise modified from the original native application format. For example, a backup copy may be stored in a compressed backup format that facilitates efficient long-term storage. Backup copies 116 can have relatively long retention periods as compared to primary data 112, which is generally highly changeable. Backup copies 116 may be stored on media with slower retrieval times than primary storage device 104. Some backup copies may have shorter retention periods than some other types of secondary copies 116, such as archive copies (described below). Backups may be stored at an offsite location.

[0126] Backup operations can include full backups, differential backups, incremental backups, “synthetic full” backups, and/or creating a “reference copy.” A full backup (or “standard full backup”) in some embodiments is generally a complete image of the data to be protected. However, because full backup copies can consume a relatively large amount of storage, it can be useful to use a full backup copy as a baseline and afterwards only store changes relative to the full backup copy. A differential backup operation (or

cumulative incremental backup operation) tracks and stores changes that occurred since the last full backup. Differential backups can grow quickly in size, but can restore relatively efficiently because a restore can be completed in some cases using only the full backup copy and the latest differential copy. An incremental backup operation generally tracks and stores changes since the most recent backup copy of any type, which can greatly reduce storage utilization. In some cases, however, restoring can be lengthy compared to full or differential backups because completing a restore operation may involve accessing a full backup in addition to multiple incremental backups. Synthetic full backups generally consolidate data without directly backing up data from the client computing device. A synthetic full backup is created from the most recent full backup (i.e., standard or synthetic) and subsequent incremental and/or differential backups. The resulting synthetic full backup is identical to what would have been created had the last backup for the subclient been a standard full backup. Unlike standard full, incremental, and differential backups, however, a synthetic full backup does not actually transfer data from primary storage to the backup media, because it operates as a backup consolidator. A synthetic full backup extracts the index data of each participating subclient. Using this index data and the previously backed up user data images, it builds new full backup images (e.g., bitmaps, or complete backup copies), one for each subclient. The new backup images consolidate the index and user data stored in the related incremental, differential, and previous full backups into a synthetic backup file that fully represents the subclient (e.g., via pointers) but does not necessarily comprise all its constituent data.

[0127] Any of the above types of backup operations can be at the volume level, file level, or block level. Volume level backup operations generally involve copying of a data volume (e.g., a logical disk or partition) as a whole. In a file-level backup, information management system 100 generally tracks changes to individual files and includes copies of files in the backup copy. For block-level backups, files are broken into constituent blocks, and changes are tracked at the block level. Upon restore, system 100 reassembles the blocks into files in a transparent fashion. Far less data may actually be transferred and copied to secondary storage devices 108 during a file-level copy than a volume-level copy. Likewise, a block-level copy may transfer less data than a file-level copy, resulting in faster execution. However, restoring a relatively higher-granularity copy can result in longer restore times. For instance, when restoring a block-level copy, the process of locating and retrieving constituent blocks can sometimes take longer than restoring file-level backups.

[0128] A reference copy may comprise copy(ies) of selected objects from backed up data, typically to help organize data by keeping contextual information from multiple sources together, and/or help retain specific data for a longer period of time, such as for legal hold needs. A reference copy generally maintains data integrity, and when the data is restored, it may be viewed in the same format as the source data. In some embodiments, a reference copy is based on a specialized client, individual subclient and associated information management policies (e.g., storage policy, retention policy, etc.) that are administered within system 100.

Archive Operations

[0129] Because backup operations generally involve maintaining a version of the copied primary data 112 and also maintaining backup copies in secondary storage device(s) 108, they can consume significant storage capacity. To reduce storage consumption, an archive operation according to certain embodiments creates an archive copy 116 by both copying and removing source data. Or, seen another way, archive operations can involve moving some or all of the source data to the archive destination. Thus, data satisfying criteria for removal (e.g., data of a threshold age or size) may be removed from source storage. The source data may be primary data 112 or a secondary copy 116, depending on the situation. As with backup copies, archive copies can be stored in a format in which the data is compressed, encrypted, deduplicated, and/or otherwise modified from the format of the original application or source copy. In addition, archive copies may be retained for relatively long periods of time (e.g., years) and, in some cases are never deleted. In certain embodiments, archive copies may be made and kept for extended periods in order to meet compliance regulations. Archiving can also serve the purpose of freeing up space in primary storage device(s) 104 and easing the demand on computational resources on client computing device 102. Similarly, when a secondary copy 116 is archived, the archive copy can therefore serve the purpose of freeing up space in the source secondary storage device(s) 108. Examples of data archiving operations are provided in U.S. Pat. No. 7,107,298.

Snapshot Operations

[0130] Snapshot operations can provide a relatively lightweight, efficient mechanism for protecting data. From an end-user viewpoint, a snapshot may be thought of as an “instant” image of primary data 112 at a given point in time, and may include state and/or status information relative to an application 110 that creates/manages primary data 112. In one embodiment, a snapshot may generally capture the directory structure of an object in primary data 112 such as a file or volume or other data set at a particular moment in time and may also preserve file attributes and contents. A snapshot in some cases is created relatively quickly, e.g., substantially instantly, using a minimum amount of file space, but may still function as a conventional file system backup.

[0131] A “hardware snapshot” (or “hardware-based snapshot”) operation occurs where a target storage device (e.g., a primary storage device 104 or a secondary storage device 108) performs the snapshot operation in a self-contained fashion, substantially independently, using hardware, firmware and/or software operating on the storage device itself. For instance, the storage device may perform snapshot operations generally without intervention or oversight from any of the other components of the system 100, e.g., a storage array may generate an “array-created” hardware snapshot and may also manage its storage, integrity, versioning, etc. In this manner, hardware snapshots can off-load other components of system 100 from snapshot processing. An array may receive a request from another component to take a snapshot and then proceed to execute the “hardware snapshot” operations autonomously, preferably reporting success to the requesting component.

[0132] A “software snapshot” (or “software-based snapshot”) operation, on the other hand, occurs where a component in system 100 (e.g., client computing device 102, etc.) implements a software layer that manages the snapshot operation via interaction with the target storage device. For instance, the component executing the snapshot management software layer may derive a set of pointers and/or data that represents the snapshot. The snapshot management software layer may then transmit the same to the target storage device, along with appropriate instructions for writing the snapshot. One example of a software snapshot product is Microsoft Volume Snapshot Service (VSS), which is part of the Microsoft Windows operating system.

[0133] Some types of snapshots do not actually create another physical copy of all the data as it existed at the particular point in time, but may simply create pointers that map files and directories to specific memory locations (e.g., to specific disk blocks) where the data resides as it existed at the particular point in time. For example, a snapshot copy may include a set of pointers derived from the file system or from an application. In some other cases, the snapshot may be created at the block-level, such that creation of the snapshot occurs without awareness of the file system. Each pointer points to a respective stored data block, so that collectively, the set of pointers reflect the storage location and state of the data object (e.g., file(s) or volume(s) or data set(s)) at the point in time when the snapshot copy was created.

[0134] An initial snapshot may use only a small amount of disk space needed to record a mapping or other data structure representing or otherwise tracking the blocks that correspond to the current state of the file system. Additional disk space is usually required only when files and directories change later on. Furthermore, when files change, typically only the pointers which map to blocks are copied, not the blocks themselves. For example for “copy-on-write” snapshots, when a block changes in primary storage, the block is copied to secondary storage or cached in primary storage before the block is overwritten in primary storage, and the pointer to that block is changed to reflect the new location of that block. The snapshot mapping of file system data may also be updated to reflect the changed block(s) at that particular point in time. In some other cases, a snapshot includes a full physical copy of all or substantially all of the data represented by the snapshot. Further examples of snapshot operations are provided in U.S. Pat. No. 7,529,782. A snapshot copy in many cases can be made quickly and without significantly impacting primary computing resources because large amounts of data need not be copied or moved. In some embodiments, a snapshot may exist as a virtual file system, parallel to the actual file system. Users in some cases gain read-only access to the record of files and directories of the snapshot. By electing to restore primary data 112 from a snapshot taken at a given point in time, users may also return the current file system to the state of the file system that existed when the snapshot was taken.

Replication Operations

[0135] Replication is another type of secondary copy operation. Some types of secondary copies 116 periodically capture images of primary data 112 at particular points in time (e.g., backups, archives, and snapshots). However, it can also be useful for recovery purposes to protect primary data 112 in a more continuous fashion, by replicating

primary data 112 substantially as changes occur. In some cases a replication copy can be a mirror copy, for instance, where changes made to primary data 112 are mirrored or substantially immediately copied to another location (e.g., to secondary storage device(s) 108). By copying each write operation to the replication copy, two storage systems are kept synchronized or substantially synchronized so that they are virtually identical at approximately the same time. Where entire disk volumes are mirrored, however, mirroring can require significant amount of storage space and utilizes a large amount of processing resources. According to some embodiments, secondary copy operations are performed on replicated data that represents a recoverable state, or “known good state” of a particular application running on the source system. For instance, in certain embodiments, known good replication copies may be viewed as copies of primary data 112. This feature allows the system to directly access, copy, restore, back up, or otherwise manipulate the replication copies as if they were the “live” primary data 112. This can reduce access time, storage utilization, and impact on source applications 110, among other benefits. Based on known good state information, system 100 can replicate sections of application data that represent a recoverable state rather than rote copying of blocks of data. Examples of replication operations (e.g., continuous data replication) are provided in U.S. Pat. No. 7,617,262.

Deduplication/Single-Instancing Operations

[0136] Deduplication or single-instance storage is useful to reduce the amount of non-primary data. For instance, some or all of the above-described secondary copy operations can involve deduplication in some fashion. New data is read, broken down into data portions of a selected granularity (e.g., sub-file level blocks, files, etc.), compared with corresponding portions that are already in secondary storage, and only new/changed portions are stored. Portions that already exist are represented as pointers to the already-stored data. Thus, a deduplicated secondary copy 116 may comprise actual data portions copied from primary data 112 and may further comprise pointers to already-stored data, which is generally more storage-efficient than a full copy. In order to streamline the comparison process, system 100 may calculate and/or store signatures (e.g., hashes or cryptographically unique IDs) corresponding to the individual source data portions and compare the signatures to already-stored data signatures, instead of comparing entire data portions. In some cases, only a single instance of each data portion is stored, and deduplication operations may therefore be referred to interchangeably as “single-instancing” operations. Depending on the implementation, however, deduplication operations can store more than one instance of certain data portions, yet still significantly reduce stored-data redundancy. Depending on the embodiment, deduplication portions such as data blocks can be of fixed or variable length. Using variable length blocks can enhance deduplication by responding to changes in the data stream, but can involve more complex processing. In some cases, system 100 utilizes a technique for dynamically aligning deduplication blocks based on changing content in the data stream, as described in U.S. Pat. No. 8,364,652.

[0137] System 100 can deduplicate in a variety of manners at a variety of locations. For instance, in some embodiments, system 100 implements “target-side” deduplication by deduplicating data at the media agent 144 after being received

from data agent **142**. In some such cases, media agents **144** are generally configured to manage the deduplication process. For instance, one or more of the media agents **144** maintain a corresponding deduplication database that stores deduplication information (e.g., data block signatures). Examples of such a configuration are provided in U.S. Pat. No. 9,020,900. Instead of or in combination with “target-side” deduplication, “source-side” (or “client-side”) deduplication can also be performed, e.g., to reduce the amount of data to be transmitted by data agent **142** to media agent **144**. Storage manager **140** may communicate with other components within system **100** via network protocols and cloud service provider APIs to facilitate cloud-based deduplication/single instancing, as exemplified in U.S. Pat. No. 8,954,446. Some other deduplication/single instancing techniques are described in U.S. Patent Pub. No. 2006/0224846 and in U.S. Pat. No. 9,098,495.

Information Lifecycle Management and Hierarchical Storage Management

[0138] In some embodiments, files and other data over their lifetime move from more expensive quick-access storage to less expensive slower-access storage. Operations associated with moving data through various tiers of storage are sometimes referred to as information lifecycle management (ILM) operations.

[0139] One type of ILM operation is a hierarchical storage management (HSM) operation, which generally automatically moves data between classes of storage devices, such as from high-cost to low-cost storage devices. For instance, an HSM operation may involve movement of data from primary storage devices **104** to secondary storage devices **108**, or between tiers of secondary storage devices **108**. With each tier, the storage devices may be progressively cheaper, have relatively slower access/restore times, etc. For example, movement of data between tiers may occur as data becomes less important over time. In some embodiments, an HSM operation is similar to archiving in that creating an HSM copy may (though not always) involve deleting some of the source data, e.g., according to one or more criteria related to the source data. For example, an HSM copy may include primary data **112** or a secondary copy **116** that exceeds a given size threshold or a given age threshold. Often, and unlike some types of archive copies, HSM data that is removed or aged from the source is replaced by a logical reference pointer or stub. The reference pointer or stub can be stored in the primary storage device **104** or other source storage device, such as a secondary storage device **108** to replace the deleted source data and to point to or otherwise indicate the new location in (another) secondary storage device **108**.

[0140] For example, files are generally moved between higher and lower cost storage depending on how often the files are accessed. When a user requests access to HSM data that has been removed or migrated, system **100** uses the stub to locate the data and can make recovery of the data appear transparent, even though the HSM data may be stored at a location different from other source data. In this manner, the data appears to the user (e.g., in file system browsing windows and the like) as if it still resides in the source location (e.g., in a primary storage device **104**). The stub may include metadata associated with the corresponding data, so that a file system and/or application can provide some information about the data object and/or a limited-

functionality version (e.g., a preview) of the data object. An HSM copy may be stored in a format other than the native application format (e.g., compressed, encrypted, deduplicated, and/or otherwise modified). In some cases, copies which involve the removal of data from source storage and the maintenance of stub or other logical reference information on source storage may be referred to generally as “on-line archive copies.” On the other hand, copies which involve the removal of data from source storage without the maintenance of stub or other logical reference information on source storage may be referred to as “off-line archive copies.” Examples of HSM and ILM techniques are provided in U.S. Pat. No. 7,343,453.

Auxiliary Copy Operations

[0141] An auxiliary copy generally comprises a copy of an existing secondary copy **116**. For instance, an initial secondary copy **116** may be derived from primary data **112** or from data residing in secondary storage subsystem **118**, whereas an auxiliary copy is generated from the initial secondary copy **116**. Auxiliary copies provide additional standby copies of data and may reside on different secondary storage devices **108** than the initial secondary copies **116**. Thus, auxiliary copies can be used for recovery purposes if initial secondary copies **116** become unavailable. Example auxiliary copy techniques are described in further detail in U.S. Pat. No. 8,230,195.

Disaster-Recovery Copy Operations

[0142] System **100** may also make and retain disaster recovery copies, often as secondary, high-availability disk copies. System **100** may create secondary copies and store them at disaster recovery locations using auxiliary copy or replication operations, such as continuous data replication technologies. Depending on the particular data protection goals, disaster recovery locations can be remote from the client computing devices **102** and primary storage devices **104**, remote from some or all of the secondary storage devices **108**, or both.

Data Manipulation, Including Encryption and Compression

[0143] Data manipulation and processing may include encryption and compression as well as integrity marking and checking, formatting for transmission, formatting for storage, etc. Data may be manipulated “client-side” by data agent **142** as well as “target-side” by media agent **144** in the course of creating secondary copy **116**, or conversely in the course of restoring data from secondary to primary.

Encryption Operations

[0144] System **100** in some cases is configured to process data (e.g., files or other data objects, primary data **112**, secondary copies **116**, etc.), according to an appropriate encryption algorithm (e.g., Blowfish, Advanced Encryption Standard (AES), Triple Data Encryption Standard (3-DES), etc.) to limit access and provide data security. System **100** in some cases encrypts the data at the client level, such that client computing devices **102** (e.g., data agents **142**) encrypt the data prior to transferring it to other components, e.g., before sending the data to media agents **144** during a secondary copy operation. In such cases, client computing device **102** may maintain or have access to an encryption key or passphrase for decrypting the data upon restore.

Encryption can also occur when media agent **144** creates auxiliary copies or archive copies. Encryption may be applied in creating a secondary copy **116** of a previously unencrypted secondary copy **116**, without limitation. In further embodiments, secondary storage devices **108** can implement built-in, high performance hardware-based encryption.

Compression Operations

[0145] Similar to encryption, system **100** may also or alternatively compress data in the course of generating a secondary copy **116**. Compression encodes information such that fewer bits are needed to represent the information as compared to the original representation. Compression techniques are well known in the art. Compression operations may apply one or more data compression algorithms. Compression may be applied in creating a secondary copy **116** of a previously uncompressed secondary copy, e.g., when making archive copies or disaster recovery copies. The use of compression may result in metadata that specifies the nature of the compression, so that data may be uncompressed on restore if appropriate.

Data Analysis, Reporting, and Management Operations

[0146] Data analysis, reporting, and management operations can differ from data movement operations in that they do not necessarily involve copying, migration or other transfer of data between different locations in the system. For instance, data analysis operations may involve processing (e.g., offline processing) or modification of already stored primary data **112** and/or secondary copies **116**. However, in some embodiments data analysis operations are performed in conjunction with data movement operations. Some data analysis operations include content indexing operations and classification operations which can be useful in leveraging data under management to enhance search and other features.

Classification Operations/Content Indexing

[0147] In some embodiments, information management system **100** analyzes and indexes characteristics, content, and metadata associated with primary data **112** (“online content indexing”) and/or secondary copies **116** (“off-line content indexing”). Content indexing can identify files or other data objects based on content (e.g., user-defined keywords or phrases, other keywords/phrases that are not defined by a user, etc.), and/or metadata (e.g., email metadata such as “to,” “from,” “cc,” “bcc,” attachment name, received time, etc.). Content indexes may be searched and search results may be restored. System **100** generally organizes and catalogues the results into a content index, which may be stored within media agent database **152**, for example. The content index can also include the storage locations of or pointer references to indexed data in primary data **112** and/or secondary copies **116**. Results may also be stored elsewhere in system **100** (e.g., in primary storage device **104** or in secondary storage device **108**). Such content index data provides storage manager **140** or other components with an efficient mechanism for locating primary data **112** and/or secondary copies **116** of data objects that match particular criteria, thus greatly increasing the search speed capability of system **100**. For instance, search criteria can be specified by a user through user interface **158**

of storage manager **140**. Moreover, when system **100** analyzes data and/or metadata in secondary copies **116** to create an “off-line content index,” this operation has no significant impact on the performance of client computing devices **102** and thus does not take a toll on the production environment. Examples of content indexing techniques are provided in U.S. Pat. No. 8,170,995.

[0148] One or more components, such as a content index engine, can be configured to scan data and/or associated metadata for classification purposes to populate a database (or other data structure) of information, which can be referred to as a “data classification database” or a “metabase.” Depending on the embodiment, the data classification database(s) can be organized in a variety of different ways, including centralization, logical sub-divisions, and/or physical sub-divisions. For instance, one or more data classification databases may be associated with different subsystems or tiers within system **100**. As an example, there may be a first metabase associated with primary storage subsystem **117** and a second metabase associated with secondary storage subsystem **118**. In other cases, metabase(s) may be associated with individual components, e.g., client computing devices **102** and/or media agents **144**. In some embodiments, a data classification database may reside as one or more data structures within management database **146**, may be otherwise associated with storage manager **140**, and/or may reside as a separate component. In some cases, metabase(s) may be included in separate database(s) and/or on separate storage device(s) from primary data **112** and/or secondary copies **116**, such that operations related to the metabase(s) do not significantly impact performance on other components of system **100**. In other cases, metabase(s) may be stored along with primary data **112** and/or secondary copies **116**. Files or other data objects can be associated with identifiers (e.g., tag entries, etc.) to facilitate searches of stored data objects. Among a number of other benefits, the metabase can also allow efficient, automatic identification of files or other data objects to associate with secondary copy or other information management operations. For instance, a metabase can dramatically improve the speed with which system **100** can search through and identify data as compared to other approaches that involve scanning an entire file system. Examples of metabases and data classification operations are provided in U.S. Pat. Nos. 7,734,669 and 7,747,579.

Management and Reporting Operations

[0149] Certain embodiments leverage the integrated ubiquitous nature of system **100** to provide useful system-wide management and reporting. Operations management can generally include monitoring and managing the health and performance of system **100** by, without limitation, performing error tracking, generating granular storage/performance metrics (e.g., job success/failure information, deduplication efficiency, etc.), generating storage modeling and costing information, and the like. As an example, storage manager **140** or another component in system **100** may analyze traffic patterns and suggest and/or automatically route data to minimize congestion. In some embodiments, the system can generate predictions relating to storage operations or storage operation information. Such predictions, which may be based on a trending analysis, may predict various network operations or resource usage, such as network traffic levels, storage media use, use of bandwidth of communication

links, use of media agent components, etc. Further examples of traffic analysis, trend analysis, prediction generation, and the like are described in U.S. Pat. No. 7,343,453.

[0150] In some configurations having a hierarchy of storage operation cells, a master storage manager **140** may track the status of subordinate cells, such as the status of jobs, system components, system resources, and other items, by communicating with storage managers **140** (or other components) in the respective storage operation cells. Moreover, the master storage manager **140** may also track status by receiving periodic status updates from the storage managers **140** (or other components) in the respective cells regarding jobs, system components, system resources, and other items. In some embodiments, a master storage manager **140** may store status information and other information regarding its associated storage operation cells and other system information in its management database **146** and/or index **150** (or in another location). The master storage manager **140** or other component may also determine whether certain storage-related or other criteria are satisfied, and may perform an action or trigger event (e.g., data migration) in response to the criteria being satisfied, such as where a storage threshold is met for a particular volume, or where inadequate protection exists for certain data. For instance, data from one or more storage operation cells is used to mitigate recognized risks dynamically and automatically, and/or to advise users of risks or suggest actions to mitigate these risks. For example, an information management policy may specify certain requirements (e.g., that a storage device should maintain a certain amount of free space, that secondary copies should occur at a particular interval, that data should be aged and migrated to other storage after a particular period, that data on a secondary volume should always have a certain level of availability and be restorable within a given time period, that data on a secondary volume may be mirrored or otherwise migrated to a specified number of other volumes, etc.). If a risk condition or other criterion is triggered, the system may notify the user of these conditions and may suggest (or automatically implement) a mitigation action to address the risk. For example, the system may indicate that data from a primary copy **112** should be migrated to a secondary storage device **108** to free up space on primary storage device **104**. Examples of the use of risk factors and other triggering criteria are described in U.S. Pat. No. 7,343,453.

[0151] In some embodiments, system **100** may also determine whether a metric or other indication satisfies particular storage criteria sufficient to perform an action. For example, a storage policy or other definition might indicate that a storage manager **140** should initiate a particular action if a storage metric or other indication drops below or otherwise fails to satisfy specified criteria such as a threshold of data protection. In some embodiments, risk factors may be quantified into certain measurable service or risk levels. For example, certain applications and associated data may be considered to be more important relative to other data and services. Financial compliance data, for example, may be of greater importance than marketing materials, etc. Network administrators may assign priority values or “weights” to certain data and/or applications corresponding to the relative importance. The level of compliance of secondary copy operations specified for these applications may also be assigned a certain value. Thus, the health, impact, and overall importance of a service may be determined, such as

by measuring the compliance value and calculating the product of the priority value and the compliance value to determine the “service level” and comparing it to certain operational thresholds to determine whether it is acceptable. Further examples of the service level determination are provided in U.S. Pat. No. 7,343,453.

[0152] System **100** may additionally calculate data costing and data availability associated with information management operation cells. For instance, data received from a cell may be used in conjunction with hardware-related information and other information about system elements to determine the cost of storage and/or the availability of particular data. Example information generated could include how fast a particular department is using up available storage space, how long data would take to recover over a particular pathway from a particular secondary storage device, costs over time, etc. Moreover, in some embodiments, such information may be used to determine or predict the overall cost associated with the storage of certain information. The cost associated with hosting a certain application may be based, at least in part, on the type of media on which the data resides, for example. Storage devices may be assigned to a particular cost categories, for example. Further examples of costing techniques are described in U.S. Pat. No. 7,343,453.

[0153] Any of the above types of information (e.g., information related to trending, predictions, job, cell or component status, risk, service level, costing, etc.) can generally be provided to users via user interface **158** in a single integrated view or console (not shown). Report types may include: scheduling, event management, media management and data aging. Available reports may also include backup history, data aging history, auxiliary copy history, job history, library and drive, media in library, restore history, and storage policy, etc., without limitation. Such reports may be specified and created at a certain point in time as a system analysis, forecasting, or provisioning tool. Integrated reports may also be generated that illustrate storage and performance metrics, risks and storage costing information. Moreover, users may create their own reports based on specific needs. User interface **158** can include an option to graphically depict the various components in the system using appropriate icons. As one example, user interface **158** may provide a graphical depiction of primary storage devices **104**, secondary storage devices **108**, data agents **142** and/or media agents **144**, and their relationship to one another in system **100**.

[0154] In general, the operations management functionality of system **100** can facilitate planning and decision-making. For example, in some embodiments, a user may view the status of some or all jobs as well as the status of each component of information management system **100**. Users may then plan and make decisions based on this data. For instance, a user may view high-level information regarding secondary copy operations for system **100**, such as job status, component status, resource status (e.g., communication pathways, etc.), and other information. The user may also drill down or use other means to obtain more detailed information regarding a particular component, job, or the like. Further examples are provided in U.S. Pat. No. 7,343,453. System **100** can also be configured to perform system-wide e-discovery operations in some embodiments. In general, e-discovery operations provide a unified collection and search capability for data in the system, such as data stored in secondary storage devices **108** (e.g., backups, archives, or

other secondary copies **116**). For example, system **100** may construct and maintain a virtual repository for data stored in system **100** that is integrated across source applications **110**, different storage device types, etc. According to some embodiments, e-discovery utilizes other techniques described herein, such as data classification and/or content indexing.

Information Management Policies

[0155] An information management policy **148** can include a data structure or other information source that specifies a set of parameters (e.g., criteria and rules) associated with secondary copy and/or other information management operations.

[0156] One type of information management policy **148** is a “storage policy.” According to certain embodiments, a storage policy generally comprises a data structure or other information source that defines (or includes information sufficient to determine) a set of preferences or other criteria for performing information management operations. Storage policies can include one or more of the following: (1) what data will be associated with the storage policy, e.g., subclient; (2) a destination to which the data will be stored; (3) datapath information specifying how the data will be communicated to the destination; (4) the type of secondary copy operation to be performed; and (5) retention information specifying how long the data will be retained at the destination (see, e.g., FIG. 1E). Data associated with a storage policy can be logically organized into subclients, which may represent primary data **112** and/or secondary copies **116**. A subclient may represent static or dynamic associations of portions of a data volume. Subclients may represent mutually exclusive portions. Thus, in certain embodiments, a portion of data may be given a label and the association is stored as a static entity in an index, database or other storage location. Subclients may also be used as an effective administrative scheme of organizing data according to data type, department within the enterprise, storage preferences, or the like. Depending on the configuration, subclients can correspond to files, folders, virtual machines, databases, etc. In one example scenario, an administrator may find it preferable to separate e-mail data from financial data using two different subclients.

[0157] A storage policy can define where data is stored by specifying a target or destination storage device (or group of storage devices). For instance, where the secondary storage device **108** includes a group of disk libraries, the storage policy may specify a particular disk library for storing the subclients associated with the policy. As another example, where the secondary storage devices **108** include one or more tape libraries, the storage policy may specify a particular tape library for storing the subclients associated with the storage policy, and may also specify a drive pool and a tape pool defining a group of tape drives and a group of tapes, respectively, for use in storing the subclient data. While information in the storage policy can be statically assigned in some cases, some or all of the information in the storage policy can also be dynamically determined based on criteria set forth in the storage policy. For instance, based on such criteria, a particular destination storage device(s) or other parameter of the storage policy may be determined based on characteristics associated with the data involved in a particular secondary copy operation, device availability (e.g., availability of a secondary storage device **108** or a

media agent **144**), network status and conditions (e.g., identified bottlenecks), user credentials, and the like.

[0158] Datapath information can also be included in the storage policy. For instance, the storage policy may specify network pathways and components to utilize when moving the data to the destination storage device(s). In some embodiments, the storage policy specifies one or more media agents **144** for conveying data associated with the storage policy between the source and destination. A storage policy can also specify the type(s) of associated operations, such as backup, archive, snapshot, auxiliary copy, or the like. Furthermore, retention parameters can specify how long the resulting secondary copies **116** will be kept (e.g., a number of days, months, years, etc.), perhaps depending on organizational needs and/or compliance criteria.

[0159] When adding a new client computing device **102**, administrators can manually configure information management policies **148** and/or other settings, e.g., via user interface **158**. However, this can be an involved process resulting in delays, and it may be desirable to begin data protection operations quickly, without awaiting human intervention. Thus, in some embodiments, system **100** automatically applies a default configuration to client computing device **102**. As one example, when one or more data agent(s) **142** are installed on a client computing device **102**, the installation script may register the client computing device **102** with storage manager **140**, which in turn applies the default configuration to the new client computing device **102**. In this manner, data protection operations can begin substantially immediately. The default configuration can include a default storage policy, for example, and can specify any appropriate information sufficient to begin data protection operations. This can include a type of data protection operation, scheduling information, a target secondary storage device **108**, data path information (e.g., a particular media agent **144**), and the like.

[0160] Another type of information management policy **148** is a “scheduling policy,” which specifies when and how often to perform operations. Scheduling parameters may specify with what frequency (e.g., hourly, weekly, daily, event-based, etc.) or under what triggering conditions secondary copy or other information management operations are to take place. Scheduling policies in some cases are associated with particular components, such as a subclient, client computing device **102**, and the like.

[0161] Another type of information management policy **148** is an “audit policy” (or “security policy”), which comprises preferences, rules and/or criteria that protect sensitive data in system **100**. For example, an audit policy may define “sensitive objects” which are files or data objects that contain particular keywords (e.g., “confidential,” or “privileged”) and/or are associated with particular keywords (e.g., in metadata) or particular flags (e.g., in metadata identifying a document or email as personal, confidential, etc.). An audit policy may further specify rules for handling sensitive objects. As an example, an audit policy may require that a reviewer approve the transfer of any sensitive objects to a cloud storage site, and that if approval is denied for a particular sensitive object, the sensitive object should be transferred to a local primary storage device **104** instead. To facilitate this approval, the audit policy may further specify how a secondary storage computing device **106** or other system component should notify a reviewer that a sensitive object is slated for transfer.

[0162] Another type of information management policy 148 is a “provisioning policy,” which can include preferences, priorities, rules, and/or criteria that specify how client computing devices 102 (or groups thereof) may utilize system resources, such as available storage on cloud storage and/or network bandwidth. A provisioning policy specifies, for example, data quotas for particular client computing devices 102 (e.g., a number of gigabytes that can be stored monthly, quarterly or annually). Storage manager 140 or other components may enforce the provisioning policy. For instance, media agents 144 may enforce the policy when transferring data to secondary storage devices 108. If a client computing device 102 exceeds a quota, a budget for the client computing device 102 (or associated department) may be adjusted accordingly or an alert may trigger.

[0163] While the above types of information management policies 148 are described as separate policies, one or more of these can be generally combined into a single information management policy 148. For instance, a storage policy may also include or otherwise be associated with one or more scheduling, audit, or provisioning policies or operational parameters thereof. Moreover, while storage policies are typically associated with moving and storing data, other policies may be associated with other types of information management operations. The following is a non-exhaustive list of items that information management policies 148 may specify:

- [0164] schedules or other timing information, e.g., specifying when and/or how often to perform information management operations;
 - [0165] the type of secondary copy 116 and/or copy format (e.g., snapshot, backup, archive, HSM, etc.);
 - [0166] a location or a class or quality of storage for storing secondary copies 116 (e.g., one or more particular secondary storage devices 108);
 - [0167] preferences regarding whether and how to encrypt, compress, deduplicate, or otherwise modify or transform secondary copies 116;
 - [0168] which system components and/or network pathways (e.g., preferred media agents 144) should be used to perform secondary storage operations;
 - [0169] resource allocation among different computing devices or other system components used in performing information management operations (e.g., bandwidth allocation, available storage capacity, etc.);
 - [0170] whether and how to synchronize or otherwise distribute files or other data objects across multiple computing devices or hosted services; and
 - [0171] retention information specifying the length of time primary data 112 and/or secondary copies 116 should be retained, e.g., in a particular class or tier of storage devices, or within the system 100.
- [0172] Information management policies 148 can additionally specify or depend on historical or current criteria that may be used to determine which rules to apply to a particular data object, system component, or information management operation, such as:
- [0173] frequency with which primary data 112 or a secondary copy 116 of a data object or metadata has been or is predicted to be used, accessed, or modified;
 - [0174] time-related factors (e.g., aging information such as time since the creation or modification of a data object);

[0175] deduplication information (e.g., hashes, data blocks, deduplication block size, deduplication efficiency or other metrics);

[0176] an estimated or historic usage or cost associated with different components (e.g., with secondary storage devices 108);

[0177] the identity of users, applications 110, client computing devices 102 and/or other computing devices that created, accessed, modified, or otherwise utilized primary data 112 or secondary copies 116;

[0178] a relative sensitivity (e.g., confidentiality, importance) of a data object, e.g., as determined by its content and/or metadata;

[0179] the current or historical storage capacity of various storage devices;

[0180] the current or historical network capacity of network pathways connecting various components within the storage operation cell;

[0181] access control lists or other security information; and

[0182] the content of a particular data object (e.g., its textual content) or of metadata associated with the data object.

Example Storage Policy and Secondary Copy Operations

[0183] FIG. 1E includes a data flow diagram depicting performance of secondary copy operations by an embodiment of information management system 100, according to an example storage policy 148A. System 100 includes a storage manager 140, a client computing device 102 having a file system data agent 142A and an email data agent 142B operating thereon, a primary storage device 104, two media agents 144A, 144B, and two secondary storage devices 108: a disk library 108A and a tape library 108B. As shown, primary storage device 104 includes primary data 112A, which is associated with a logical grouping of data associated with a file system (“file system subclient”), and primary data 112B, which is a logical grouping of data associated with email (“email subclient”). The techniques described with respect to FIG. 1E can be utilized in conjunction with data that is otherwise organized as well. As indicated by the dashed box, the second media agent 144B and tape library 108B are “off-site,” and may be remotely located from the other components in system 100 (e.g., in a different city, office building, etc.). Indeed, “off-site” may refer to a magnetic tape located in remote storage, which must be manually retrieved and loaded into a tape drive to be read. In this manner, information stored on the tape library 108B may provide protection in the event of a disaster or other failure at the main site(s) where data is stored.

[0184] The file system subclient 112A in certain embodiments generally comprises information generated by the file system and/or operating system of client computing device 102, and can include, for example, file system data (e.g., regular files, file tables, mount points, etc.), operating system data (e.g., registries, event logs, etc.), and the like. The e-mail subclient 112B can include data generated by an e-mail application operating on client computing device 102, e.g., mailbox information, folder information, emails, attachments, associated database information, and the like. As described above, the subclients can be logical containers, and the data included in the corresponding primary data 112A and 112B may or may not be stored contiguously. The example storage policy 148A includes backup copy prefer-

ences or rule set 160, disaster recovery copy preferences or rule set 162, and compliance copy preferences or rule set 164. Backup copy rule set 160 specifies that it is associated with file system subclient 166 and email subclient 168. Each of subclients 166 and 168 are associated with the particular client computing device 102. Backup copy rule set 160 further specifies that the backup operation will be written to disk library 108A and designates a particular media agent 144A to convey the data to disk library 108A. Finally, backup copy rule set 160 specifies that backup copies created according to rule set 160 are scheduled to be generated hourly and are to be retained for 30 days. In some other embodiments, scheduling information is not included in storage policy 148A and is instead specified by a separate scheduling policy. Disaster recovery copy rule set 162 is associated with the same two subclients 166 and 168. However, disaster recovery copy rule set 162 is associated with tape library 108B, unlike backup copy rule set 160. Moreover, disaster recovery copy rule set 162 specifies that a different media agent, namely 144B, will convey data to tape library 108B. Disaster recovery copies created according to rule set 162 will be retained for 60 days and will be generated daily. Disaster recovery copies generated according to disaster recovery copy rule set 162 can provide protection in the event of a disaster or other catastrophic data loss that would affect the backup copy 116A maintained on disk library 108A. Compliance copy rule set 164 is only associated with the email subclient 168, and not the file system subclient 166. Compliance copies generated according to compliance copy rule set 164 will therefore not include primary data 112A from the file system subclient 166. For instance, the organization may be under an obligation to store and maintain copies of email data for a particular period of time (e.g., 10 years) to comply with state or federal regulations, while similar regulations do not apply to file system data. Compliance copy rule set 164 is associated with the same tape library 108B and media agent 144B as disaster recovery copy rule set 162, although a different storage device or media agent could be used in other embodiments. Finally, compliance copy rule set 164 specifies that the copies it governs will be generated quarterly and retained for 10 years.

Secondary Copy Jobs

[0185] A logical grouping of secondary copy operations governed by a rule set and being initiated at a point in time may be referred to as a “secondary copy job” (and sometimes may be called a “backup job,” even though it is not necessarily limited to creating only backup copies). Secondary copy jobs may be initiated on demand as well. Steps 1-9 below illustrate three secondary copy jobs based on storage policy 148A.

[0186] Referring to FIG. 1E, at step 1, storage manager 140 initiates a backup job according to the backup copy rule set 160, which logically comprises all the secondary copy operations necessary to effectuate rules 160 in storage policy 148A every hour, including steps 1-4 occurring hourly. For instance, a scheduling service running on storage manager 140 accesses backup copy rule set 160 or a separate scheduling policy associated with client computing device 102 and initiates a backup job on an hourly basis. Thus, at the scheduled time, storage manager 140 sends instructions to client computing device 102 (i.e., to both data agent 142A and data agent 142B) to begin the backup job. At step 2, file

system data agent 142A and email data agent 142B on client computing device 102 respond to instructions from storage manager 140 by accessing and processing the respective subclient primary data 112A and 112B involved in the backup copy operation, which can be found in primary storage device 104. Because the secondary copy operation is a backup copy operation, the data agent(s) 142A, 142B may format the data into a backup format or otherwise process the data suitable for a backup copy. At step 3, client computing device 102 communicates the processed file system data (e.g., using file system data agent 142A) and the processed email data (e.g., using email data agent 142B) to the first media agent 144A according to backup copy rule set 160, as directed by storage manager 140. Storage manager 140 may further keep a record in management database 146 of the association between media agent 144A and one or more of: client computing device 102, file system subclient 112A, file system data agent 142A, email subclient 112B, email data agent 142B, and/or backup copy 116A.

[0187] The target media agent 144A receives the data-agent-processed data from client computing device 102, and at step 4 generates and conveys backup copy 116A to disk library 108A to be stored as backup copy 116A, again at the direction of storage manager 140 and according to backup copy rule set 160. Media agent 144A can also update its index 153 to include data and/or metadata related to backup copy 116A, such as information indicating where the backup copy 116A resides on disk library 108A, where the email copy resides, where the file system copy resides, data and metadata for cache retrieval, etc. Storage manager 140 may similarly update its index 150 to include information relating to the secondary copy operation, such as information relating to the type of operation, a physical location associated with one or more copies created by the operation, the time the operation was performed, status information relating to the operation, the components involved in the operation, and the like. In some cases, storage manager 140 may update its index 150 to include some or all of the information stored in index 153 of media agent 144A. At this point, the backup job may be considered complete. After the 30-day retention period expires, storage manager 140 instructs media agent 144A to delete backup copy 116A from disk library 108A and indexes 150 and/or 153 are updated accordingly. At step 5, storage manager 140 initiates another backup job for a disaster recovery copy according to the disaster recovery rule set 162. Illustratively this includes steps 5-7 occurring daily for creating disaster recovery copy 116B. Illustratively, and by way of illustrating the scalable aspects and off-loading principles embedded in system 100, disaster recovery copy 116B is based on backup copy 116A and not on primary data 112A and 112B. At step 6, illustratively based on instructions received from storage manager 140 at step 5, the specified media agent 144B retrieves the most recent backup copy 116A from disk library 108A. At step 7, again at the direction of storage manager 140 and as specified in disaster recovery copy rule set 162, media agent 144B uses the retrieved data to create a disaster recovery copy 116B and store it to tape library 108B. In some cases, disaster recovery copy 116B is a direct, mirror copy of backup copy 116A, and remains in the backup format. In other embodiments, disaster recovery copy 116B may be further compressed or encrypted, or may be generated in some other manner, such as by using primary data 112A and 112B from primary storage device 104 as sources. The disaster recovery

copy operation is initiated once a day and disaster recovery copies **116B** are deleted after 60 days; indexes **153** and/or **150** are updated accordingly when/after each information management operation is executed and/or completed. The present backup job may be considered completed. At step **8**, storage manager **140** initiates another backup job according to compliance rule set **164**, which performs steps **8-9** quarterly to create compliance copy **116C**. For instance, storage manager **140** instructs media agent **144B** to create compliance copy **116C** on tape library **108B**, as specified in the compliance copy rule set **164**. At step **9** in the example, compliance copy **116C** is generated using disaster recovery copy **116B** as the source. This is efficient, because disaster recovery copy resides on the same secondary storage device and thus no network resources are required to move the data. In other embodiments, compliance copy **116C** is instead generated using primary data **112B** corresponding to the email subclient or using backup copy **116A** from disk library **108A** as source data. As specified in the illustrated example, compliance copies **116C** are created quarterly, and are deleted after ten years, and indexes **153** and/or **150** are kept up-to-date accordingly.

Example Applications of Storage Policies-Information Governance Policies and Classification

[0188] Again referring to FIG. 1E, storage manager **140** may permit a user to specify aspects of storage policy **148A**. For example, the storage policy can be modified to include information governance policies to define how data should be managed in order to comply with a certain regulation or business objective. The various policies may be stored, for example, in management database **146**. An information governance policy may align with one or more compliance tasks that are imposed by regulations or business requirements. Examples of information governance policies might include a Sarbanes-Oxley policy, a HIPAA policy, an electronic discovery (e-discovery) policy, and so on. Information governance policies allow administrators to obtain different perspectives on an organization's online and offline data, without the need for a dedicated data silo created solely for each different viewpoint. As described previously, the data storage systems herein build an index that reflects the contents of a distributed data set that spans numerous clients and storage devices, including both primary data and secondary copies, and online and offline copies. An organization may apply multiple information governance policies in a top-down manner over that unified data set and indexing schema in order to view and manipulate the data set through different lenses, each of which is adapted to a particular compliance or business goal. Thus, for example, by applying an e-discovery policy and a Sarbanes-Oxley policy, two different groups of users in an organization can conduct two very different analyses of the same underlying physical set of data/copies, which may be distributed throughout the information management system.

[0189] An information governance policy may comprise a classification policy, which defines a taxonomy of classification terms or tags relevant to a compliance task and/or business objective. A classification policy may also associate a defined tag with a classification rule. A classification rule defines a particular combination of criteria, such as users who have created, accessed or modified a document or data object; file or application types; content or metadata keywords; clients or storage locations; dates of data creation

and/or access; review status or other status within a workflow (e.g., reviewed or un-reviewed); modification times or types of modifications; and/or any other data attributes in any combination, without limitation. A classification rule may also be defined using other classification tags in the taxonomy. The various criteria used to define a classification rule may be combined in any suitable fashion, for example, via Boolean operators, to define a complex classification rule. As an example, an e-discovery classification policy might define a classification tag "privileged" that is associated with documents or data objects that (1) were created or modified by legal department staff, or (2) were sent to or received from outside counsel via email, or (3) contain one of the following keywords: "privileged" or "attorney" or "counsel," or other like terms. Accordingly, all these documents or data objects will be classified as "privileged."

[0190] One specific type of classification tag, which may be added to an index at the time of indexing, is an "entity tag." An entity tag may be, for example, any content that matches a defined data mask format. Examples of entity tags might include, e.g., social security numbers (e.g., any numerical content matching the formatting mask XXX-XX-XXXX), credit card numbers (e.g., content having a 13-16 digit string of numbers), SKU numbers, product numbers, etc. A user may define a classification policy by indicating criteria, parameters or descriptors of the policy via a graphical user interface, such as a form or page with fields to be filled in, pull-down menus or entries allowing one or more of several options to be selected, buttons, sliders, hypertext links or other known user interface tools for receiving user input, etc. For example, a user may define certain entity tags, such as a particular product number or project ID. In some implementations, the classification policy can be implemented using cloud-based techniques. For example, the storage devices may be cloud storage devices, and the storage manager **140** may execute cloud service provider API over a network to classify data stored on cloud storage devices.

Restore Operations from Secondary Copies

[0191] While not shown in FIG. 1E, at some later point in time, a restore operation can be initiated involving one or more of secondary copies **116A**, **116B**, and **116C**. A restore operation logically takes a selected secondary copy **116**, reverses the effects of the secondary copy operation that created it, and stores the restored data to primary storage where a client computing device **102** may properly access it as primary data. A media agent **144** and an appropriate data agent **142** (e.g., executing on the client computing device **102**) perform the tasks needed to complete a restore operation. For example, data that was encrypted, compressed, and/or deduplicated in the creation of secondary copy **116** will be correspondingly rehydrated (reversing deduplication), uncompressed, and unencrypted into a format appropriate to primary data. Metadata stored within or associated with the secondary copy **116** may be used during the restore operation. In general, restored data should be indistinguishable from other primary data **112**. Preferably, the restored data has fully regained the native format that may make it immediately usable by application **110**.

[0192] As one example, a user may manually initiate a restore of backup copy **116A**, e.g., by interacting with user interface **158** of storage manager **140** or with a web-based console with access to system **100**. Storage manager **140** may access data in its index **150** and/or management

database **146** (and/or the respective storage policy **148A**) associated with the selected backup copy **116A** to identify the appropriate media agent **144A** and/or secondary storage device **108A** where the secondary copy resides. The user may be presented with a representation (e.g., stub, thumbnail, listing, etc.) and metadata about the selected secondary copy, in order to determine whether this is the appropriate copy to be restored, e.g., date that the original primary data was created. Storage manager **140** will then instruct media agent **144A** and an appropriate data agent **142** on the target client computing device **102** to restore secondary copy **116A** to primary storage device **104**. A media agent may be selected for use in the restore operation based on a load balancing algorithm, an availability based algorithm, or other criteria. The selected media agent, e.g., **144A**, retrieves secondary copy **116A** from disk library **108A**. For instance, media agent **144A** may access its index **153** to identify a location of backup copy **116A** on disk library **108A**, or may access location information residing on disk library **108A** itself.

[0193] In some cases, a backup copy **116A** that was recently created or accessed, may be cached to speed up the restore operation. In such a case, media agent **144A** accesses a cached version of all or part of backup copy **116A** residing in index **153**, without having to access disk library **108A** for some or all of the data. Once it has retrieved backup copy **116A**, the media agent **144A** communicates the data to the requesting client computing device **102**. Upon receipt, file system data agent **142A** and email data agent **142B** may unpack (e.g., restore from a backup format to the native application format) the data in backup copy **116A** and restore the unpackaged data to primary storage device **104**. In general, secondary copies **116** may be restored to the same volume or folder in primary storage device **104** from which the secondary copy was derived; to another storage location or client computing device **102**; to shared storage, etc. In some cases, the data may be restored so that it may be used by an application **110** of a different version/vintage from the application that created the original primary data **112**.

Example Secondary Copy Formatting

[0194] The formatting and structure of secondary copies **116** can vary depending on the embodiment. In some cases, secondary copies **116** are formatted as a series of logical data units or “chunks” (e.g., 512 MB, 1 GB, 2 GB, 4 GB, or 8 GB chunks). This can facilitate efficient communication and writing to secondary storage devices **108**, e.g., according to resource availability. For example, a single secondary copy **116** may be written on a chunk-by-chunk basis to one or more secondary storage devices **108**. In some cases, users can select different chunk sizes, e.g., to improve throughput to tape storage devices. Generally, each chunk can include a header and a payload. The payload can include files (or other data units) or subsets thereof included in the chunk, whereas the chunk header generally includes metadata relating to the chunk, some or all of which may be derived from the payload. For example, during a secondary copy operation, media agent **144**, storage manager **140**, or other component may divide files into chunks and generate headers for each chunk by processing the files. Headers can include a variety of information such as file and/or volume identifier(s), offset(s), and/or other information associated with the payload data items, a chunk sequence number, etc. Importantly,

in addition to being stored with secondary copy **116** on secondary storage device **108**, chunk headers can also be stored to index **153** of the associated media agent(s) **144** and/or to index **150** associated with storage manager **140**. This can be useful for providing faster processing of secondary copies **116** during browsing, restores, or other operations. In some cases, once a chunk is successfully transferred to a secondary storage device **108**, the secondary storage device **108** returns an indication of receipt, e.g., to media agent **144** and/or storage manager **140**, which may update their respective indexes **153**, **150** accordingly. During restore, chunks may be processed (e.g., by media agent **144**) according to the information in the chunk header to reassemble the files.

[0195] Data can also be communicated within system **100** in data channels that connect client computing devices **102** to secondary storage devices **108**. These data channels can be referred to as “data streams,” and multiple data streams can be employed to parallelize an information management operation, improving data transfer rate, among other advantages. Example data formatting techniques including techniques involving data streaming, chunking, and the use of other data structures in creating secondary copies are described in U.S. Pat. Nos. 7,315,923, 8,156,086, and 8,578,120. FIGS. 1F and 1G are diagrams of example data streams **170** and **171**, respectively, which may be employed for performing information management operations. Referring to FIG. 1F, data agent **142** forms data stream **170** from source data associated with a client computing device **102** (e.g., primary data **112**). Data stream **170** is composed of multiple pairs of stream header **172** and stream data (or stream payload) **174**. Data streams **170** and **171** shown in the illustrated example are for a single-instanced storage operation, and a stream payload **174** therefore may include both single-instance (SI) data and/or non-SI data. A stream header **172** includes metadata about the stream payload **174**. This metadata may include, for example, a length of the stream payload **174**, an indication of whether the stream payload **174** is encrypted, an indication of whether the stream payload **174** is compressed, an archive file identifier (ID), an indication of whether the stream payload **174** is single instanceable, and an indication of whether the stream payload **174** is a start of a block of data.

[0196] Referring to FIG. 1G, data stream **171** has the stream header **172** and stream payload **174** aligned into multiple data blocks. In this example, the data blocks are of size 64 KB. The first two stream header **172** and stream payload **174** pairs comprise a first data block of size 64 KB. The first stream header **172** indicates that the length of the succeeding stream payload **174** is 63 KB and that it is the start of a data block. The next stream header **172** indicates that the succeeding stream payload **174** has a length of 1 KB and that it is not the start of a new data block. Immediately following stream payload **174** is a pair comprising an identifier header **176** and identifier data **178**. The identifier header **176** includes an indication that the succeeding identifier data **178** includes the identifier for the immediately previous data block. The identifier data **178** includes the identifier that the data agent **142** generated for the data block. The data stream **171** also includes other stream header **172** and stream payload **174** pairs, which may be for SI data and/or non-SI data.

[0197] FIG. 1H is a diagram illustrating data structures **180** that may be used to store blocks of SI data and non-SI

data on a storage device (e.g., secondary storage device **108**). According to certain embodiments, data structures **180** do not form part of a native file system of the storage device. Data structures **180** include one or more volume folders **182**, one or more chunk folders **184/185** within the volume folder **182**, and multiple files within chunk folder **184**. Each chunk folder **184/185** includes a metadata file **186/187**, a metadata index file **188/189**, one or more container files **190/191/193**, and a container index file **192/194**. Metadata file **186/187** stores non-SI data blocks as well as links to SI data blocks stored in container files. Metadata index file **188/189** stores an index to the data in the metadata file **186/187**. Container files **190/191/193** store SI data blocks. Container index file **192/194** stores an index to container files **190/191/193**. Among other things, container index file **192/194** stores an indication of whether a corresponding block in a container file **190/191/193** is referred to by a link in a metadata file **186/187**. For example, data block B2 in the container file **190** is referred to by a link in metadata file **187** in chunk folder **185**. Accordingly, the corresponding index entry in container index file **192** indicates that data block B2 in container file **190** is referred to. As another example, data block B1 in container file **191** is referred to by a link in metadata file **187**, and so the corresponding index entry in container index file **192** indicates that this data block is referred to.

[0198] As an example, data structures **180** illustrated in FIG. 1H may have been created as a result of separate secondary copy operations involving two client computing devices **102**. For example, a first secondary copy operation on a first client computing device **102** could result in the creation of the first chunk folder **184**, and a second secondary copy operation on a second client computing device **102** could result in the creation of the second chunk folder **185**. Container files **190/191** in the first chunk folder **184** would contain the blocks of SI data of the first client computing device **102**. If the two client computing devices **102** have substantially similar data, the second secondary copy operation on the data of the second client computing device **102** would result in media agent **144** storing primarily links to the data blocks of the first client computing device **102** that are already stored in the container files **190/191**. Accordingly, while a first secondary copy operation may result in storing nearly all of the data subject to the operation, subsequent secondary storage operations involving similar data may result in substantial data storage space savings, because links to already stored data blocks can be stored instead of additional instances of data blocks.

[0199] If the operating system of the secondary storage computing device **106** on which media agent **144** operates supports sparse files, then when media agent **144** creates container files **190/191/193**, it can create them as sparse files. A sparse file is a type of file that may include empty space (e.g., a sparse file may have real data within it, such as at the beginning of the file and/or at the end of the file, but may also have empty space in it that is not storing actual data, such as a contiguous range of bytes all having a value of zero). Having container files **190/191/193** be sparse files allows media agent **144** to free up space in container files **190/191/193** when blocks of data in container files **190/191/193** no longer need to be stored on the storage devices. In some examples, media agent **144** creates a new container file **190/191/193** when a container file **190/191/193** either includes 100 blocks of data or when the size of the container

file **190** exceeds 50 MB. In other examples, media agent **144** creates a new container file **190/191/193** when a container file **190/191/193** satisfies other criteria (e.g., it contains from approx. 100 to approx. 1000 blocks or when its size exceeds approximately 50 MB to 1 GB). In some cases, a file on which a secondary copy operation is performed may comprise a large number of data blocks. For example, a 100 MB file may comprise 400 data blocks of size 256 KB. If such a file is to be stored, its data blocks may span more than one container file, or even more than one chunk folder. As another example, a database file of 20 GB may comprise over 40,000 data blocks of size 512 KB. If such a database file is to be stored, its data blocks will likely span multiple container files, multiple chunk folders, and potentially multiple volume folders. Restoring such files may require accessing multiple container files, chunk folders, and/or volume folders to obtain the requisite data blocks.

Using Backup Data for Replication and Disaster Recovery (“Live Synchronization”)

[0200] There is an increased demand to off-load resource intensive information management tasks (e.g., data replication tasks) away from production devices (e.g., physical or virtual client computing devices) in order to maximize production efficiency. At the same time, enterprises expect access to readily-available up-to-date recovery copies in the event of failure, with little or no production downtime.

[0201] FIG. 2A illustrates a system **200** configured to address these and other issues by using backup or other secondary copy data to synchronize a source subsystem **201** (e.g., a production site) with a destination subsystem **203** (e.g., a failover site). Such a technique can be referred to as “live synchronization” and/or “live synchronization replication.” In the illustrated embodiment, the source client computing devices **202a** include one or more virtual machines (or “VMs”) executing on one or more corresponding VM host computers **205a**, though the source need not be virtualized. The destination site **203** may be at a location that is remote from the production site **201**, or may be located in the same data center, without limitation. One or more of the production site **201** and destination site **203** may reside at data centers at known geographic locations, or alternatively may operate “in the cloud.” The synchronization can be achieved by generally applying an ongoing stream of incremental backups from the source subsystem **201** to the destination subsystem **203**, such as according to what can be referred to as an “incremental forever” approach. FIG. 2A illustrates an embodiment of a data flow which may be orchestrated at the direction of one or more storage managers (not shown). At step 1, the source data agent(s) **242a** and source media agent(s) **244a** work together to write backup or other secondary copies of the primary data generated by the source client computing devices **202a** into the source secondary storage device(s) **208a**. At step 2, the backup/secondary copies are retrieved by the source media agent(s) **244a** from secondary storage. At step 3, source media agent(s) **244a** communicate the backup/secondary copies across a network to the destination media agent(s) **244b** in destination subsystem **203**. As shown, the data can be copied from source to destination in an incremental fashion, such that only changed blocks are transmitted, and in some cases multiple incremental backups are consolidated at the source so that only the most current changed blocks are transmitted to and applied at the destination. An example of live

synchronization of virtual machines using the “incremental forever” approach is found in U.S. Pat. No. 10,228,962 entitled “Live Synchronization and Management of Virtual Machines across Computing and Virtualization Platforms and Using Live Synchronization to Support Disaster Recovery.” Moreover, a deduplicated copy can be employed to further reduce network traffic from source to destination. For instance, the system can utilize the deduplicated copy techniques described in U.S. Pat. No. 9,239,687, entitled “Systems and Methods for Retaining and Using Data Block Signatures in Data Protection Operations.”

[0202] At step 4, destination media agent(s) 244b write the received backup/secondary copy data to the destination secondary storage device(s) 208b. At step 5, the synchronization is completed when the destination media agent(s) and destination data agent(s) 242b restore the backup/secondary copy data to the destination client computing device(s) 202b. The destination client computing device(s) 202b may be kept “warm” awaiting activation in case failure is detected at the source. This synchronization/replication process can incorporate the techniques described in U.S. Patent Pub. No. 2016/0350391 entitled “Replication Using Deduplicated Secondary Copy Data.” Where the incremental backups are applied on a frequent, on-going basis, the synchronized copies can be viewed as mirror or replication copies. Moreover, by applying the incremental backups to the destination site 203 using backup or other secondary copy data, the production site 201 is not burdened with the synchronization operations. Because the destination site 203 can be maintained in a synchronized “warm” state, the downtime for switching over from the production site 201 to the destination site 203 is substantially less than with a typical restore from secondary storage. Thus, the production site 201 may flexibly and efficiently fail over, with minimal downtime and with relatively up-to-date data, to a destination site 203, such as a cloud-based failover site. The destination site 203 can later be reverse synchronized back to the production site 201, such as after repairs have been implemented or after the failure has passed.

Integrating with the Cloud Using File System Protocols

[0203] Given the ubiquity of cloud computing, it can be increasingly useful to provide data protection and other information management services in a scalable, transparent, and highly plug-able fashion. FIG. 2B illustrates an information management system 200 having an architecture that provides such advantages, and incorporates use of a standard file system protocol between primary storage subsystem 217 and secondary storage subsystem 218. As shown, the use of the Network File System (NFS) protocol (or any another appropriate file system protocol such as that of the Common Internet File System (CIFS)) allows data agent 242 to operate in the secondary storage subsystem 218. For instance, as indicated by the dashed box 206 around data agent 242 and media agent 244, data agent 242 can co-reside with media agent 244 on the same server (e.g., a secondary storage computing device such as component 106), or in some other location in secondary storage subsystem 218.

[0204] Where NFS is used, for example, secondary storage subsystem 218 allocates an NFS network path to the client computing device 202 or to one or more target applications 210 running on client computing device 202. During a backup or other secondary copy operation, the client computing device 202 mounts the designated NFS path and writes data to that NFS path. The NFS path may be

obtained from NFS path data 215 stored locally at the client computing device 202, and which may be a copy of or otherwise derived from NFS path data 219 stored in the secondary storage subsystem 218. Write requests issued by client computing device(s) 202 are received by data agent 242 in secondary storage subsystem 218, which translates the requests and works in conjunction with media agent 244 to process and write data to a secondary storage device(s) 208, thereby creating a backup or other secondary copy. Storage manager 240 can include a pseudo-client manager 216, which coordinates the process by, among other things, communicating information relating to client computing device 202 and application 210 (e.g., application type, client computing device identifier, etc.) to data agent 242, obtaining appropriate NFS path data from the data agent 242 (e.g., NFS path information), and delivering such data to client computing device 202. Conversely, during a restore or recovery operation, client computing device 202 reads from the designated NFS network path, and the read request is translated by data agent 242. The data agent 242 then works with media agent 244 to retrieve, re-process (e.g., re-hydrate, decompress, decrypt), and forward the requested data to client computing device 202 using NFS.

[0205] By moving specialized software associated with system 200 such as data agent 242 off the client computing devices 202, the illustrative architecture effectively decouples the client computing devices 202 from the installed components of system 200, improving both scalability and plug-ability of system 200. Indeed, the secondary storage subsystem 218 in such environments can be treated simply as a read/write NFS target for primary storage subsystem 217, without the need for information management software to be installed on client computing devices 202. As one example, an enterprise implementing a cloud production computing environment can add VM client computing devices 202 without installing and configuring specialized information management software on these VMs. Rather, backups and restores are achieved transparently, where the new VMs simply write to and read from the designated NFS path. An example of integrating with the cloud using file system protocols or so-called “infinite backup” using NFS share is found in U.S. Patent Pub. No. 2017/0235647 entitled “Data Protection Operations Based on Network Path Information.” Examples of improved data restoration scenarios based on network-path information, including using stored backups effectively as primary data sources, may be found in U.S. Pat. No. 10,684,924 entitled “Data Restoration Operations Based on Network Path Information.”

Highly Scalable Managed Data Pool Architecture

[0206] Enterprises are seeing explosive data growth in recent years, often from various applications running in geographically distributed locations. FIG. 2C shows a block diagram of an example of a highly scalable, managed data pool architecture useful in accommodating such data growth. The illustrated system 200, which may be referred to as a “web-scale” architecture according to certain embodiments, can be readily incorporated into both open compute/storage and common-cloud architectures. The illustrated system 200 includes a grid 245 of media agents 244 logically organized into a control tier 231 and a secondary or storage tier 233. Media agents assigned to the storage tier 233 can be configured to manage a secondary

storage pool 208 as a deduplication store, and be configured to receive client write and read requests from the primary storage subsystem 217, and direct those requests to the secondary tier 233 for servicing. For instance, media agents CMA1-CMA3 in the control tier 231 maintain and consult one or more deduplication databases 247, which can include deduplication information (e.g., data block hashes, data block links, file containers for deduplicated files, etc.) sufficient to read deduplicated files from secondary storage pool 208 and write deduplicated files to secondary storage pool 208. For instance, system 200 can incorporate any of the deduplication systems and methods shown and described in U.S. Pat. No. 9,020,900, entitled “Distributed Deduplicated Storage System,” and U.S. Pat. No. 9,633,033 entitled “High Availability Distributed Deduplicated Storage System.”

[0207] Media agents SMA1-SMA6 assigned to the secondary tier 233 receive write and read requests from media agents CMA1-CMA3 in control tier 231, and access secondary storage pool 208 to service those requests. Media agents CMA1-CMA3 in control tier 231 can also communicate with secondary storage pool 208, and may execute read and write requests themselves (e.g., in response to requests from other control media agents CMA1-CMA3) in addition to issuing requests to media agents in secondary tier 233. Moreover, while shown as separate from the secondary storage pool 208, deduplication database(s) 247 can in some cases reside in storage devices in secondary storage pool 208. As shown, each of the media agents 244 (e.g., CMA1-CMA3, SMA1-SMA6, etc.) in grid 245 can be allocated a corresponding dedicated partition 251A-251I, respectively, in secondary storage pool 208. Each partition 251 can include a first portion 253 containing data associated with (e.g., stored by) media agent 244 corresponding to the respective partition 251. System 200 can also implement a desired level of replication, thereby providing redundancy in the event of a failure of a media agent 244 in grid 245. Along these lines, each partition 251 can further include a second portion 255 storing one or more replication copies of the data associated with one or more other media agents 244 in the grid.

[0208] System 200 can also be configured to allow for seamless addition of media agents 244 to grid 245 via automatic configuration. As one illustrative example, a storage manager (not shown) or other appropriate component may determine that it is appropriate to add an additional node to control tier 231, and perform some or all of the following: (i) assess the capabilities of a newly added or otherwise available computing device as satisfying a minimum criteria to be configured as or hosting a media agent in control tier 231; (ii) confirm that a sufficient amount of the appropriate type of storage exists to support an additional node in control tier 231 (e.g., enough disk drive capacity exists in storage pool 208 to support an additional deduplication database 247); (iii) install appropriate media agent software on the computing device and configure the computing device according to a pre-determined template; (iv) establish a partition 251 in the storage pool 208 dedicated to the newly established media agent 244; and (v) build any appropriate data structures (e.g., an instance of deduplication database 247). An example of highly scalable managed data pool architecture or so-called web-scale architecture for storage and data management is found in U.S. Pat. No.

10,255,143 entitled “Deduplication Replication In A Distributed Deduplication Data Storage System.”

[0209] The embodiments and components thereof disclosed in FIGS. 2A, 2B, and 2C, as well as those in FIGS. 1A-1H, may be implemented in any combination and permutation to satisfy data storage management and information management needs at one or more locations and/or data centers.

Improved Ransomware Detection Accuracy Based on Machine Learning Analysis of Filename Extension Patterns

[0210] All references herein to computing devices, servers, laptops, and the like may be implemented in some embodiments, in whole or in part, as compute resources provided by a cloud computing environment. Likewise, all references to data storage devices may be implemented in some embodiments, in whole or in part, as data storage resources supplied by a cloud computing or cloud storage environment. In a cloud computing environment, any computing device described herein is deployed as a compute resource of the cloud computing environment (e.g., a virtual machine instance, a pod in a Kubernetes cluster or in another application orchestrator, etc.). Although the compute resource is accessed as a service, it is provided by one or more hardware processors and associated computer memory. Likewise, in a cloud computing environment, any data storage described herein may be deployed as a cloud storage service of the cloud computing environment (e.g., “blob storage” on Microsoft Azure, etc.). Although the storage is accessed as a service, it is provided by one or more data storage devices.

[0211] A filename extension is well-known in the art to mean a suffix to the name of a computer file (e.g., .txt, .docx, .ppt, .jpg). The filename extension may indicate an association with an application or workload that created the file, e.g., Microsoft Word, Microsoft PowerPoint, etc. or may indicate a characteristic of the file contents, e.g., picture, movie, text, type of encoding, etc.

[0212] Thus, a data file that becomes encrypted, or converted, or otherwise infected by malware may experience a change in its filename extension, and the malware deletes or hides the original uninfected file. If this happens, the number of files counted by a file system scan may not have changed, but the number of different filename extensions has, thanks to the malware-induced change in the filename extension. This phenomenon is exploited by the analysis disclosed herein. In the present disclosure, a change from one filename extension to another filename extension is also referred to as a “rename” even though in some cases the rename may be legitimate and not a malware symptom. Generally, the disclosed approach is concerned with detecting substantially increased counts of one filename extension, accompanied by corresponding drops in the count of other filename extensions, which suggests that ransomware or malware may have caused renames or file conversions among the existing files. Because the disclosed approach is concerned with changes in total counts of different filename extensions, and does not follow the activity of each individual file, what looks like a rename may be in fact a legitimate file addition or file deletion activity that did not actually change filename extensions on existing files. Thus, changes detected when counting filename extensions should be regarded initially as “potential renames” until such time as the dataset is analyzed. The potential renames are used in the pattern analysis

to determine whether the potential renames calculated for the current backup job conform to the longer-term pattern seen in the file system.

[0213] The number of potential renames of filename extensions calculated in a backup job is based on changed counts among the filename extension counts taken in the backup job as compared to counts taken in a preceding backup job. In some embodiments, the number of potential renames of filename extensions is based on a difference between: (i) how many increases in filename extensions were counted in the current backup job compared to counts taken in a preceding job, and (ii) how many additional files were counted in the current backup job compared to how many were counted in the preceding backup job.

[0214] According to the illustrative data storage management system, every file system backup job includes a file system scan, whether to identify all files for a full backup or whether to identify changes since a preceding backup for an incremental or differential backup job. Thus, the file system scan identifies all files and directory structures in the file system (or in a targeted subset of the file system, like a given drive or directory). The present approach leverages the file system scan to build and maintain an internal “extensions database” that tracks how many files in the scanned file system are associated with each distinct filename extension found in the file system scan. For example a file system may comprise 100,000.pdf files and 200,000.docx files and 300,000.xlsx files. The extensions database keeps counts of how many files the scan found for each distinct filename extension, i.e., .pdf, .docx, and .xlsx. These counts are stored in the extensions database, and will accumulate over time as more and more backup jobs cause the file system to be scanned. The counts are taken with each successive backup job, so that they represent successive point-in-time views of the file system. The extensions database is thoroughly protected, so that its important information can be used with confidence in the anomaly detection analysis.

[0215] The disclosed anomaly detection analysis evaluates changes in the distribution among different filename extensions, relative to each other. The anomaly analysis compares filename extension counts gathered in a current backup job to counts gathered in an earlier job, e.g., one day previously. So, for example, if after yesterday’s backup job malware attacked a file system, encrypted a substantial number of files, and changed their filename extensions, the disclosed analysis will pick up a spike in a certain filename extension (which resulted from the encryption), and will simultaneously detect a sharp drop in other filename extensions, as those files became malware-encrypted and lost their native extensions. Thus, as between yesterday’s backup job and today’s backup job, the appearance of an all-new filename extension or a substantial spike in an existing filename extension that is coupled with a substantial drop in other existing extensions will indicate an anomaly compared to the ordinary day-to-day fluctuation pattern. The anomaly may be a symptom of a malware attack. One could imagine a scenario where a user intentionally and legitimately encrypts a lot of files, or receives a major download of files, or some other such large-scale activity occurs that skews the relative counts from one day to the next. If the relative changes are substantial, an anomaly alert will issue, and the phenomenon should be evaluated by a human analyst to determine whether the behavior is legitimate. Thus, according to the disclosed approach, a sudden and substantial

change in the distribution of filename extensions, as determined in the course of performing a backup job of the file system, rather than realtime monitoring of changes in the file system, will trigger an anomaly alert.

[0216] Preferably, machine learning techniques are used to analyze the distributions of filename extensions and to determine patterns of filename extension fluctuations over time. However, machine learning requires several rounds (illustratively, at least 30 backup jobs of the same file system), in order to gather enough data points for the machine learning to be meaningful and accurate. Thus a substantial history of filename extension counts taken in past backup jobs is needed to train the machine learning model and make it useful. To provide useful analytics before machine learning can be meaningfully applied, the present inventors use a so-called “static algorithm” or “static model” that analyzes filename extension distributions when fewer historical data points are available. The static model generates an anomaly warning or alert based on proprietary threshold percentages of potential renames that are detected relative to an earlier backup job. Thus, for example, if the count of potential renames is calculated at above 10% of all filename extensions counted in the file system scan, an anomaly may be declared. In the longer term, it may turn out that ten percent is a normal fluctuation pattern of this particular file system, which the machine learning would determine over time. For now though, the static model will raise an alert that can be examined by an analyst. After a sufficient number of backup jobs have completed on a target file system, the illustrative system has gathered enough data history and, going forward, applies machine learning to the anomaly analysis.

[0217] The system is also configured to use the same anomaly detection principles to detect slow-moving malware that takes days or even weeks to infect a file system. If the time gap between two jobs being compared by the anomaly detection analysis is approximately one day, slow-moving infections may creep up gradually and go unnoticed. Such malware may not be readily detected by looking for sudden spikes and drops in filename extension counts, because the counts will change gradually day to day. However, by lengthening the time gap period between backup jobs being analyzed, a different pattern may emerge, which the disclosed analysis will detect. Thus, for example, a gradual and persistent encryption attack that progresses relatively slowly every day, will likely be missed by comparing filename extension counts from day to day, as counts may remain “under the radar.” However, by changing its timeframe perspective, the present approach will catch a spike and corresponding drop when comparing today’s backup job to one conducted a week ago or a month ago, for example. Thus, the illustrative system additionally conducts anomaly analysis on longer time gaps between backup jobs in order to account for the possibility of slow-moving infections. More details are given below.

[0218] FIG. 3 is a block diagram that depicts some components of a system 300 that improves ransomware detection accuracy based on machine learning analysis of filename extension patterns, according to an illustrative embodiment. The present figure depicts: client computing device 102-1 comprising file system 310-1 and data agent 342-1; client computing device 102-2 comprising file system 310-2; primary storage device 104-1 comprising primary data 112-1; primary storage device 104-2 comprising

primary data 112-1; secondary storage devices 108 comprising secondary copies 116; data access node 306 comprising media agent 144 and data agent 342-2; and index server 350 hosted by host computing device 351.

[0219] System 300 is a data storage management system that is analogous to system 100 and system 200 herein, and is additionally configured to provide improved ransomware detection accuracy based on analyzing filename extension patterns, including using machine learning for the purpose. System 300 comprises any number of the depicted components, as well as a storage manager 440 (shown in FIG. 4), as well as other components shown in other figures herein.

[0220] Data access node 306 is a computing device, which comprises one or more computer hardware processors and associated computer memory, and which is analogous to secondary storage computing device 106. Illustratively, data access node 306 hosts both media agent 144 and data agent 342-2, which is coupled to file system 310-2. Media agent 144 is configured to receive data from both data agent 342-1 and data agent 342-2 and to generate secondary copies 116 that it stores at secondary storage devices 108, although the system is not limited to this arrangement, and any number of media agents 144 may operate throughout system 300, as shown for example in FIGS. 1D and 1E.

[0221] File system 310 (e.g., 310-1, 310-2) is an embodiment of a file system, which is a way of organizing data such that the data of the file system is made accessible to a computing device, such as client computing device 102-1, client computing device 102-2, respectively. File systems are well known in the art. File system 310 (e.g., 310-1, 310-2) is depicted here as being hosted by a client computing device 102 (e.g., 102-1, 102-2, respectively) to illustrate the communicative pathway taken by each data agent 342 to access primary data 112, via the corresponding file system 310.

[0222] Data agent 342 (e.g., 342-1, 342-2) is a functional component of system 300 that executes on a computing device, such as client computing device 102, data access node 306, or any compute resource that generates data or is in communication with a data source. Data agent 342 is analogous to file system data agent 142A (see FIG. 1E) and is additionally enhanced with new features that perform much of the anomaly detection analysis disclosed herein. Because it is particularly configured to access and interoperate with file systems such as file system 310, data agent 342 may be referred to herein as a “file system data agent.”

[0223] Data agent 342 is illustratively configured to perform at least part of a first backup job of a file system (e.g., 310) that comprises primary data (the media agent 144 performing part of the first backup job as well), wherein the first backup job comprises: in a scan of the file system, count how many of each distinct filename extension were found; store a count of each distinct filename extension found in the scan of the file system in a database configured at the first computing device; cause a copy of the database to be stored at a second computing device; after one or more backup copies of the primary data are generated: identify, based on information in the database, a plurality of first counts, wherein each first count corresponds to a distinct filename extension found in the scan of the file system, identify, based on information in the database, a plurality of second counts, wherein each second count corresponds to a distinct filename extension found in a scan of the file system that was performed in an earlier backup job before the first backup

job, determine a first number of potential renames of filename extensions as compared to the earlier backup job, use a machine learning model to determine whether the first number of potential renames of filename extensions conforms to a pattern generated by the machine learning model for the file system, wherein the pattern is based on a history of filename extension counts taken in past backup jobs of the file system that precede the first backup job, based at least in part on determining that the first number does not conform to the pattern, generate an anomaly alert before completing the first backup job. More details on data agent 342 are given in FIG. 4.

[0224] Index server 350 is a component of system 300 that is generally responsible for storing, sorting, classifying, and serving indexing information gathered from various other components of system 300. Index server 350 may comprise one or more than one, indexing engines or indexing technologies (e.g., ctree, solr, etc.) that operate on different kinds of data. Index server 350 may comprise a plurality of distinct index data structures, each of which may have been generated and maintained by a distinct indexing technology. As described in more detail in FIG. 4, index server 350 comprises a data storage area (“remote file cache”) in which information received from data agent 342 is stored and safeguarded. Index server 350 executes on host computing device 351, which is a computing device that comprises one or more computer hardware processors and associated computer memory. System 300 may comprise more than one index server 350, but only one is shown here for simplicity.

[0225] FIG. 4 is a block diagram that depicts several components of system 300, including some subcomponents of data agent 342, according to an embodiment. The present figure depicts: file system 310, index server 350 comprising a database copy 450 of extensions database 424 received from data agent 342; storage manager 440; database copy 45116; command center user interface 460 executing on command center computing device 461; and data agent 342 comprising filename extension counter 422, extensions database 424, machine learning model 426, filename extension change calculator 428, and anomaly detector 429. The arrows depict logical views of data flows among the depicted components. For example, filename extension counter 422 stores information received via a file system scan of file system 310; extensions database 424 is copied to index server 350; and anomaly detector 429 reports anomalies to storage manager 440 for remedial action and for display at command center user interface 460.

[0226] Filename extension counter 422 is a functional component (e.g., executable software program or routine or process) of data agent 342, which is configured to track the different filename extensions discovered in the file system scan of file system 310, and which is further configured to keep count of how many of each distinct filename extension is found in the file system scan. The file system scan is conducted by data agent 342 during each backup job. filename extension counter 422 stores its counts in extensions database 424.

[0227] Extensions database 424 comprises one or more data structures organized as a database. extensions database 424 comprises information relating to filename extensions gathered by data agent 342 (e.g., using filename extension counter 422), such as counts of filename extensions discovered in file system scans of file system 310. Preferably, for each file system scan, associations are stored in 424 between

the backup job that executed the file system scan, a time frame of the backup job, an identifier of the file system or subset of the file system that was scanned (e.g., represented as a subclient in system 300), and various counts of filename extensions counted by filename extension counter 422, without limitation. The contents of extensions database 424 are copied to a “remote file cache” configured at index server 350, which will safeguard the information in the event that extensions database 424 is corrupted or destroyed. The information in extensions database 424 is used as input for training machine learning model 426, and is also used by filename extension change calculator 428 when analyzing a backup job.

[0228] Machine learning model 426 is a functional component (e.g., executable software program or routine or process) of data agent 342, which is configured to intake a plurality of data points distributed over time and discern a pattern of behavior of the data points. Specifically, machine learning model 426 is configured with statistical algorithms that learn from filename extension distributions found in a series of file system scans conducted by data agent 342 (e.g., file system scans from at least 30 backup jobs). Preferably, machine learning model 426 is updated on an ongoing basis, in each new backup job, with new filename extension counts supplied by filename extension counter 422, although machine learning model 426 is designed to exclude data found to be anomalous (e.g., by anomaly detector 429; see also block 914 in FIG. 9). Thus machine learning model 426 is configured to keep “learning” with new data obtained in each backup job. Once a configured maximum number of backup job has been fed into the machine learning model 426 (e.g., 1,000 backup jobs, without limitation), machine learning model 426 implements a rolling window that comprises the most recent backup jobs.

[0229] Based on these inputs, machine learning model 426 is configured to generalize a pattern of filename extension distributions and fluctuations over time and to predict an filename extension distribution in a next file system scan, i.e., in the file system scan of the current backup job. This prediction will be used by anomaly detector 429 to determine whether a new file system scan shows that the filename extension distributions at the file system conform to the pattern (i.e., agree with the prediction) or diverge from the pattern (i.e., are substantially different from the prediction). According to the illustrative embodiments, machine learning model 426 needs at least 30 filename extension datasets distributed over time, i.e., at least 30 file system scans performed by 30 different backup jobs that target the same file system or subset of the file system (the same subclient). Although machine learning model 426 is not computationally or algorithmically limited to a maximum number of backup jobs or datasets, the illustrative embodiments limit the modeling at 1,000 backup jobs in order to cap the processing load on the data agent’s host computing device, e.g., client computing device 102, data access node 306, etc.

[0230] Machine learning model 426 is able to consume any collection of data and discern a pattern therefrom. In some embodiments, machine learning model 426 may be deployed on a component such as index server 350 to perform anomaly analyses that are outside the scope of and separate from the backup jobs executing at data agent 342. In some embodiments, machine learning model 426 consumes only some, and not the distinct filename extensions found in a file system scan in order to conserve compute

resources, e.g., it will analyze only the “top 10” most frequent filename extensions and will not analyze others.

[0231] Filename extension change calculator 428 is a functional component (e.g., executable software program or routine or process) of data agent 342, which is configured to analyze changes in the filename extension counts that are evident in the current file system scan as compared to an earlier scan or in an earlier job. Thus, filename extension change calculator 428 determines how many “potential renames” occurred between the earlier job and the current job. Potential renames indicates how many files might have had their extensions changed between the earlier job and the current job. Since we are calculating only the counts of each distinct filename extension, filename extension change calculator 428 does not differentiate between an actual extension rename versus a create and delete pair. For example, if x.pdf gets renamed or converted to x.encryl, it counts as one (1) potential rename. But if file a.pdf was deleted and file b.docx was created, that would also count as one (1) potential rename. To calculate the potential renames between an earlier job and the current job, filename extension change calculator 428 takes a sum of filename extensions that increased in count and subtracts the total or net delta of all items counted, as shown in Table 1 below.

TABLE 1

Filename Extension	Count in earlier job	Count in current job	Delta
.pdf	100,000	81,000	−19,000
.docx	50,000	80,000	+30,000
.txt	50,000	60,000	+10,000
Total Files	200,000	221,000	+21,000

[0232] Sum of increased filename extension counts=30,000 (docx)+10,000 (txt)=+40,000.

[0233] Net delta of all counted filename extensions=−19,000+30,000+10,000=+21,000.

[0234] Potential renames=sum of increased minus net delta=40,000−21,000=19,000=8.6% of 221,000 total files counted in the current job.

[0235] Another example appears in Table 2 below.

TABLE 2

Filename Extension	Count in earlier job	Count in current job	Delta
.pdf	100,000	99,000	−1,000
.docx	50,000	51,000	+1,000
.txt	50,000	52,000	+2,000
Total Files	200,000	202,000	+2,000

[0236] Sum of increased filename extension counts=1,000 (docx)+2,000 (txt)=+3,000.

[0237] Net delta of all counted filename extensions=−1,000+1,000+2,000=+2,000.

[0238] Potential renames=sum of increased minus net delta=3,000−2,000=1,000=0.5% of 202,000 total files counted in the current job.

[0239] Another example appears in Table 3 below.

TABLE 3

Filename Extension	Count in earlier job	Count in current job	Delta
.pdf	100,000	10,000	-90,000
.docx	50,000	49,000	-1,000
.txt	50,000	52,000	+2,000
newextension	zero	102,000	+102,000
Total Files	200,000	213,000	+13,000

[0240] Sum of increased filename extension counts=2,000 (txt)+102,000 (newextension)=+104,000.

[0241] Net delta of all counted filename extensions=-90,000-1,000+2,000+102,000=+13,000.

[0242] Potential renames=sum of increased minus net delta=104,000-13,000=91,000=43% of 213,000 total files counted in the current job.

[0243] Thus, the analyses presented in Table 1, Table 2, and Table 3 generate potential renames under different scenarios. The potential renames in each example is based on changed counts among the various filename extensions relative to each other between the earlier job and the current job. The relationships among the changed counts result in very different outcomes in terms of the potential renames identified from job to job. Thus, the number of potential renames of filename extensions calculated in a backup job is based on changed counts among the filename extension counts taken in the backup job as compared to counts taken in a preceding backup job. In some embodiments such as the examples shown above, the number of potential renames of filename extensions is based on a difference between: (i) how many increases in filename extensions were counted in the current backup job compared to counts taken in a preceding job, and (ii) how many additional files were counted in the current backup job compared to how many were counted in the preceding backup job.

[0244] These results, produced by filename extension change calculator 428, may be stored in extensions database 424 for reference and/or for future use. These results will also be used as inputs to anomaly detector 429. See also FIGS. 8 and 9.

[0245] Anomaly detector 429 is a functional component (e.g., executable software program or routine or process) of data agent 342, which is configured to determine whether the filename extension counts in the current backup job being analyzed are anomalous as compared a pattern or other applicable criteria; if they are anomalous, anomaly detector 429 will report an anomaly to storage manager 440. Anomaly detector 429 is configured to apply a different kind of analysis depending on how many backup jobs of the particular subclient have been counted so far. For fewer than a threshold number of jobs (e.g., 30 backup jobs), anomaly detector 429 will use a static “two-point” anomaly detection algorithm described in more detail in FIG. 8. On the other hand, when at least 30 backup jobs have been counted, anomaly detector 429 will leverage the machine learning model 426 to apply a “multi-point” and machine learning algorithm, as described in more detail in FIG. 9. In some embodiments, anomaly detector 429 comprises machine learning model 426. Elements 422, 426, 428, and 429 are shown here as distinct components in order to ease the reader’s understanding of the present disclosure, but in other embodiments they may be combined or differently configured. Additionally, data agent 342 also comprises function-

ality for extracting data for backup from file system 310 and for transmitting the extracted data to media agent 144, but these are not depicted here for the sake of simplicity.

[0246] Database copy 450 of extensions database 424 is a copy of the information in extensions database 424 and is illustratively stored in a “remote file cache” configured at and maintained by index server 350. The remote file cache may be any data storage technology configured at host computing device 351. Illustratively, in order to preserve the information gathered in extensions database 424 immediately or very soon after it is populated during the file system scan, data agent 342 transmits the contents of 424 to index server 350. In some networked configurations, data agent 342 is said to store the contents of 424 at, or to copy extensions database 424 to, the remote file cache at index server 350. In addition, index server 350 is configured to initiate a backup operation that generates a backup copy of database copy 450, e.g., database copy 45116, which is stored on secondary storage media such as secondary storage devices 108. Database copy 45116 is a secondary copy, analogous to secondary copies 116, which is generated by a suitable data agent 142 and media agent 144 and stored on backup media at secondary storage devices 108. Thus, system 300 is configured to aggressively protect the file system scan data collected in extensions database 424 by copying it to database copy 450 and by backing it up promptly to database copy 45116—all part of the current backup job in which the source file system is scanned. See also blocks 508-510 in FIG. 5.

[0247] Storage manager 440 is analogous to storage manager 140, and is additionally enhanced with new features related to improving ransomware detection accuracy based on machine learning analysis of filename extension patterns. Like storage manager 140, storage manager 440 comprises management database 146 and performs tasks that manage system 300. In addition, storage manager 440 is configured to receive anomaly reporting from one or more data agents 342, to initiate remedial actions within system 300, to communicate with external threat systems, and to cause the reported anomaly information to be displayed by command center computing device 461. See also block 618 in FIGS. 6 and 10.

[0248] Command center user interface 460 comprises a user interface that enables users to interact with system 300, e.g., a graphical user interface. Part of command center user interface 460 comprises display generation and rendering features that show graphical representations of filename extension trends, such as the example shown in FIGS. 4A and 4AA. Command center user interface 460 executes on command center computing device 461, which is a computing device comprising one or more computer hardware processors and associated computer memory.

[0249] FIG. 4A depicts an example screenshot of command center user interface 460, which displays an “unusual file activity” screen depicting filename extension trends over several backup jobs. The graphic shows counts, on a per-job basis, of different kinds of filename extensions. In this example graphic, all legitimate filename extension counts (e.g., .ext5 depicted in purple and .xls depicted in yellow) drop to zero in the newest job and the .vcency count (depicted in red) jumps from zero in the preceding job to all files scanned in the file system. This is an example of a sudden spike in one filename extension being accompanied by corresponding catastrophic drops (to zero) in other file-

name extensions. This is the kind of phenomenon that the anomaly analyses disclosed herein (two-point and multi-point) would identify as an anomaly and would report the anomaly to storage manager 440. See also FIGS. 8 and 9.

[0250] FIG. 4AA depicts an example screenshot of command center user interface 460, which displays a different view of the “unusual file activity” screen of FIG. 4A, and depicts filename extension trends over several backup jobs extending further back in time than FIG. 4A. Like FIG. 4A, the graphic shows counts, on a per-job basis, of different kinds of filename extensions. In this example graphic, it appears that the malware infection, depicted by the red data points, was eradicated as evidenced by its counts dropping to zero at job ID 2589760, but then returned to reinfect the file system.

[0251] FIG. 5 is a flowchart that depicts some operations of a method 500, according to an illustrative embodiment. Method 500 is performed by components of system 300 as described in more detail below. At block 502, system 300 initiates a file system backup job, which includes scanning the target file system (subclient), e.g., file system 310. Illustratively, storage manager 440 initiates the backup job at a suitable time according to preferences for protecting the subclient that are stored in management database 146. Accordingly, storage manager 440 instructs the data agent 342 and media agent 144 that are specified in the preferences, to go ahead and perform the backup job, e.g., a full backup, an incremental back, a differential backup. The data agent 342 selected for the backup job initiates a scan of the target file system, e.g., file system 310. Among other aspects, the file system scan inventories the files and directory structure of the file system, whether to identify all files for a full backup or whether to identify changes since a preceding backup for an incremental or differential backup job. Thus, the file system scan inventories all files and directory structures in the file system (or in a targeted subset of the file system, like a given drive or directory, e.g., file system subclient 166). The present approach leverages the file system scan to build and maintain the internal extensions database 424, which tracks how many files in the scanned file system are associated with each filename extension found in the dataset.

[0252] At block 504, as it scans file system 310, data agent 342 (e.g., using filename extension counter 422) saves filename extension counts in local extensions database 424. Thus, whether or not a particular file in file system 310 will be backed up in the current backup job, its filename extension is counted and tracked in extensions database 424. However, before information is saved in extensions database 424, system 300 checks the integrity of extensions database 424. The information in extensions database 424 is critical to the accuracy of the anomaly analysis disclosed herein. Because there may be malware at client computing device 102 that could have corrupted extensions database 424, or because a user of client computing device 102 may be tempted to modify extensions database 424 in some way, system 300 checks the integrity of extensions database 424 before adding fresh new information into it. See more details in FIG. 7. In some alternative embodiments, this integrity check is performed at the beginning of anomaly analysis blocks 512 and 514 (not shown), to ensure that the anomaly analysis is using an uncorrupted extensions database 424.

[0253] At block 506, system 300 (e.g., using data agent 342 and media agent 144) backs up the target file system

dataset into one or more secondary copies 116 and stores the secondary copies 116 at secondary storage devices 108. Preferably, this operation occurs before the anomaly detection analysis is initiated, in order to timely secure the file system data before it can be analyzed. Thus, once the targeted dataset has been securely protected by way of secondary copies 116, the anomaly analysis may proceed. System 300 maintains point-in-time copies of extensions database 424 (or equivalent point-in-time information), because of the importance of the scanned file system information. The importance is manifested in the various copies made from extensions database 424 at various stages and in various places in system 300, e.g., database copy 450, database copy 45116. By associating, within each database, filename extension datasets with a time frame of when they were captured, system 300 is able to reconstruct datasets from various points in time and use those datasets for anomaly analysis using any desired time gap. Thus, for example, if one wanted to conduct an anomaly analysis of year-ago data, one could retrieve such datasets from database copy 450 or from database copy 45116 thereof. The machine learning model 426 is able to consume any collection of data and discern a pattern therefrom. In some embodiments, machine learning model 426 may be deployed on a component such as index server 350 to perform anomaly analyses outside the scope of backup jobs executing at a data agent 342.

[0254] At block 508, system 300 (e.g., using data agent 342) transmits a copy of 424 to index server 350. Illustratively the copy is stored as database copy 450, as described in more detail in FIG. 4. At block 510, system 300 generates a backup copy of database copy 450, and stores the database copy 45116 in backup media, such as at one or more secondary storage devices 108. As described in more detail in FIG. 4, blocks 508 and 510 provide additional protection for the information gathered at block 502 and stored at extensions database 424. This information forms the basis for the subsequent steps needed to discover an anomaly, so securing the information is important. Like the secondary copies 116 of file system 310, database copy 450 and database copy 45116 are generated before the step-wise anomaly detection analysis begins at block 512, in order to ensure that the file system scan data is quickly and thoroughly protected.

[0255] At block 512, system 300 performs a filename extension anomaly analysis, as described in more detail in FIG. 6. This analysis relies on using a certain time gap between filename extension datasets being analyzed, e.g., preferably 24 hours, without limitation. This time gap is in contrast to a substantially longer time gap used for the analysis of block 514. At block 514, system 300 performs a filename extension anomaly analysis, as described in more detail in FIG. 10. This analysis uses a substantially longer time gap between filename extension datasets being analyzed than the time gap used at block 512. This analysis is intended to find slow-moving infections that become evident over longer gaps of time than the sudden spikes detected at block 512. This longer-time-gap analysis may be performed less frequently than block 512, to reduce the processing burden on the host computing device that performs the analysis. However, the frequency of this analysis need not match the longer time gap between datasets. Method 500 ends here.

[0256] At block 516, system 300 takes remedial action if an anomaly was found at block 512 and/or block 514. For example, based on the anomaly alert, storage manager 440 may cause a graphical user interface (e.g., command center user interface 460) to display a file extension trend of the file system that is based on the history of filename extension counts, such as depicted in FIG. 4A and FIG. 4AA. Additionally, storage manager 440 may also initiate certain pro-active operations within system 300, such as: suspending any further backup jobs of file system 310 or suspending backup jobs at client computing device 102; suspending pruning of backup copies that are based on primary data 112 generated at client computing device 102; transmitting information to other systems, such as a threat analysis system that is separate from system 300; etc., without limitation. In addition, storage manager 440 may store information about the anomaly at management database 146, such as identifiers of: the current backup job (e.g., job ID), the subclient being backed up, and client computing device 102 that generated the subclient data; a timestamp; information about the anomaly detection analysis, such as how many backup jobs were used in the history, the time gap, whether the machine learning model was used, what thresholds were exceeded, etc., without limitation.

[0257] FIG. 6 is a flow chart depicting some operations of block 512 of method 500, according to an illustrative embodiment. Block 512 is generally directed at system 300 (e.g., using data agent 342 at client computing device 102) performing a filename extension anomaly detection analysis. As noted at block 504, before block 604 is initiated, an integrity check of extensions database 424 may be performed, as shown in FIG. 7.

[0258] At block 604, system 300 selects a comparison set from extensions database 424. For example, the comparison set may include file system scan data obtained in the current backup job (block 502) and scan data obtained in an earlier backup job, e.g., performed 24 hours earlier and at least 23 hours earlier. Thus, the time gap in the comparison set here is 24 hours, without limitation. The time gap may be set to any value, but since daily backups are a common frequency, a corresponding time gap may be selected. In some systems, backup jobs may not execute exactly 24 hours apart, due to system loads, laptops being turned off, power outages, or other conditions. Thus, at block 604, system 300 identifies an earlier backup job that was performed at least 23 hours before the current job.

[0259] At block 606, system 300 counts, for each distinct filename extension in the selected comparison set, how many of each distinct filename extension are in the dataset; and determines changes in counts between the earlier job and the current job. See, e.g., Table 1, Table 2, and Table 3 and accompanying text. At block 608, system 300 normalizes the changed counts over the time gap between the earlier job time and the current (newer) job time. For example, system 300 determines a 24-hour interval as a convenient normalization interval in the event that the real time gap between the current job and the earlier job was not exactly 24 hours.

[0260] At block 610, which is a decision point, system 300 determines whether the historical data in extensions database 424 represents fewer than a threshold number of backup jobs, e.g., fewer than 30 backup jobs, without limitation. If extensions database 424 comprises information

for at least 30 past backup jobs, control passes to block 614; otherwise control passes to block 612.

[0261] At block 612, system 300 applies a so-called “two-point” anomaly detection algorithm to determine whether the filename extensions counted in the current backup job are anomalous relative to the job history data available. More details are given in FIG. 8.

[0262] At block 614, system 300 applies a so-called “multi-point and machine learning” anomaly detection algorithm to determine whether the filename extensions counted in the current backup job are anomalous relative to the machine-learned patterns of the historical data. More details are given in FIG. 9.

[0263] At block 616, which is a decision point, system 300 determines whether the anomaly detection analysis (block 612 or 614) declared an anomaly for the current backup job. If no anomaly was declared, block 512 ends here; if an anomaly was declared, control passes to block 618. At block 618, data agent 342 reports the anomaly to storage manager 440, which will take remedial actions at block 516. Block 512 ends here.

[0264] FIG. 7 is a flow chart depicting some operations of block 504 of method 500, according to an illustrative embodiment. Preferably, before information is saved in extensions database 424, system 300 performs an integrity check of extensions database 424. At block 702, system 300 (e.g., using data agent 342) compares a hash value of extensions database 424 with a stored hash value. In a preceding round of block 504, or whenever extensions database 424 is changed or updated, data agent 342 generated a hash value of extensions database 424 and securely stored the hash value in one or more locations, e.g., at index server 350, at storage manager 440, at an authentication server (not shown), etc. The hash value is retrieved at block 702 and compared to a newly calculated hash value of extensions database 424 in its current form, as stored at client computing device 102.

[0265] At block 704, system 300 (e.g., using data agent 342) determines whether the two hash values being compared are the same or match. If so, extensions database 424 as currently available in client computing device 102 is considered to be correct and control passes to block 710. Otherwise, if the two hash values do not match, extensions database 424 as currently available in client computing device 102 is considered to be incorrect (out of date, corrupted, missing, etc.) and a correct version of it needs to be retrieved. Accordingly, control passes to block 706.

[0266] At block 706, system 300 (e.g., using data agent 342) attempts to retrieve database copy 450 from index server 350. If the hash values match as described at block 704 as between the stored hash value and database copy 450, database copy 450 is retrieved from index server 350 and installed as the current extensions database 424 at client computing device 102. At this point, extensions database 424 as currently available in client computing device 102 is considered to be correct and control passes to block 710. However, if the hash values do not match as between the stored hash value and database copy 450, or if database copy 450 is not available at index server 350, control passes to block 708. At block 708, system 300 (e.g., using data agent 342) data agent 342 initiates a restore operation in which database copy 45116 is restored from secondary storage devices 108 to client computing device 102. Presumably, the hash values will match as described at block 704 as between

the stored hash value and extensions database 424 as restored from database copy 45116; if so, extensions database 424 is considered to be correct and control passes to block 710. If there is still a mismatch between the hash values, an alarm needs to be raised, and the subsequent anomaly detection operations needs to be suspended, because extensions database 424 cannot be reliably used in the current round; accordingly control passes to block 516, wherein storage manager 440 will take suitable remedial actions.

[0267] At block 710, having successfully checked and/or restored the integrity of extensions database 424 at data agent 342, data agent 342 (e.g., using filename extension counter 422) saves filename extension counts obtained in the current backup job in local extensions database 424. Thus, whether or not a particular file in file system 310 will be backed up in the current backup job, its filename extension is counted and tracked in extensions database 424. Block 504 ends here.

[0268] FIG. 8 is a flow chart depicting some operations of block 612 of method 500, according to an illustrative embodiment. At block 612, which is reached when job history comprises fewer than a threshold number of backup jobs (e.g., 30 backup jobs), system 300 (e.g., using data agent 342) uses a so-called “two-point” anomaly detection algorithm to determine whether the filename extensions counted in the current backup job are anomalous relative to the job history data available. At block 802, system 300 analyzes the changed counts determined at block 606 and normalized at block 608 to calculate potential renames since the earlier job. The potential renames discerned in the current backup job are calculated according to examples in Table 1, Table 2, and/or Table 3 and accompanying text.

[0269] At block 804, system 300 applies a so-called “static model” to the current count of potential renames. The static model includes static parameters or metrics and pre-defined thresholds therefor. Examples of static parameters include, without limitation: potential renames count as a percentage of the total number of filename extensions counted in the file system scan of the current backup job; potential renames count increase percentage versus a count of potential renames in the earlier job to which the present job is being compared; a number of potential renames calculated for the current backup job; etc. Examples of pre-defined thresholds include, without limitation: potential renames count equals at least 10% of the total number of filename extensions counted in the file system scan of the current backup job; potential renames count is at least 10% more than a corresponding potential renames count of the earlier job to which the present job is being compared; potential renames count is at least N (e.g., 1,000) calculated for the current backup job; etc. The static model’s thresholds are configurable, and each implementation of system 300 may be configured with different pre-defined threshold values that the system’s owner or subscriber considers suitable for their data protection environment and for their own acceptable levels of risk.

[0270] At block 806, which is a decision point, system 300 determines whether the current dataset (e.g., the various static parameters or metrics calculated at block 804) exceed their corresponding pre-defined thresholds. If not, i.e., none of the pre-defined thresholds are passed, control passes to block 810. On the other hand, if one or more of the pre-defined thresholds at block 804 are exceeded, control passes to block 808. At block 808, system 300 declares an

anomaly and block 612 ends here. Declaring an anomaly may include transmitting an anomaly indication to another process within data agent 342; setting a flag in extensions database 424; and/or establishing another indicator of anomaly that data agent 342 will use at block 616, which follows block 612. At block 810, system 300 refrains from declaring an anomaly and block 612 ends here.

[0271] FIG. 9 is a flow chart depicting some operations of block 614 of method 500, according to an illustrative embodiment. At block 614, which is reached when job history data comprises at least a threshold number of backup jobs (e.g., 30 backup jobs), system 300 (e.g., using data agent 342) applies a so-called “multi-point and machine learning” anomaly detection algorithm to determine whether the filename extensions counted in the current backup job are anomalous relative to patterns of the historical data. At block 902, system 300 analyzes the changed counts determined at block 606 and normalized at block 608 to calculate potential renames since the earlier job, and other metrics, as described in regard to Table 1, Table 2, and Table 3.

[0272] At block 904, system 300 applies a machine learning model, e.g., machine learning model 426, to evaluate the potential renames and changed filename extension counts relative to a pattern of changes over time. Here, machine learning model 426 determines a pattern of changes in filename extension counts over the time frame of backup jobs with which machine learning model 426 has been trained. Thus, for example, if machine learning model 426 uses filename extension counts collected from 500 previous jobs, machine learning model 426 will determine a pattern of filename extension changes manifested over these 500 jobs. For example, machine learning model 426 may determine that the pattern includes an increase of approx. 1,000 files in the Total Files count every Friday, but few Total Files increases on other days. This may be the result of a weekly data download that occurs at client computing device 102. For example, machine learning model 426 may determine that the pattern includes approximately the same ratio among the .pdf, .docx, and .txt filename extensions as the figures shown in Table 1 for the earlier job. For example, machine learning model 426 may determine that the pattern includes no more than 800 potential renames between backup jobs.

[0273] Machine learning model 426 can now determine whether the filename extension counts and potential renames of the current backup job deviate substantially from the pattern defined by machine learning model 426 or whether they conform to the pattern. Acceptable margins for what is considered conforming to the pattern are configurable in the machine learning model and may vary among customers, based on their level of acceptable risk. For example, machine learning model 426 may evaluate the data in Table 1 and determine that it sees an anomaly in the current job, because the potential renames count far exceeds the pattern of 800 or fewer modeled from the historical data. See also Table 2 and Table 3 for other examples.

[0274] At block 906, which is a decision point, system 300 determines whether machine learning model 426 indicates an anomaly according to the model’s long-term historical data pattern. Does the number of potential renames conform to the pattern calculated by the machine learning model? If no anomaly is reported by machine learning model 426, control passes to block 916, and block 614 ends here. On the other hand, if machine learning model 426 reports an

anomaly, i.e., reports that the potential renames do not conform to the pattern or fall outside the acceptable margins of the pattern, system 300 may (preferably) proceed to block 908 to perform an additional verification. This additional verification comprises a set of follow-up steps to determine whether the anomaly is sufficiently severe to be reported. Illustratively, if the anomaly or divergence found relative to the machine learning model falls within the pre-defined thresholds that were discussed in more detail in FIG. 8, system 300 will refrain from reporting the anomaly. This additional verification need not be used in all embodiments; however, the additional verification will reduce the chances of issuing false positive alarms when patterns change legitimately due to a recent change. In some embodiments, the static parameters and their corresponding pre-defined thresholds will be different here in the additional verification than those used in FIG. 8, because they are configurable.

[0275] At block 908, system 300 generates a shorter-term “recent” dataset that eschews the longer-term and focuses instead on a more recent period of time, which is analyzed here with one or more static parameters, such as those described at block 804. Thus, for example, rather than encompassing the last 500 backup jobs, as trained in the machine learning model 426, block 908 instead uses only the last week, or only the last 15 days, of backup jobs. The recent dataset is then analyzed using static parameters. Examples of static parameters include, without limitation: a count of potential renames as a percentage of the total number of filename extensions counted in the file system scan of the current backup job; potential renames count increased percentage versus a count of potential renames in the earlier job to which the present job is being compared; a number of potential renames calculated for the current backup job; etc. Each static parameter has an associated (and configurable) pre-defined threshold, which is calculated at block 910. At block 910, system 300 applies the pre-defined threshold values to the static parameters calculated at block 908. For example, the threshold figures used in block 804 may be used here as well.

[0276] At block 912, which is a decision point analogous to block 806, system 300 determines whether the pre-defined thresholds are exceeded. If one or more of the pre-defined thresholds at block 910 are exceeded by the potential renames and other static parameters calculated at block 908, control passes to block 914; otherwise control passes to block 916.

[0277] At block 914, which is similar to block 808, system 300 declares an anomaly and block 614 ends here. Declaring an anomaly may include transmitting an anomaly indication to another process within data agent 342; setting a flag in extensions database 424; and/or establishing another indicator of anomaly that data agent 342 will use at block 616, which follows block 614. At block 916, which is similar to block 810, system 300 refrains from declaring an anomaly and block 614 ends here.

[0278] FIG. 10 is a flow chart depicting some operations of block 514 of method 500, according to an illustrative embodiment. At block 514, system 300 performs a filename extension anomaly analysis that uses a substantially longer time gap between the filename extension datasets being analyzed than the time gap used at block 512. For example, rather than analyzing data collected in daily backup jobs, the present analysis picks backup jobs that are one week apart, or two weeks apart. Spikes and drops that do not conform to

the one-week or two-week pattern will be identified this way. This analysis is intended to find slow-moving infections that become evident over longer gaps of time than the sudden spikes detected at block 512. Thus, the operations of block 514 are similar and almost identical to those of block 512, with a few differences that account for the longer time gap. Thus, at block 1002, system 300 (e.g., using data agent 342) selects a comparison set that has a longer time gap between the current job and the earlier job, e.g., 10 days, rather than 24 hours as disclosed at block 604. The longer time gap period is configurable, and is preferably in the 1-2-week range. Block 1002 is followed by block 606 and 608, followed by block 1004. Block 1004, which is a decision point, is similar to block 610, but here the history is made up of data from backup jobs that are farther apart in time, i.e., have the longer time gap between them. After this, blocks 612-618 follow and operate as described in the preceding figures. Thus, an anomaly may be found in regard to the longer time-gapped backup jobs, which might indicate the presence of a slower-moving malware that does not show up on a day-to-day analysis. Block 514 ends here and control passes back to block 516.

[0279] In regard to the figures described herein, other embodiments are possible within the scope of the present invention, such that the above-recited components, steps, blocks, operations, messages, requests, queries, and/or instructions are differently arranged, sequenced, sub-divided, organized, and/or combined. In some embodiments, a different component may initiate or execute a given operation.

EXAMPLE EMBODIMENTS

[0280] Some example enumerated embodiments of the present invention may be in the form of computer-implemented methods, systems, and/or non-transitory computer-readable media, without limitation.

[0281] In some aspects, the techniques described herein relate to a system including: a first computing device including one or more hardware processors and computer memory carrying computer programming instructions that, when executed by the one or more hardware processors, configure the first computing device to perform a first backup job of a file system that includes primary data, wherein the first backup job includes: in a scan of the file system, count how many of each distinct filename extension were found; store a count of each distinct filename extension found in the scan of the file system in a database configured at the first computing device; cause a copy of the database to be stored at a second computing device; after one or more backup copies of the primary data are generated: identify, based on information in the database, a plurality of first counts, wherein each first count corresponds to a distinct filename extension found in the scan of the file system, identify, based on information in the database, a plurality of second counts, wherein each second count corresponds to a distinct filename extension found in a scan of the file system that was performed in an earlier backup job before the first backup job, determine a first number of potential renames of filename extensions as compared to the earlier backup job, use a machine learning model to determine whether the first number of potential renames of filename extensions conforms to a pattern generated by the machine learning model for the file system, wherein the pattern is based on a history of filename extension counts taken in past backup jobs of the

file system that precede the first backup job, based at least in part on determining that the first number does not conform to the pattern, generate an anomaly alert before completing the first backup job.

[0282] In some aspects, the techniques described herein relate to a system, wherein the first number of potential renames of filename extensions is based on changed counts among the plurality of first counts as compared to the plurality of second counts. In some aspects, the techniques described herein relate to a system, wherein the first number of potential renames of filename extensions is based on a difference between: (i) how many increases in filename extensions were counted in the plurality of first counts compared to the plurality of second counts, and (ii) how many additional files were counted in the plurality of first counts compared to the plurality of second counts. In some aspects, the techniques described herein relate to a system, wherein the first backup job further includes: based on determining that the first number does not conform to the pattern, determine whether the first number is greater than a pre-defined threshold percentage of the plurality of first counts, and based on determining that the first number: does not conform to the pattern and is greater than the pre-defined threshold percentage of the plurality of first counts, generate the anomaly alert before completing the first backup job. In some aspects, the techniques described herein relate to a system, wherein the anomaly alert is further based on a determination that the first number of potential renames of filename extensions is also greater than a threshold number of potential renames. In some aspects, the techniques described herein relate to a system, wherein the first number of potential renames is normalized according to a time gap between the first backup job and the earlier backup job.

[0283] In some aspects, the techniques described herein relate to a system further including the second computing device, which includes one or more second hardware processors and computer memory carrying computer programming instructions that, when executed by the one or more second hardware processors, configure the second computing device to: receive the copy of the database from the first computing device; store the copy of the database at a data storage area configured at the second computing device; and cause a backup copy of the copy of the database to be generated and stored at a secondary storage that is distinct from the second computing device and from the first computing device. In some aspects, the techniques described herein relate to a system, wherein the first backup job further includes: check an integrity of the database before storing filename extension counts in the database; and based on failing the check of the integrity of the database, cause the backup copy of the copy of the database to be restored from the secondary storage to the first computing device. In some aspects, the techniques described herein relate to a system, wherein the first backup job further includes: check an integrity of the database before storing filename extension counts in the database. In some aspects, the techniques described herein relate to a system, wherein the first backup job further includes: based on failing the check of the integrity of the database, retrieve the copy of the database from the second computing device.

[0284] In some aspects, the techniques described herein relate to a system, wherein the first computing device is further configured to: transmit the anomaly alert to a third computing device, which includes one or more third hard-

ware processors and computer memory carrying computer programming instructions that, when executed by the one or more third hardware processors, configure the third computing device to: manage backup jobs in the system, including the first backup job; based on the anomaly alert, cause a graphical user interface to display a file extension trend of the file system that is based on the history of filename extension counts; and manage a remedial action responsive to the anomaly alert. In some aspects, the techniques described herein relate to a system, wherein the first computing device is configured to execute a data agent that performs at least some of the first backup job. In some aspects, the techniques described herein relate to a system, wherein the first computing device is configured to execute a data agent and a media agent that collectively perform the first backup job, wherein the data agent is configured to determine the first number and to use the machine learning model, and wherein the media agent is configured to store the one or more backup copies of the primary data, in a backup format, at a secondary storage that is distinct from the first computing device.

[0285] In some other aspects, the techniques described herein relate to a system including: a first computing device including one or more hardware processors and computer memory carrying computer programming instructions that, when executed by the one or more hardware processors, configure the first computing device to perform a first backup job of a file system that includes primary data, wherein the first backup job includes: count how many of each distinct filename extension are found in a scan of the file system; store a count of each distinct filename extension found in the scan of the file system in a database configured at the first computing device; cause a copy of the database to be stored at a second computing device; after one or more backup copies of the primary data are generated: identify, based on information in the database, a plurality of first counts, wherein each first count corresponds to a distinct filename extension found in the scan of the file system, identify, based on information in the database, a plurality of second counts, wherein each second count corresponds to a distinct filename extension found in a scan of the file system that was performed in an earlier backup job before the first backup job, determine a first number of potential renames of filename extensions as compared to the earlier backup job, wherein the first number is based on a difference between: (i) how many increases in filename extensions were counted in the plurality of first counts compared to the plurality of second counts, and (ii) how many additional files were counted in the plurality of first counts compared to the plurality of second counts, use a machine learning model to determine that the first number of potential renames of filename extensions does not conform to a pattern generated by the machine learning model for the file system, wherein the pattern is based on a history of filename extension counts taken in past backup jobs of the file system that precede the first backup job, and generate an anomaly alert before completing the first backup job.

[0286] In some aspects, the techniques described herein relate to a system, wherein the first backup job further includes: based on determining that the first number does not conform to the pattern, determine whether the first number is greater than a pre-defined threshold percentage of the plurality of first counts, and based on determining that the first number: does not conform to the pattern and is greater

than the pre-defined threshold percentage of the plurality of first counts, generate the anomaly alert before completing the first backup job. In some aspects, the techniques described herein relate to a system further including the second computing device, which includes one or more second hardware processors and computer memory carrying computer programming instructions that, when executed by the one or more second hardware processors, configure the second computing device to: receive the copy of the database from the first computing device; store the copy of the database at a data storage area configured at the second computing device; and cause a backup copy of the copy of the database to be generated and stored at a secondary storage that is distinct from the second computing device and from the first computing device. In some aspects, the techniques described herein relate to a system, wherein the first backup job further includes: perform an integrity check of the database before storing filename extension counts in the database; and based on a failure of the integrity check, cause the backup copy of the copy of the database to be restored from the secondary storage to the first computing device. In some aspects, the techniques described herein relate to a system, wherein the first backup job further includes: perform an integrity check of the database before storing filename extension counts in the database; and based on a failure of the integrity check, retrieve the copy of the database from the second computing device.

[0287] In some aspects, the techniques described herein relate to a system, wherein the first computing device is further configured to: transmit the anomaly alert to a third computing device, which includes one or more third hardware processors and computer memory carrying computer programming instructions that, when executed by the one or more third hardware processors, configure the third computing device to: manage backup jobs in the system, including the first backup job; based on the anomaly alert, cause a graphical user interface to display a file extension trend of the file system that is based on the history of filename extension counts; and manage a remedial action responsive to the anomaly alert. In some aspects, the techniques described herein relate to a system, wherein the first computing device is configured to execute a data agent and a media agent that collectively perform the first backup job, wherein the data agent is configured to determine the first number and to use the machine learning model, and wherein the media agent is configured to store the one or more backup copies of the primary data, in a backup format, at a secondary storage that is distinct from the first computing device. In some aspects, the techniques described herein relate to a system, wherein the pattern is based on a limited number of filename extension types among the history of filename extension counts taken in the past backup jobs, wherein the limited number is fewer than all distinct filename extensions found in the past backup jobs.

[0288] In other embodiments according to the present invention, a system or systems operates according to one or more of the methods and/or computer-readable media recited in the preceding paragraphs. In yet other embodiments, a method or methods operates according to one or more of the systems and/or computer-readable media recited in the preceding paragraphs. In yet more embodiments, a non-transitory computer-readable medium or media causes one or more computing devices having one or more proces-

sors and computer-readable memory to operate according to one or more of the systems and/or methods recited in the preceding paragraphs.

Terminology

[0289] Conditional language, such as, among others, “can,” “could,” “might,” or “may,” unless specifically stated otherwise, or otherwise understood within the context as used, is generally intended to convey that certain embodiments include, while other embodiments do not include, certain features, elements and/or steps. Thus, such conditional language is not generally intended to imply that features, elements and/or steps are in any way required for one or more embodiments or that one or more embodiments necessarily include logic for deciding, with or without user input or prompting, whether these features, elements and/or steps are included or are to be performed in any particular embodiment. Unless the context clearly requires otherwise, throughout the description and the claims, the words “comprise,” “comprising,” and the like are to be construed in an inclusive sense, as opposed to an exclusive or exhaustive sense, i.e., in the sense of “including, but not limited to.” As used herein, the terms “connected,” “coupled,” or any variant thereof means any connection or coupling, either direct or indirect, between two or more elements; the coupling or connection between the elements can be physical, logical, or a combination thereof. Additionally, the words “herein,” “above,” “below,” and words of similar import, when used in this application, refer to this application as a whole and not to any particular portions of this application. Where the context permits, words using the singular or plural number may also include the plural or singular number respectively. The word “or” in reference to a list of two or more items, covers all of the following interpretations of the word: any one of the items in the list, all of the items in the list, and any combination of the items in the list. Likewise the term “and/or” in reference to a list of two or more items, covers all of the following interpretations of the word: any one of the items in the list, all of the items in the list, and any combination of the items in the list.

[0290] In some embodiments, certain operations, acts, events, or functions of any of the algorithms described herein can be performed in a different sequence, can be added, merged, or left out altogether (e.g., not all are necessary for the practice of the algorithms). In certain embodiments, operations, acts, functions, or events can be performed concurrently, e.g., through multi-threaded processing, interrupt processing, or multiple processors or processor cores or on other parallel architectures, rather than sequentially. Systems and modules described herein may comprise software, firmware, hardware, or any combination (s) of software, firmware, or hardware suitable for the purposes described. Software and other modules may reside and execute on servers, workstations, personal computers, computerized tablets, PDAs, and other computing devices suitable for the purposes described herein. Software and other modules may be accessible via local computer memory, via a network, via a browser, or via other means suitable for the purposes described herein. Data structures described herein may comprise computer files, variables, programming arrays, programming structures, or any electronic information storage schemes or methods, or any combinations thereof, suitable for the purposes described herein. User interface elements described herein may com-

prise elements from graphical user interfaces, interactive voice response, command line interfaces, and other suitable interfaces.

[0291] Further, processing of the various components of the illustrated systems can be distributed across multiple machines, networks, and other computing resources. Two or more components of a system can be combined into fewer components. Various components of the illustrated systems can be implemented in one or more virtual machines, rather than in dedicated computer hardware systems and/or computing devices. Likewise, the data repositories shown can represent physical and/or logical data storage, including, e.g., storage area networks or other distributed storage systems. Moreover, in some embodiments the connections between the components shown represent possible paths of data flow, rather than actual connections between hardware. While some examples of possible connections are shown, any of the subset of the components shown can communicate with any other subset of components in various implementations. Embodiments are also described above with reference to flow chart illustrations and/or block diagrams of methods, apparatus (systems) and computer program products. Each block of the flow chart illustrations and/or block diagrams, and combinations of blocks in the flow chart illustrations and/or block diagrams, may be implemented by computer program instructions. Such instructions may be provided to a processor of a general purpose computer, special purpose computer, specially-equipped computer (e.g., comprising a high-performance database server, a graphics subsystem, etc.) or other programmable data processing apparatus to produce a machine, such that the instructions, which execute via the processor(s) of the computer or other programmable data processing apparatus, create means for implementing the acts specified in the flow chart and/or block diagram block or blocks. These computer program instructions may also be stored in a non-transitory computer-readable memory that can direct a computer or other programmable data processing apparatus to operate in a particular manner, such that the instructions stored in the computer-readable memory produce an article of manufacture including instruction means which implement the acts specified in the flow chart and/or block diagram block or blocks. The computer program instructions may also be loaded to a computing device or other programmable data processing apparatus to cause operations to be performed on the computing device or other programmable apparatus to produce a computer implemented process such that the instructions which execute on the computing device or other programmable apparatus provide steps for implementing the acts specified in the flow chart and/or block diagram block or blocks.

[0292] Any patents and applications and other references noted above, including any that may be listed in accompanying filing papers, are incorporated herein by reference. Aspects of the invention can be modified, if necessary, to employ the systems, functions, and concepts of the various references described above to provide yet further implementations of the invention. These and other changes can be made to the invention in light of the above Detailed Description of the invention, and describes the best mode contemplated, no matter how detailed the above appears in text, the invention can be practiced in many ways. Details of the system may vary considerably in its specific implementa-

tion, while still being encompassed by the invention disclosed herein. As noted above, particular terminology used when describing certain features or aspects of the invention should not be taken to imply that the terminology is being redefined herein to be restricted to any specific characteristics, features, or aspects of the invention with which that terminology is associated. In general, the terms used in the following claims should not be construed to limit the invention to the specific examples disclosed in the specification, unless the above Detailed Description section explicitly defines such terms. Accordingly, the actual scope of the invention encompasses not only the disclosed examples, but also all equivalent ways of practicing or implementing the invention under the claims.

[0293] To reduce the number of claims, certain aspects of the invention are presented below in certain claim forms, but the applicant contemplates other aspects of the invention in any number of claim forms. For example, while only one aspect of the invention is recited as a means-plus-function claim under 35 U.S.C. sec. 112 (f) (AIA), other aspects may likewise be embodied as a means-plus-function claim, or in other forms, such as being embodied in a computer-readable medium. Any claims intended to be treated under 35 U.S.C. § 112 (f) will begin with the words “means for,” but use of the term “for” in any other context is not intended to invoke treatment under 35 U.S.C. § 112 (f). Accordingly, the applicant reserves the right to pursue additional claims after filing this application, in either this application or in a continuing application.

What is claimed is:

1. A system comprising:

a first computing device comprising one or more hardware processors and computer memory carrying computer programming instructions that, when executed by the one or more hardware processors, configure the first computing device to perform a first backup job of a file system that comprises primary data, wherein the first backup job comprises:

in a scan of the file system, count how many of each distinct filename extension were found;

store a count of each distinct filename extension found in the scan of the file system in a database configured at the first computing device;

cause a copy of the database to be stored at a second computing device;

after one or more backup copies of the primary data are generated:

identify, based on information in the database, a plurality of first counts, wherein each first count corresponds to a distinct filename extension found in the scan of the file system,

identify, based on information in the database, a plurality of second counts, wherein each second count corresponds to a distinct filename extension found in a scan of the file system that was performed in an earlier backup job before the first backup job,

determine a first number of potential renames of filename extensions as compared to the earlier backup job,

use a machine learning model to determine whether the first number of potential renames of filename extensions conforms to a pattern generated by the machine learning model for the file system, wherein the pattern

is based on a history of filename extension counts taken in past backup jobs of the file system that precede the first backup job,

based at least in part on determining that the first number does not conform to the pattern, generate an anomaly alert before completing the first backup job.

2. The system of claim 1, wherein the first number of potential renames of filename extensions is based on changed counts among the plurality of first counts as compared to the plurality of second counts.

3. The system of claim 1, wherein the first number of potential renames of filename extensions is based on a difference between: (i) how many increases in filename extensions were counted in the plurality of first counts compared to the plurality of second counts, and (ii) how many additional files were counted in the plurality of first counts compared to the plurality of second counts.

4. The system of claim 1, wherein the first backup job further comprises:

- based on determining that the first number does not conform to the pattern, determine whether the first number is greater than a pre-defined threshold percentage of the plurality of first counts, and
- based on determining that the first number: does not conform to the pattern and is greater than the pre-defined threshold percentage of the plurality of first counts, generate the anomaly alert before completing the first backup job.

5. The system of claim 4, wherein the anomaly alert is further based on a determination that the first number of potential renames of filename extensions is also greater than a threshold number of potential renames.

6. The system of claim 1, wherein the first number of potential renames is normalized according to a time gap between the first backup job and the earlier backup job.

7. The system of claim 1 further comprising the second computing device, which comprises one or more second hardware processors and computer memory carrying computer programming instructions that, when executed by the one or more second hardware processors, configure the second computing device to:

- receive the copy of the database from the first computing device;
- store the copy of the database at a data storage area configured at the second computing device; and
- cause a backup copy of the copy of the database to be generated and stored at a secondary storage that is distinct from the second computing device and from the first computing device.

8. The system of claim 7, wherein the first backup job further comprises:

- check an integrity of the database before storing filename extension counts in the database; and
- based on failing the check of the integrity of the database, cause the backup copy of the copy of the database to be restored from the secondary storage to the first computing device.

9. The system of claim 1, wherein the first backup job further comprises:

- check an integrity of the database before storing filename extension counts in the database.

10. The system of claim 9, wherein the first backup job further comprises:

based on failing the check of the integrity of the database, retrieve the copy of the database from the second computing device.

11. The system of claim 1, wherein the first computing device is further configured to:

- transmit the anomaly alert to a third computing device, which comprises one or more third hardware processors and computer memory carrying computer programming instructions that, when executed by the one or more third hardware processors, configure the third computing device to:

- manage backup jobs in the system, including the first backup job;

- based on the anomaly alert, cause a graphical user interface to display a file extension trend of the file system that is based on the history of filename extension counts; and

- manage a remedial action responsive to the anomaly alert.

12. The system of claim 1, wherein the first computing device is configured to execute a data agent that performs at least some of the first backup job.

13. The system of claim 1, wherein the first computing device is configured to execute a data agent and a media agent that collectively perform the first backup job,

- wherein the data agent is configured to determine the first number and to use the machine learning model, and

- wherein the media agent is configured to store the one or more backup copies of the primary data, in a backup format, at a secondary storage that is distinct from the first computing device.

14. A system comprising:

- a first computing device comprising one or more hardware processors and computer memory carrying computer programming instructions that, when executed by the one or more hardware processors, configure the first computing device to perform a first backup job of a file system that comprises primary data, wherein the first backup job comprises:

- count how many of each distinct filename extension are found in a scan of the file system;

- store a count of each distinct filename extension found in the scan of the file system in a database configured at the first computing device;

- cause a copy of the database to be stored at a second computing device;

- after one or more backup copies of the primary data are generated:

- identify, based on information in the database, a plurality of first counts, wherein each first count corresponds to a distinct filename extension found in the scan of the file system,

- identify, based on information in the database, a plurality of second counts, wherein each second count corresponds to a distinct filename extension found in a scan of the file system that was performed in an earlier backup job before the first backup job,

- determine a first number of potential renames of filename extensions as compared to the earlier backup job, wherein the first number is based on a difference between: (i) how many increases in filename extensions were counted in the plurality of first counts compared to the plurality of second counts, and (ii) how many additional files were counted in the plurality of first counts compared to the plurality of second counts,

use a machine learning model to determine that the first number of potential renames of filename extensions does not conform to a pattern generated by the machine learning model for the file system, wherein the pattern is based on a history of filename extension counts taken in past backup jobs of the file system that precede the first backup job, and

generate an anomaly alert before completing the first backup job.

15. The system of claim **14**, wherein the first backup job further comprises:

based on determining that the first number does not conform to the pattern, determine whether the first number is greater than a pre-defined threshold percentage of the plurality of first counts, and

based on determining that the first number: does not conform to the pattern and is greater than the pre-defined threshold percentage of the plurality of first counts, generate the anomaly alert before completing the first backup job.

16. The system of claim **14** further comprising the second computing device, which comprises one or more second hardware processors and computer memory carrying computer programming instructions that, when executed by the one or more second hardware processors, configure the second computing device to:

receive the copy of the database from the first computing device;

store the copy of the database at a data storage area configured at the second computing device; and

cause a backup copy of the copy of the database to be generated and stored at a secondary storage that is distinct from the second computing device and from the first computing device.

17. The system of claim **16**, wherein the first backup job further comprises:

perform an integrity check of the database before storing filename extension counts in the database; and
based on a failure of the integrity check, cause the backup copy of the copy of the database to be restored from the secondary storage to the first computing device.

18. The system of claim **14**, wherein the first backup job further comprises:

perform an integrity check of the database before storing filename extension counts in the database; and
based on a failure of the integrity check, retrieve the copy of the database from the second computing device.

19. The system of claim **14**, wherein the first computing device is further configured to:

transmit the anomaly alert to a third computing device, which comprises one or more third hardware processors and computer memory carrying computer programming instructions that, when executed by the one or more third hardware processors, configure the third computing device to:

manage backup jobs in the system, including the first backup job;

based on the anomaly alert, cause a graphical user interface to display a file extension trend of the file system that is based on the history of filename extension counts; and

manage a remedial action responsive to the anomaly alert.

20. The system of claim **14**, wherein the first computing device is configured to execute a data agent and a media agent that collectively perform the first backup job,

wherein the data agent is configured to determine the first number and to use the machine learning model, and
wherein the media agent is configured to store the one or more backup copies of the primary data, in a backup format, at a secondary storage that is distinct from the first computing device.

* * * * *